

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 7_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 47.5

Section 1 : Coding

1. Problem Statement

Jake is learning about binary search trees(BST) and their operations. He wants to implement a program that can delete a node from a BST based on the given key value and print the remaining nodes in an in-order traversal.

Assist Jake in the program.

Input Format

The first line of input consists of an integer n , representing the number of elements in BST.

The second line consists of n space-separated integers, representing the elements of the tree.

The third line consists of an integer x, representing the key value of the node to be deleted.

Output Format

The first line of output prints "Before deletion: " followed by the in-order traversal of the initial BST.

The second line prints "After deletion: " followed by the in-order traversal after the deletion of the key value.

If the key value is not present in the BST, print the original tree as it is.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

8 6 4 3 1

4

Output: Before deletion: 1 3 4 6 8

After deletion: 1 3 6 8

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Definition for a binary tree node.
```

```
struct TreeNode {  
    int val;  
    struct TreeNode *left;  
    struct TreeNode *right;  
};
```

```
// Function to create a new node
```

```
struct TreeNode* newNode(int val) {  
    struct TreeNode* temp = (struct TreeNode*)malloc(sizeof(struct TreeNode));  
    temp->val = val;  
    temp->left = temp->right = NULL;  
    return temp;  
}
```

// Function to insert a node into BST

```
struct TreeNode* insert(struct TreeNode* root, int val) {  
    if (root == NULL) {  
        return newNode(val);  
    }  
    if (val < root->val) {  
        root->left = insert(root->left, val);  
    } else if (val > root->val) {  
        root->right = insert(root->right, val);  
    }  
    return root;  
}
```

// Function to find the in-order successor (smallest in the right subtree)

```
struct TreeNode* findMin(struct TreeNode* node) {  
    struct TreeNode* current = node;  
    while (current && current->left != NULL) {  
        current = current->left;  
    }  
    return current;  
}
```

// Function to delete a node from BST

```
struct TreeNode* deleteNode(struct TreeNode* root, int key) {  
    if (root == NULL) {  
        return root;  
    }
```

// If the key to be deleted is smaller than the root's key,

// then it lies in the left subtree

```
if (key < root->val) {  
    root->left = deleteNode(root->left, key);  
}
```

// If the key to be deleted is greater than the root's key,

// then it lies in the right subtree

```
else if (key > root->val) {  
    root->right = deleteNode(root->right, key);  
}
```

// If key is same as root's key, then this is the node to be deleted

```
else {  
    // node with only one child or no child
```

```

    if (root->left == NULL) {
        struct TreeNode* temp = root->right;
        free(root);
        return temp;
    } else if (root->right == NULL) {
        struct TreeNode* temp = root->left;
        free(root);
        return temp;
    }

    // node with two children: Get the inorder successor (smallest
    // in the right subtree)
    struct TreeNode* temp = findMin(root->right);

    // Copy the inorder successor's content to this node
    root->val = temp->val;

    // Delete the inorder successor
    root->right = deleteNode(root->right, temp->val);
}
return root;
}

```

```

// Function for in-order traversal
void inorderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%d ", root->val);
        inorderTraversal(root->right);
    }
}

```

```

int main() {
    int n;
    scanf("%d", &n);

    struct TreeNode* root = NULL;
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        root = insert(root, val);
    }
}

```

```

int keyToDelete;
scanf("%d", &keyToDelete);

printf("Before deletion: ");
inorderTraversal(root);
printf("\n");

root = deleteNode(root, keyToDelete);

printf("After deletion: ");
inorderTraversal(root);
printf("\n");

// Free allocated memory (optional but good practice)
// A proper free function would traverse and free all nodes.
// For this problem, it might not be strictly necessary for correctness,
// but it's good practice.
// Example: a simple post-order traversal for freeing memory
void freeTree(struct TreeNode* node) {
    if (node == NULL) return;
    freeTree(node->left);
    freeTree(node->right);
    free(node);
}
freeTree(root);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Thilak is studying binary search trees (BSTs) and wants to implement a program that constructs a binary search tree from a given set of characters. He aims to perform a pre-order traversal of the constructed tree to understand its structure better.

Your task is to help Thilak by designing a program that accomplishes this.

Input Format

The first line of input consists of an integer N, representing the number of characters to be inserted into the BST.

The second line consists of N space-separated characters, representing the data elements to be inserted into the BST.

Output Format

The output prints the pre-order traversal of the constructed BST.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

C A E B D

Output: C A B E D

Answer

```
import java.util.*;
```

```
class Node {  
    char data;  
    Node left, right;  
  
    Node(char data) {  
        this.data = data;  
        left = right = null;  
    }  
}
```

```
class Main {
```

```
    // Insert into BST
```

```
    public static Node insert(Node root, char data) {  
        if (root == null) {
```

```

        return new Node(data);
    }

    if (data < root.data) {
        root.left = insert(root.left, data);
    } else {
        root.right = insert(root.right, data);
    }

    return root;
}

// Pre-order traversal
public static void preorder(Node root) {
    if (root != null) {
        System.out.print(root.data + " ");
        preorder(root.left);
        preorder(root.right);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    Node root = null;

    for (int i = 0; i < n; i++) {
        char ch = sc.next().charAt(0);
        root = insert(root, ch);
    }

    preorder(root);
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are required to implement basic operations on a Binary Search Tree

(BST), like insertion and searching.

Insertion: Given a list of integers, construct a Binary Search Tree by repeatedly inserting each integer into the tree according to the rules of a BST.

Searching: Given an integer, search for its presence in the constructed Binary Search Tree. Print whether the integer is found or not.

Write a program to calculate this efficiently.

Input Format

The first line of input consists of an integer n, representing the number of nodes in the binary search tree.

The second line consists of the values of the nodes, separated by space as integers.

The third line consists of an integer representing, the value that is to be searched.

Output Format

The output prints, "Value <value> is found in the tree." if the given value is present, otherwise it prints: "Value <value> is not found in the tree."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
8 3 10 1 6 14 23
6

Output: Value 6 is found in the tree.

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
```



```

Node* left;
Node* right;

Node(int val) {
    data = val;
    left = right = NULL;
}
};

// Insert into BST
Node* insert(Node* root, int data) {
    if (root == NULL) {
        return new Node(data);
    }
    if (data < root->data) {
        root->left = insert(root->left, data);
    } else {
        root->right = insert(root->right, data);
    }

    return root;
}

// Search in BST
bool search(Node* root, int key) {
    if (root == NULL) {
        return false;
    }

    if (root->data == key) {
        return true;
    } else if (key < root->data) {
        return search(root->left, key);
    } else {
        return search(root->right, key);
    }
}

int main() {
    int n;
    cin >> n;

```

```

Node* root = NULL;

// Insert elements
for (int i = 0; i < n; i++) {
    int val;
    cin >> val;
    root = insert(root, val);
}

int key;
cin >> key;

// Search and print result
if (search(root, key)) {
    cout << "Value " << key << " is found in the tree." << endl;
} else {
    cout << "Value " << key << " is not found in the tree." << endl;
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Thiru is learning about binary search trees and wants to practice creating and traversing them. He needs to write a program that takes a series of characters as input and constructs a binary search tree with those characters.

After constructing the tree, Thiru wants to perform an in-order traversal and print all the characters in sorted order.

Input Format

The first line contains an integer n , representing the number of characters to be inserted into the binary search tree.

The second line contains n space-separated characters c_i , representing the

characters to be inserted into the binary search tree.

Output Format

The output prints a single line containing all the characters of the binary search tree in sorted order, obtained by performing an in-order traversal of the tree, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

r e f s

Output: e f r s

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    char data;
    Node* left;
    Node* right;
```

```
    Node(char val) {
        data = val;
        left = right = NULL;
    }
};
```

```
// Insert into BST
```

```
Node* insert(Node* root, char data) {
    if (root == NULL) {
        return new Node(data);
    }
```

```
    if (data < root->data) {
        root->left = insert(root->left, data);
    } else {
        root->right = insert(root->right, data);
    }
```

```

    }
    return root;
}

// In-order traversal
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

int main() {
    int n;
    cin >> n;

    Node* root = NULL;

    for (int i = 0; i < n; i++) {
        char ch;
        cin >> ch;
        root = insert(root, ch);
    }

    inorder(root);

    return 0;
}

```

Status : Partially correct

Marks : 7.5/10

5. Problem Statement

A publishing company is building a digital index system for storing book references. Each reference is represented by a single uppercase character. To ensure efficient organization, the system uses a Binary Search Tree (BST):

If the new character is smaller than the current node, it is placed in the left subtree. If the new character is greater, it is placed in the right subtree.

Once all references are stored in the BST, the system needs to perform a postorder traversal to generate a summary sequence, which is then used for indexing and cross-referencing.

Your task is to help the publishing company build the BST from the given characters and print the postorder traversal.

Example:

Input:

5

Z E W T Y

Output:

Postorder traversal: T Y W E Z

Input Format

The first line consists of an integer N, representing the number of characters.

The second line consists of N space-separated characters.

Output Format

The output displays the post-order traversal of the given inputs as space-separated.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

Z E W T Y

Output: Postorder traversal: T Y W E Z

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    char data;
    Node* left;
    Node* right;

    Node(char val) {
        data = val;
        left = right = NULL;
    }
};
```

```
// Insert into BST
Node* insert(Node* root, char data) {
    if (root == NULL) {
        return new Node(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    } else {
        root->right = insert(root->right, data);
    }

    return root;
}
```

```
// Postorder traversal
void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}
```

```
int main() {
    int n;
    cin >> n;
```

```
Node* root = NULL;

for (int i = 0; i < n; i++) {
    char ch;
    cin >> ch;
    root = insert(root, ch);
}

cout << "Postorder traversal: ";
postorder(root);

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 9_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be moved.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j , representing the column index of the fixed queen.

Output Format

If solutions exist: Print each valid board configuration, where "Q" represents a queen and "." represents an empty cell.

Each row of the board should be space-separated.

Each configuration should be followed by a blank line.

If no configuration is possible, print "No solution".

Refer to the sample output for the formatting specification.

Sample Test Case

Input: 4

0

1

Output: . Q . .

. . . Q

Q . . .

. . Q .

Answer

```
import java.util.*;
```

```
public class Main {
```

```

static int N;
static int fixedRow, fixedCol;
static int[] board;
static boolean found = false;

static boolean isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        int c = board[i];

        // same column
        if (c == col) return false;

        // diagonal check
        if (Math.abs(i - row) == Math.abs(c - col)) return false;
    }
    return true;
}

static void solve(int row) {
    if (row == N) {
        printBoard();
        found = true;
        return;
    }

    // If this is fixed row, force placement
    if (row == fixedRow) {
        if (isSafe(row, fixedCol)) {
            board[row] = fixedCol;
            solve(row + 1);
        }
        return;
    }

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            solve(row + 1);
        }
    }
}

```

```

static void printBoard() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            System.out.print((board[i] == j ? "Q " : ". "));
        }
        System.out.println();
    }
    System.out.println();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    N = sc.nextInt();
    fixedRow = sc.nextInt();
    fixedCol = sc.nextInt();

    board = new int[N];

    solve(0);

    if (!found) {
        System.out.print("No solution");
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Consider a rat placed at (0, 0) in a square matrix of order $N \times N$. It has to reach the destination at $(N - 1, N - 1)$. Find all possible paths that the rat can take to reach from the source to the destination. The directions in which the rat can move are 'U'(up), 'D'(down), 'L' (left), and 'R' (right). Value 0 at a cell in the matrix represents that it is blocked and the rat cannot move to it while value 1 at a cell in the matrix represents that the rat can travel through it. Return the list of paths in lexicographically increasing order.

Note: In a path, no cell can be visited more than once. If the source cell is 0, the rat cannot move to any other cell.

Example 1

Input:

N = 4

m[][] =

{{1, 0, 0, 0},

{1, 1, 0, 1},

{1, 1, 0, 0},

{0, 1, 1, 1}}

Output:

DDRDRR DRDDRR

Explanation:

The rat can reach the destination at (3, 3) from (0, 0) by two paths - DRDDRR and DDRDRR, when printed in sorted order we get DDRDRR DRDDRR.

Example 2

Input:

N = 2

m[][] =

{{1, 0},

{1, 0}}

Output:

-1

Explanation:

No path exists and the destination cell is blocked.

Input Format

The first line of input contains an integer n, representing the size of the maze.

The next n lines each contain n space-separated integers, representing the maze.

Output Format

The output consists of

If at least one path is found, print all valid paths as space-separated strings.

If no path is found, prints -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

1 0 0 0

1 1 0 1

1 1 0 0

0 1 1 1

Output: DDRDRR DRDDRR

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static int N;
```

```
    static int[][] maze;
```

```
    static boolean[][] visited;
```

```
    static List<String> result = new ArrayList<>();
```

```
    static int[] dr = {1, 0, 0, -1}; // D, L, R, U
```

```
    static int[] dc = {0, -1, 1, 0};
```

```
    static char[] move = {'D', 'L', 'R', 'U'};
```

```

static void solve(int r, int c, String path) {
    // reached destination
    if (r == N - 1 && c == N - 1) {
        result.add(path);
        return;
    }

    // explore 4 directions
    for (int i = 0; i < 4; i++) {
        int nr = r + dr[i];
        int nc = c + dc[i];

        if (nr >= 0 && nc >= 0 && nr < N && nc < N &&
            !visited[nr][nc] && maze[nr][nc] == 1) {

            visited[nr][nc] = true;
            solve(nr, nc, path + move[i]);
            visited[nr][nc] = false;
        }
    }
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

```

```

    N = sc.nextInt();
    maze = new int[N][N];
    visited = new boolean[N][N];

```

```

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            maze[i][j] = sc.nextInt();
        }
    }

```

```

    if (maze[0][0] == 0 || maze[N - 1][N - 1] == 0) {
        System.out.print("-1");
        return;
    }

```

```

    visited[0][0] = true;

```

```

solve(0, 0, "");
    if (result.isEmpty()) {
        System.out.print("-1");
    } else {
        Collections.sort(result);
        for (String s : result) {
            System.out.print(s + " ");
        }
    }
}
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Given a partially filled 9x9 2D array, the goal is to assign digits (from 1 to 9) to the empty cells so that every row, column, and sub-grid of size 3x3 contains exactly one instance of the digits from 1 to 9.

Write a program for the same.

Example

Input:

```

3 0 6 5 0 8 4 0 0
5 2 0 0 0 0 0 0 0
0 8 7 0 0 0 0 3 1
0 0 3 0 1 0 0 8 0
9 0 0 8 6 3 0 0 5
0 5 0 0 9 0 6 0 0
1 3 0 0 0 0 2 5 0
0 0 0 0 0 0 0 7 4
0 0 5 2 0 6 3 0 0

```

Output:

```
3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Explanation

To solve the given Sudoku puzzle, we apply the fundamental rules of Sudoku: each row, column, and 3x3 subgrid must contain the digits 1 through 9 without any repetition. Starting with the given partially filled grid, we systematically fill in the empty cells. For each empty cell, we check the numbers already present in the corresponding row, column, and 3x3 subgrid to determine which number can legally go in that position. This process involves logical deduction and elimination. In the case of the provided puzzle, the empty cells are filled one by one, ensuring that each row, column, and subgrid adheres to the Sudoku rules. For example, in Row 1, the missing numbers (1, 7, 9, 2) are placed in the available spots based on their placement in the columns and subgrids. This continues for each row, progressively narrowing down the possibilities for the empty cells until the entire grid is complete. The final solution, after applying this reasoning, looks like the output grid:

Input Format

The input consists of a 9x9 matrix in 9 rows and 9 columns separated by space.

Output Format

The output displays the 9x9 matrix with the updated values.

If the conditions are not satisfied, then display "No Solution exists".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 0 6 5 0 8 4 0 0

5 2 0 0 0 0 0 0 0

0 8 7 0 0 0 0 3 1

0 0 3 0 1 0 0 8 0

9 0 0 8 6 3 0 0 5

0 5 0 0 9 0 6 0 0

1 3 0 0 0 0 2 5 0

0 0 0 0 0 0 0 7 4

0 0 5 2 0 6 3 0 0

Output: 3 1 6 5 7 8 4 9 2

5 2 9 1 3 4 7 6 8

4 8 7 6 2 9 5 3 1

2 6 3 4 1 5 9 8 7

9 7 4 8 6 3 1 2 5

8 5 1 7 9 2 6 4 3

1 3 8 9 4 7 2 5 6

6 9 2 3 5 1 8 7 4

7 4 5 2 8 6 3 1 9

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static int N = 9;
```

```
    static boolean isSafe(int[][] board, int row, int col, int num) {
```

```
        // Check row
```

```
        for (int i = 0; i < 9; i++) {
```

```
            if (board[row][i] == num) return false;
```

```
        }
```

```
        // Check column
```

```

    for (int i = 0; i < 9; i++) {
        if (board[i][col] == num) return false;
    }

    // Check 3x3 box
    int startRow = row - row % 3;
    int startCol = col - col % 3;

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[startRow + i][startCol + j] == num)
                return false;
        }
    }

    return true;
}

```

```

static boolean solveSudoku(int[][] board) {

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {

            if (board[row][col] == 0) {

                for (int num = 1; num <= 9; num++) {

                    if (isSafe(board, row, col, num)) {

                        board[row][col] = num;

                        if (solveSudoku(board)) {
                            return true;
                        }

                        board[row][col] = 0; // backtrack
                    }
                }

                return false; // no number fits
            }
        }
    }
}

```

```

    }
    return true; // solved
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int[][] board = new int[9][9];

    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            board[i][j] = sc.nextInt();
        }
    }

    if (solveSudoku(board)) {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                System.out.print(board[i][j] + " ");
            }
            System.out.println();
        }
    } else {
        System.out.print("No Solution exists");
    }
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given an undirected graph and a number m , determine if the graph can be colored with at most m colors such that no two adjacent vertices of the graph are colored with the same color.

Note: Here, the coloring of a graph means the assignment of colors to all vertices.

Example

Input:

0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

3

Output:

Solution Exists:

1 2 3 2

Explanation:

A minimum of 3 colors is required for the above graph.

Input Format

The input consists of four lines each containing four space-separated integers (1 if connected, 0 if not), representing the adjacency matrix of a graph where each element represents the connection between vertices.

The last line consists of an integer m, representing the number of colors available.

Output Format

If a solution exists:

1. The first line prints "Solution Exists:".
2. The second line prints the assigned colors for each vertex, separated by a space.

If no solution exists, the output prints "Solution does not exist".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

3

Output: Solution Exists:

1 2 3 2

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static int V = 4;
```

```
    static int[][] graph = new int[4][4];
```

```
    static int[] color = new int[4];
```

```
    static int m;
```

```
    static boolean isSafe(int v, int c) {
```

```
        for (int i = 0; i < V; i++) {
```

```
            if (graph[v][i] == 1 && color[i] == c) {
```

```
                return false;
```

```
            }
```

```
        }
```

```
        return true;
```

```
    }
```

```
    static boolean solve(int v) {
```

```
        if (v == V) {
```

```
            return true;
```

```
        }
```

```
        for (int c = 1; c <= m; c++) {
```

```
            if (isSafe(v, c)) {
```

```
                color[v] = c;
```

```

        if (solve(v + 1)) {
            return true;
        }

        color[v] = 0; // backtrack
    }
}

return false;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Read adjacency matrix
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            graph[i][j] = sc.nextInt();
        }
    }

    m = sc.nextInt();

    Arrays.fill(color, 0);

    if (solve(0)) {
        System.out.println("Solution Exists:");
        for (int i = 0; i < V; i++) {
            System.out.print(color[i] + " ");
        }
    } else {
        System.out.print("Solution does not exist");
    }
}
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 9_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. In general, backtracking can be used to solve?

Answer

Combinatorial problems

Status : Correct

Marks : 1/1

2. Backtracking may lead to a solution that is _____.

Answer

suboptimal

Status : Correct

Marks : 1/1

3. What is the main idea behind Backtracking to check if a graph is connected?

Answer

Trying all possible paths from a starting vertex and marking visited vertices

Status : Correct

Marks : 1/1

4. In Backtracking, what is the purpose of marking vertices as visited during graph traversal?

Answer

To avoid revisiting the same vertex

Status : Correct

Marks : 1/1

5. In Backtracking, what is the base case for checking connectivity in an undirected graph?

Answer

All vertices are visited

Status : Correct

Marks : 1/1

6. In a Sudoku solver, when does the backtracking algorithm backtrack?

Answer

When no valid number can be placed in a cell

Status : Correct

Marks : 1/1

7. Which of the following problems cannot be solved using backtracking?

Answer

Finding shortest path in weighted graph

Status : Wrong

Marks : 0/1

8. Which of the following is essential for backtracking to function correctly?

Answer

A recursive call with a rollback (backtrack step)

Status : Correct

Marks : 1/1

9. What is a base case in a backtracking algorithm?

Answer

A condition when a solution is found or invalid

Status : Correct

Marks : 1/1

10. Which data structure is commonly used in backtracking to store partial solutions?

Answer

Stack or List

Status : Correct

Marks : 1/1

11. In the Hamiltonian Cycle problem using backtracking, what must be true for a path to be valid?

Answer

All vertices are included once, and the last connects to the first

Status : Correct

Marks : 1/1

12. What is the primary technique used to solve the Rat in a Maze problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

13. Which of the following best describes the backtracking approach to generating permutations of a string?

Answer

Fix one character, recursively permute the rest, and backtrack

Status : Correct

Marks : 1/1

14. In backtracking solutions for maze problems, when do we backtrack from a cell?

Answer

When all directions from the current cell are either visited or blocked

Status : Correct

Marks : 1/1

15. Which of the following constraints are typically used in Sudoku backtracking?

Answer

All the mentioned options

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 5_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : MCQ

1. After adding a new node at the end of a linked list, what does the current last node become?

Answer

The second last node

Status : Correct

Marks : 1/1

2. When inserting a new element at the beginning of a linked list, what happens to the current head of the list?

Answer

It becomes the second element

Status : Correct

Marks : 1/1

3. Which of the following are true about a singly linked list?

Answer

It can grow dynamically during runtime.

The last node's "next" pointer points to NULL in a non-circular list.

Status : Correct

Marks : 1/1

4. When inserting a new node into a singly linked list, which of the following are the possible cases for insertion?

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

5. Given the linked list: 5 -> 10 -> 15 -> 20 -> 25 -> NULL. What will be the output of traversing the list and printing each node's data?

Answer

5 10 15 20 25

Status : Correct

Marks : 1/1

6. Which of the following is the process of finding information in a linked list?

Answer

Start from the head of the list and traverse sequentially until finding (or not finding) the node

Status : Correct

Marks : 1/1

7. Which of the following actions needs to be taken when the last item in a linked list is deleted?

Answer

Change the reference of the second last item to NULL reference

Status : Correct

Marks : 1/1

8. Which pointer helps in traversing a doubly linked list in reverse order?

Answer

prev

Status : Correct

Marks : 1/1

9. Which of the following is true about the last node in a doubly linked list?

Answer

Its next pointer is NULL

Status : Correct

Marks : 1/1

10. What will be the effect of setting the prev pointer of a node to NULL in a doubly linked list?

Answer

The node will become the new head

Status : Correct

Marks : 1/1

11. How do you delete a node from the middle of a doubly linked list?

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

12. What happens if we insert a node at the beginning of a doubly linked list?

Answer

The previous pointer of the new node is NULL

Status : Correct

Marks : 1/1

13. How many pointers does a node in a doubly linked list have?

Answer

2

Status : Correct

Marks : 1/1

14. In deletion from the end of a doubly linked list, which pointer needs to be updated?

Answer

prev->next = NULL

Status : Correct

Marks : 1/1

15. Which of the following is true about a Doubly Linked List?

Answer

Last node's next pointer is always NULL

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 7_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given a BST, write an efficient function to delete a given key in it.

There are three cases that can be considered:

Deleting a node with no children: remove the node from the tree. Deleting a node with two children: Deleting a node with one child

Input Format

The first line of the input consists of an integer n representing, the number of nodes.

The second line of the input consists of integers space-separated representing, input node values.

The third line of the input consists of an integer q representing, the node value to

be deleted.

Output Format

The output prints the node values before and after the deletion of the key value in the format:

"Before deletion of key value:"

"After deleting key value:"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

10 30 5 6 7

30

Output: Before deletion of key value:

5 6 7 10 30

After deleting key value:

5 6 7 10

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;
```

```
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};
```

```
// Insert into BST
```

```
Node* insert(Node* root, int data) {
    if (root == NULL) return new Node(data);
```



```

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}

```

```

// Find minimum value node (used in deletion)
Node* findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

```

```

// Delete node from BST
Node* deleteNode(Node* root, int key) {
    if (root == NULL) return root;

    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    }
    else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    }
    else {
        // Case 1 & 2: node with 0 or 1 child
        if (root->left == NULL) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
    }
}

```

```

// Case 3: node with 2 children
Node* temp = findMin(root->right);
root->data = temp->data;

```

```
        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}
```

```
// Inorder traversal
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}
```

```
int main() {
    int n;
    cin >> n;

    Node* root = NULL;

    for (int i = 0; i < n; i++) {
        int val;
        cin >> val;
        root = insert(root, val);
    }
```

```
    int key;
    cin >> key;
```

```
    cout << "Before deletion of key value:\n";
    inorder(root);
    cout << "\n";
```

```
    root = deleteNode(root, key);
```

```
    cout << "After deleting key value:\n";
    inorder(root);
```

```
    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Revi is studying binary search trees (BSTs) and wants to explore all the unique BSTs that can be constructed using distinct integers from 1 to n. He decides to write a program that generates all such BSTs and prints their preorder traversals.

Write a program to ensure that the preorder traversal of all possible BSTs constructed from numbers 1 to n is printed, each on a new line.

Example

Input

3

Output

1 2 3

1 3 2

2 1 3

3 1 2

3 2 1

Explanation

Input Format

The input consists of a single integer n, representing the number of nodes.

Output Format

Each line contains the preorder traversal (space-separated) of one possible BST using values from 1 to n.

Each traversal is printed on a new line.

There should be no blank lines between the outputs, and the order of output may vary depending on the internal recursive tree generation.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 1 2 3

1 3 2

2 1 3

3 1 2

3 2 1

Answer

```
#include <iostream>
#include <vector>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;
```

```
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};
```

```
// Generate all BSTs from start to end
vector<Node*> generateTrees(int start, int end) {
    vector<Node*> trees;
```

```
    if (start > end) {
        trees.push_back(NULL);
        return trees;
    }
```

```
    for (int i = start; i <= end; i++) {
```

```

vector<Node*> leftSubtrees = generateTrees(start, i - 1);
vector<Node*> rightSubtrees = generateTrees(i + 1, end);

    for (auto left : leftSubtrees) {
        for (auto right : rightSubtrees) {
            Node* root = new Node(i);
            root->left = left;
            root->right = right;
            trees.push_back(root);
        }
    }
}

return trees;
}

// Preorder traversal
void preorder(Node* root) {
    if (root == NULL) return;

    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

int main() {
    int n;
    cin >> n;

    vector<Node*> allTrees = generateTrees(1, n);

    for (auto tree : allTrees) {
        preorder(tree);
        cout << "\n";
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Jai is fascinated by binary trees and wants to create and manipulate them. He needs your help to implement basic operations like insertion and deletion in a binary search tree.

Given a sequence of characters representing the values to be inserted into a binary search tree, followed by a character V to be deleted from the tree, your task is to construct the binary search tree and delete the specified character from it.

Your program should perform the following operations:

Construct a binary search tree from the given sequence of characters. Delete the specified character V from the constructed tree. Print the elements of the tree in sorted order after deletion.

Input Format

The first line contains an integer N, denoting the number of characters in the sequence.

The second line contains N space-separated characters, representing the sequence of characters to be inserted into the binary search tree.

The third line contains a single character V (which is guaranteed to be present in the tree), representing the value to be deleted from the binary search tree.

Output Format

The output prints a single line containing the elements of the binary search tree in sorted order after deleting the specified character V.

If the specified value is not available in the tree, print the given input values in order traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

F B G A D

B

Output: A D F G

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    char data;
    Node* left;
    Node* right;
```

```
    Node(char val) {
        data = val;
        left = right = NULL;
    }
};
```

```
// Insert into BST
```

```
Node* insert(Node* root, char data) {
    if (root == NULL) return new Node(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}
```

```
// Find minimum node
```

```
Node* findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}
```

```
// Delete node
```

```
Node* deleteNode(Node* root, char key) {
    if (root == NULL) return root;
```

```

if (key < root->data)
    root->left = deleteNode(root->left, key);
else if (key > root->data)
    root->right = deleteNode(root->right, key);
else {
    // Case 1 & 2
    if (root->left == NULL) {
        Node* temp = root->right;
        delete root;
        return temp;
    }
    else if (root->right == NULL) {
        Node* temp = root->left;
        delete root;
        return temp;
    }

    // Case 3: two children
    Node* temp = findMin(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
return root;
}

```

```

// Inorder traversal
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

```

```

// Search (optional safety)
bool search(Node* root, char key) {
    if (root == NULL) return false;
    if (root->data == key) return true;
    if (key < root->data) return search(root->left, key);
    return search(root->right, key);
}

```



```

int main() {
    int n;
    cin >> n;

    Node* root = NULL;

    for (int i = 0; i < n; i++) {
        char ch;
        cin >> ch;
        root = insert(root, ch);
    }

    char key;
    cin >> key;

    // If key exists, delete
    if (search(root, key)) {
        root = deleteNode(root, key);
    }

    inorder(root);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement:

Arun is exploring operations on binary search trees (BST). He wants to write a program that takes an unsorted array of distinct integers and constructs a BST by inserting elements in the given order (not height-balanced).

After constructing the BST, he wants to delete all nodes at the deepest level of the tree and perform a post-order traversal to print the remaining nodes.

Example

Input:

7

1 7 4 9 2 11 12

Output:

Before deleting: 2 4 12 11 9 7 1

After deleting: 2 4 11 9 7 1

Explanation:

The BST is constructed by inserting elements in the given order: 1, 7, 4, 9, 2, 11, 12. This creates a regular BST (not height-balanced) where each element is inserted according to BST properties.

The tree is traversed in post-order before deleting the deepest level, showing: 2, 4, 12, 11, 9, 7, 1.

The deepest level nodes (in this case, node 12) are then deleted.

The tree is traversed in post-order after deleting the deepest level, showing: 2, 4, 11, 9, 7, 1.

Input Format

The first line contains an integer n, representing the number of nodes to be inserted in the BST.

The second line contains n space-separated integers, representing the elements to be inserted into the BST in the given order.

Output Format

The first line prints "Before deleting: " followed by the space-separated elements of the tree in post-order traversal before deleting the deepest level, ending with a space and newline.

The second line prints "After deleting: " followed by the space-separated elements of the tree in post-order traversal after deleting the deepest level, ending with a space and newline.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

1 7 4 9 2 11 12

Output: Before deleting: 2 4 12 11 9 7 1

After deleting: 2 4 11 9 7 1

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;
```

```
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};
```

```
// Insert into BST
```

```
Node* insert(Node* root, int data) {
    if (root == NULL) return new Node(data);
```

```
    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);
```

```
    return root;
```

```
}
```

```
// Find height of tree
```

```
int height(Node* root) {
    if (root == NULL) return 0;
```

```

    return 1 + max(height(root->left), height(root->right));
}

// Delete nodes at deepest level
Node* deleteDeepest(Node* root, int level, int maxLevel) {
    if (root == NULL) return NULL;

    // If this is deepest level node delete it
    if (level == maxLevel) {
        delete root;
        return NULL;
    }

    root->left = deleteDeepest(root->left, level + 1, maxLevel);
    root->right = deleteDeepest(root->right, level + 1, maxLevel);

    return root;
}

```

```

// Postorder traversal
void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}

```

```

int main() {
    int n;
    cin >> n;

    Node* root = NULL;

    for (int i = 0; i < n; i++) {
        int val;
        cin >> val;
        root = insert(root, val);
    }

    cout << "Before deleting: ";
    postorder(root);
}

```

```
cout << "\n";  
int h = height(root);  
root = deleteDeepest(root, 1, h);  
  
cout << "After deleting: ";  
postorder(root);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 10_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Jovika is learning about heaps and wants to implement a program that constructs a max heap and inserts elements into it.

She wants to write a program that implements max heap insertion. The program defines functions to swap elements and perform maximum heapification. She wants to extend this program to take a list of integers, build a max heap, and insert a new element into the heap.

Help her in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements to be inserted into the max heap.

The second line consists of N space-separated integers, representing the elements to be inserted into the max heap.

The third line consists of an integer representing the new element to be inserted into the max heap.

Output Format

The output prints the space-separated integers, representing the elements in the max heap after inserting the new element in the order of insertion.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

3 1 4 2 5

6

Output: 6 3 5 2 1 4

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static void heapifyDown(ArrayList<Integer> heap, int n, int i) {
```

```
        int largest = i;
```

```
        int l = 2 * i + 1;
```

```
        int r = 2 * i + 2;
```

```
        if (l < n && heap.get(l) > heap.get(largest))
```

```
            largest = l;
```

```
        if (r < n && heap.get(r) > heap.get(largest))
```

```
            largest = r;
```

```
        if (largest != i) {
```

```
            Collections.swap(heap, i, largest);
```

```
            heapifyDown(heap, n, largest);
```

```
        }
```

```

    }
    static void buildHeap(ArrayList<Integer> heap) {
        int n = heap.size();
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapifyDown(heap, n, i);
        }
    }
}

```

```

static void insertHeap(ArrayList<Integer> heap, int val) {
    heap.add(val);

```

```

    int i = heap.size() - 1;

```

```

    // sift up
    while (i > 0) {
        int parent = (i - 1) / 2;

```

```

        if (heap.get(parent) < heap.get(i)) {
            Collections.swap(heap, parent, i);
            i = parent;
        } else {
            break;
        }
    }
}

```

```

}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

```

```

    int n = sc.nextInt();
    ArrayList<Integer> heap = new ArrayList<>();

```

```

    for (int i = 0; i < n; i++) {
        heap.add(sc.nextInt());
    }

```

```

    int val = sc.nextInt();

```

```

    // IMPORTANT: build heap first
    buildHeap(heap);

```



```
// then insert new element
insertHeap(heap, val);

for (int x : heap) {
    System.out.print(x + " ");
}
}
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Bob is fascinated by numbers and loves arranging them in various ways. His recent exploration led him to Max Heaps.

Bob likes odd numbers more than even numbers. After inserting elements into a Max Heap, he wants to remove all even numbers from it while keeping the Max Heap property intact.

Your task is to help Bob build a Max Heap from the given elements, display it, delete all the even numbers from the resultant array and then rearrange the elements in the form of Max Heap.

Input Format

The first line of input consists of an integer N, representing the number of elements initially to be inserted into the Max Heap.

The second line consists of N space-separated integers, representing the elements to be inserted into the heap.

Output Format

The first line of output prints the space-separated elements of the Max Heap after building it from the given N elements.

The second line prints the space-separated elements of the Max Heap after removing even elements.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3 2 1

Output: 3 2 1

3 1

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static void swap(ArrayList<Integer> heap, int i, int j) {
```

```
        int temp = heap.get(i);
```

```
        heap.set(i, heap.get(j));
```

```
        heap.set(j, temp);
```

```
    }
```

```
    // Heapify down (max heap)
```

```
    static void heapify(ArrayList<Integer> heap, int n, int i) {
```

```
        int largest = i;
```

```
        int l = 2 * i + 1;
```

```
        int r = 2 * i + 2;
```

```
        if (l < n && heap.get(l) > heap.get(largest))
```

```
            largest = l;
```

```
        if (r < n && heap.get(r) > heap.get(largest))
```

```
            largest = r;
```

```
        if (largest != i) {
```

```
            swap(heap, i, largest);
```

```
            heapify(heap, n, largest);
```

```
        }
```

```
    }
```

```
    // Build heap
```

```
    static void buildHeap(ArrayList<Integer> heap) {
```

```
        int n = heap.size();
```

```
        for (int i = n / 2 - 1; i >= 0; i--) {  
            heapify(heap, n, i);  
        }  
    }
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    int n = sc.nextInt();  
    ArrayList<Integer> heap = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++) {  
        heap.add(sc.nextInt());  
    }
```

```
    // Step 1: Build max heap  
    buildHeap(heap);
```

```
    // Print initial heap  
    for (int x : heap) {  
        System.out.print(x + " ");  
    }  
    System.out.println();
```

```
    // Step 2: Remove even numbers  
    ArrayList<Integer> oddHeap = new ArrayList<>();  
    for (int x : heap) {  
        if (x % 2 != 0) {  
            oddHeap.add(x);  
        }  
    }
```

```
    // Step 3: Rebuild heap with odd numbers  
    buildHeap(oddHeap);
```

```
    // Print final heap  
    for (int x : oddHeap) {  
        System.out.print(x + " ");  
    }  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Iniya is studying data structures and algorithms, and she is currently learning about heaps.

She wants to practice implementing Min-Heap and perform a deletion operation on it. A min-heap is a binary tree in which the value of each node is less than or equal to the values of its children.

Write a program to assist Iniya in building a min-heap and delete the root element.

Input Format

The first line of input consists of an integer N, representing the number of elements in the initial Min-Heap.

The second line consists of N space-separated integers, where each integer represents an element in the Min-Heap.

Output Format

The output prints the space-separated elements of the Min-Heap after performing the deletion operation of the root.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

80 40 30 10 50 33 66

Output: 30 40 33 80 50 66

Answer

```
import java.util.*;  
  
public class Main {
```

```
static void swap(ArrayList<Integer> heap, int i, int j) {  
    int temp = heap.get(i);  
    heap.set(i, heap.get(j));  
    heap.set(j, temp);  
}
```

// Min heapify (downward)

```
static void heapify(ArrayList<Integer> heap, int n, int i) {  
    int smallest = i;  
    int l = 2 * i + 1;  
    int r = 2 * i + 2;
```

```
    if (l < n && heap.get(l) < heap.get(smallest))  
        smallest = l;
```

```
    if (r < n && heap.get(r) < heap.get(smallest))  
        smallest = r;
```

```
    if (smallest != i) {  
        swap(heap, i, smallest);  
        heapify(heap, n, smallest);  
    }
```

```
}
```

// Build Min Heap

```
static void buildHeap(ArrayList<Integer> heap) {  
    int n = heap.size();  
    for (int i = n / 2 - 1; i >= 0; i--) {  
        heapify(heap, n, i);  
    }
```

```
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);
```

```
    int n = sc.nextInt();  
    ArrayList<Integer> heap = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++) {  
        heap.add(sc.nextInt());  
    }
```

```

// Step 1: Build Min Heap
buildHeap(heap);

// Step 2: Delete root (swap with last + remove)
if (heap.size() > 0) {
    heap.set(0, heap.get(heap.size() - 1));
    heap.remove(heap.size() - 1);

    // Step 3: Heapify from root
    heapify(heap, heap.size(), 0);
}

// Step 4: Print result
for (int x : heap) {
    System.out.print(x + " ");
}
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

A trie (pronounced as "try") or prefix tree is a tree data structure used to store and retrieve keys in a dataset of strings efficiently. This data structure has various applications, such as autocomplete and spellchecker.

Implement the Trie class:

Trie() Initializes the trie object.
 void insert(String word): Inserts the string word into the trie.
 boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
 boolean startsWith(String prefix) Returns true if a previously inserted string word has the prefix, and false otherwise.

Example

Input:

4

apple

cat

orange

mat

cat

ora

Output:

The word is present in the trie.

There is a word with the given prefix in the trie.

Input Format

The first line of the input consists of the number of words to insert, N.

The next N lines of input consist of the words in each line.

The next line of input consists of the word to search.

The last line of the input consists of the prefix to search.

Output Format

When searching for a word using the search method, the program will output one of the following messages:

- If the word is present in the trie, the output prints "The word is present in the trie."
- If the word is not present in the trie, the output prints "The word is not present in the trie."

When checking for a prefix using the startsWith method, the program will output one of the following messages:

- If there is a word in the trie with the given prefix, the output prints "There is a word with the given prefix in the trie."

- If there is no word in the trie with the given prefix, the output prints "There is no word with the given prefix in the trie."

Sample Test Case

Input: 4

apple

cat

orange

mat

catlyn

ora

Output: The word is not present in the trie.

There is a word with the given prefix in the trie.

Answer

```
import java.util.*;
```

```
class TrieNode {  
    TrieNode[] children = new TrieNode[26];  
    boolean isEnd;
```

```
    TrieNode() {  
        isEnd = false;  
        for (int i = 0; i < 26; i++) {  
            children[i] = null;
```

```
        }  
    }  
}
```

```
class Trie {  
    TrieNode root;
```

```
    Trie() {  
        root = new TrieNode();  
    }
```

```
    // Insert word
```

```
    void insert(String word) {  
        TrieNode node = root;
```

```
        for (char ch : word.toCharArray()) {  
            int idx = ch - 'a';
```



```
        if (node.children[idx] == null) {  
            node.children[idx] = new TrieNode();  
        }  
  
        node = node.children[idx];  
    }  
  
    node.isEnd = true;  
}
```

```
// Search full word  
boolean search(String word) {  
    TrieNode node = root;  
  
    for (char ch : word.toCharArray()) {  
        int idx = ch - 'a';  
  
        if (node.children[idx] == null) {  
            return false;  
        }  
  
        node = node.children[idx];  
    }  
  
    return node.isEnd;  
}
```

```
// Search prefix  
boolean startsWith(String prefix) {  
    TrieNode node = root;  
  
    for (char ch : prefix.toCharArray()) {  
        int idx = ch - 'a';  
  
        if (node.children[idx] == null) {  
            return false;  
        }  
  
        node = node.children[idx];  
    }  
}
```

```

        return true;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine();

        Trie trie = new Trie();

        for (int i = 0; i < n; i++) {
            String word = sc.nextLine();
            trie.insert(word);
        }

        String searchWord = sc.nextLine();
        String prefix = sc.nextLine();

        // Search word
        if (trie.search(searchWord)) {
            System.out.println("The word is present in the trie.");
        } else {
            System.out.println("The word is not present in the trie.");
        }

        // Search prefix
        if (trie.startsWith(prefix)) {
            System.out.println("There is a word with the given prefix in the trie.");
        } else {
            System.out.println("There is no word with the given prefix in the trie.");
        }
    }
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Given an array of integers representing the combat power of N players.

You need to efficiently perform the following operations:

Query (Q L R): Find the maximum combat power between ranks L and R (inclusive). Update (U i val): Update the power of the player at index i to a new value val.

To ensure efficiency, implement these operations using a Segment Tree.

Input Format

The first line contains two space-separated integers, N and Q, representing the number of players and the number of operations.

The second line contains N space-separated integers representing the initial combat power scores of all players, where index 0 is the top-ranked player.

The next Q lines describe the operations:

- "Q L R" Query represents the maximum combat power from ranks L to R (inclusive).
- "U i val" Updates the power score of the player at rank i to val.

All indices are 0-based.

Output Format

For each query Q L R, print the maximum combat power in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
4 7 2 9 5 1
Q 1 4
U 3 6
Q 1 4
U 0 10

Q 0 2

Output: 9

7

10

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static int[] seg;
```

```
    static int n;
```

```
    static void build(int[] arr, int idx, int l, int r) {
```

```
        if (l == r) {
```

```
            seg[idx] = arr[l];
```

```
            return;
```

```
        }
```

```
        int mid = (l + r) / 2;
```

```
        build(arr, 2 * idx + 1, l, mid);
```

```
        build(arr, 2 * idx + 2, mid + 1, r);
```

```
        seg[idx] = Math.max(seg[2 * idx + 1], seg[2 * idx + 2]);
```

```
    }
```

```
    static void update(int idx, int l, int r, int pos, int val) {
```

```
        if (l == r) {
```

```
            seg[idx] = val;
```

```
            return;
```

```
        }
```

```
        int mid = (l + r) / 2;
```

```
        if (pos <= mid) {
```

```
            update(2 * idx + 1, l, mid, pos, val);
```

```
        } else {
```

```
            update(2 * idx + 2, mid + 1, r, pos, val);
```

```
        }
```

```
    seg[idx] = Math.max(seg[2 * idx + 1], seg[2 * idx + 2]);  
}
```

```
static int query(int idx, int l, int r, int ql, int qr) {
```

```
    if (qr < l || r < ql) return Integer.MIN_VALUE;
```

```
    if (ql <= l && r <= qr) return seg[idx];
```

```
    int mid = (l + r) / 2;
```

```
    int left = query(2 * idx + 1, l, mid, ql, qr);
```

```
    int right = query(2 * idx + 2, mid + 1, r, ql, qr);
```

```
    return Math.max(left, right);
```

```
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    n = sc.nextInt();
```

```
    int q = sc.nextInt();
```

```
    int[] arr = new int[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        arr[i] = sc.nextInt();
```

```
    }
```

```
    seg = new int[4 * n];
```

```
    build(arr, 0, 0, n - 1);
```

```
    while (q-- > 0) {
```

```
        String type = sc.next();
```

```
        if (type.equals("Q")) {
```

```
            int l = sc.nextInt();
```

```
            int r = sc.nextInt();
```

```
            System.out.println(query(0, 0, n - 1, l, r));
```

```
        } else {
```

```
            int i = sc.nextInt();
```

```
            int val = sc.nextInt();
```

```
        update(0, 0, n - 1, i, val);  
    }  
}  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 12_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an amount, find the minimum number of notes of different denominations that sum up to the given amount. Starting from the highest denomination note, try to accommodate as many notes as possible for a given amount.

We may assume that we have infinite supply of notes of values {2000, 500, 200, 100, 50, 20, 10, 5, 1}

Examples:

Input: 800

Output: Currency Count

500 : 1

200 : 1

100 : 1

Input: 2456

Output: Currency Count

2000 : 1

200 : 2

50 : 1

5 : 1

1 : 1

Input Format

The input contains an integer representing the amount to be broken down.

Output Format

The output prints "Currency Count" followed by each denomination used in the format:

"note_value : count"

(Only print denominations that are used.)

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 800

Output: Currency Count

500 : 1

200 : 1

100 : 1

Answer

```
import java.util.*;
```



```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int amount = sc.nextInt();

        int[] notes = {2000, 500, 200, 100, 50, 20, 10, 5, 1};

        System.out.println("Currency Count");

        for (int note : notes) {
            if (amount >= note) {
                int count = amount / note;
                amount = amount % note;

                if (count > 0) {
                    System.out.println(note + " : " + count);
                }
            }
        }
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given a lock which is made up of n-different circular rings and each ring has 0-9 digit printed serially on it. Initially all n-rings together show a n-digit integer but there is particular code only which can open the lock.

You can rotate each ring any number of time in either direction. You have to find the minimum number of rotation done on rings of lock to open the lock.

Examples:

Input: Input = 2345, Unlock code = 5432

Output: Rotations required = 8

Explanation: 1st ring is rotated thrice as 2->3->4->5

2nd ring is rotated once as 3->4

3rd ring is rotated once as 4->3

4th ring is rotated thrice as 5->4->3->2

Input: Input = 1919, Unlock code = 0000

Output: Rotations required = 4

Explanation: 1st ring is rotated once as 1->0

2nd ring is rotated once as 9->0

3rd ring is rotated once as 1->0

4th ring is rotated once as 9->0

Input Format

The first line contains an integer representing the current input on the lock.

The second line contains an integer representing the unlock code.

Output Format

The output prints "Minimum Rotation = X" where X is the minimum total number of rotations needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 28756

98234

Output: Minimum Rotation = 12

Answer

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

String start = sc.next();
String target = sc.next();

int total = 0;

for (int i = 0; i < start.length(); i++) {
    int a = start.charAt(i) - '0';
    int b = target.charAt(i) - '0';

    int diff = Math.abs(a - b);
    total += Math.min(diff, 10 - diff);
}

System.out.println("Minimum Rotation = " + total);
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement:

Sarah is organizing a race in her local community park, where participants must jump from one marker to the next. Each participant starts at the first marker and can make jumps to other markers based on the distance provided for each marker. Sarah wants to calculate the minimum number of jumps it would take for a participant to reach the final marker.

You are tasked with writing a program that helps Sarah determine the minimum number of jumps required for a participant to reach the last marker.

Each element in the array represents the maximum number of steps a participant can jump forward from that marker. For example, if you're at marker i , you can jump to any marker within the range $[i+1, i + \text{nums}[i]]$.

Write a program to calculate the minimum number of jumps to reach the last marker.

Example 1:

Input: nums = 2 3 1 1 4

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2.
Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: nums = 2 3 0 1 4

Output: 2

Input Format

The input consists of space-separated integers representing the jump lengths from each index.

Output Format

The output prints an integer, representing the minimum number of jumps required to reach the last index.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2 3 1 1 4

Output: 2

Answer

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String line = sc.nextLine().trim();
        String[] parts = line.split(" ");

        int n = parts.length;
```

```

int[] nums = new int[n];

for (int i = 0; i < n; i++) {
    nums[i] = Integer.parseInt(parts[i]);
}

int jumps = 0;
int currentEnd = 0;
int farthest = 0;

for (int i = 0; i < n - 1; i++) {
    farthest = Math.max(farthest, i + nums[i]);

    if (i == currentEnd) {
        jumps++;
        currentEnd = farthest;
    }
}

System.out.println(jumps);
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Shoot the Kuruvi (Balloons)

There are some spherical balloons taped onto a flat wall that represent the XY-plane. The balloons are represented as a 2D integer array points, where points[i] = [xstart, xend] denotes a balloon whose horizontal diameter stretches between xstart and xend. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with xstart and xend is burst by an arrow shot at x if xstart ≤ x ≤ xend. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the number of array points, followed by the array points. Return the minimum number of arrows that must be shot to burst all balloons.

Example

Input

4

10 16

2 8

1 6

7 12

Output

2

Explanation

The balloons can be burst by 2 arrows:

Shoot an arrow at $x = 6$, bursting the balloons $[2,8]$ and $[1,6]$.

Shoot an arrow at $x = 11$, bursting the balloons $[10,16]$ and $[7,12]$.

Input

4

1 2

3 4

5 6

7 8

Output

4

Explanation

One arrow needs to be shot for each balloon for a total of 4 arrows.

Input

4

1 2

2 3

3 4

4 5

Output

2

Explanation

The balloons can be burst by 2 arrows:

Shoot an arrow at $x = 2$, bursting the balloons $[1,2]$ and $[2,3]$.

Shoot an arrow at $x = 4$, bursting the balloons $[3,4]$ and $[4,5]$.

Input Format

The first line of input consists of a single integer n , representing the Number of balloons

The next n lines contain two integers $xstart$ and $xend$, representing the start and end positions of each balloon

Output Format

The output prints a single integer, which represents the minimum number of arrows needed to burst all the balloons.

Refer to the output for the formatting specifications.

Sample Test Case

Input: 4

1 2

2 3

3 4

4 5

Output: 2

Answer

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[][] arr = new int[n][2];

        for (int i = 0; i < n; i++) {
            arr[i][0] = sc.nextInt(); // start
            arr[i][1] = sc.nextInt(); // end
        }

        // Sort by end coordinate
        Arrays.sort(arr, (a, b) -> Integer.compare(a[1], b[1]));

        int arrows = 1;
        int arrowPos = arr[0][1];

        for (int i = 1; i < n; i++) {
            if (arr[i][0] > arrowPos) {
                arrows++;
                arrowPos = arr[i][1];
            }
        }

        System.out.println(arrows);
    }
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 9_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers and a sum B, find all unique combinations in the array where the sum is equal to B. The same number may be chosen from the array any number of times to make B.

1. All numbers will be positive integers.
2. Elements in a combination ($a_1, a_2, \dots, a_k$) must be in non-descending order. (ie, $a_1 \leq a_2 \leq \dots \leq a_k$).
3. The combinations themselves must be sorted in ascending order.

Examples:

Input:

arr[] = {7,2,6,5}

B = 16

Output:

(2 2 2 2 2 2 2 2)

(2 2 2 2 2 6)

(2 2 2 5 5)

(2 2 5 7)

(2 2 6 6)

(2 7 7)

(5 5 6)

Input:

arr[] = {6,5,7,1,8,2,9,9,7,7,9}

B = 6

Output:

(1 1 1 1 1 1)

(1 1 1 1 2)

(1 1 2 2)

(1 5)

(2 2 2)

(6)

Input Format

The first line of input consists of space-separated positive integers representing the elements of the array.

The second line consists of a single positive integer representing the target sum B.

Output Format

The output prints "Combinations that sum to [B] are:" followed by the combinations in parentheses, each combination separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7 2 6 5

16

Output: Combinations that sum to 16 are:

(2 2 2 2 2 2 2) (2 2 2 2 2 6) (2 2 2 5 5) (2 2 5 7) (2 2 6 6) (2 7 7) (5 5 6)

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static void findCombinations(int[] arr, int target, List<Integer> current, int index)
    {
        if (target == 0) {
            System.out.print(" (");
            for (int num : current) {
                System.out.print(num + " ");
            }
            System.out.print(") ");
            return;
        }
```

```
        for (int i = index; i < arr.length; i++) {
            if (arr[i] > target) break;
```

```
            current.add(arr[i]);
            findCombinations(arr, target - arr[i], current, i); // reuse same element
            current.remove(current.size() - 1);
```

```
        }
    }
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

// Read full line of array input
String line = sc.nextLine();
String[] parts = line.trim().split("\\s+");

int[] arr = new int[parts.length];
for (int i = 0; i < parts.length; i++) {
    arr[i] = Integer.parseInt(parts[i]);
}

int B = sc.nextInt();

// Sort array
Arrays.sort(arr);

// Remove duplicates
int[] unique = Arrays.stream(arr).distinct().toArray();

System.out.println("Combinations that sum to " + B + " are:");
findCombinations(unique, B, new ArrayList<>(), 0);
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Imagine you're helping a friend plan gift packages with a total budget of N. Your task is to find all unique ways to spend the entire budget on different combinations of gift prices, ensuring no combination is repeated by rearranging the prices.

Note:

By unique, it means that no matter that no other composition can be expressed as a permutation of the generated composition. For eg. [1,2,1] and [1, 1, 2] are not unique. You need to print all combinations in non-decreasing order, for eg. [1, 2, 1] or [1,1,2] will be printed as [1,1,2]. The order of printing all the combinations should be in lexicographical order.

Use backtracking to solve the given problem.

Input Format

The input consists of a single integer, n, which represents the number to be broken down.

Output Format

The output consists of a list of lists, where each inner list represents a possible breakdown of the given number.

Each breakdown is represented as a list of integers that add up to the given number.

The breakdowns are printed in lexicographic order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

Output: [[1]]

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static List<List<Integer>> result = new ArrayList<>();
```

```
    static void backtrack(int remaining, int start, List<Integer> current) {
```

```
        if (remaining == 0) {
            result.add(new ArrayList<>(current));
            return;
        }
```

```
        for (int i = start; i <= remaining; i++) {
```

```
            current.add(i);
            backtrack(remaining - i, i, current); // allow reuse, enforce non-decreasing
            current.remove(current.size() - 1);
        }
```

```
    }
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();

    backtrack(n, 1, new ArrayList<>());

    // Print in required format
    System.out.print("[");
    for (int i = 0; i < result.size(); i++) {
        System.out.print("[");
        for (int j = 0; j < result.get(i).size(); j++) {
            System.out.print(result.get(i).get(j));
            if (j < result.get(i).size() - 1) {
                System.out.print(", ");
            }
        }
        System.out.print("]");
        if (i < result.size() - 1) {
            System.out.print(", ");
        }
    }
    System.out.print("]");
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rohan is designing a security system that generates all possible rearrangements of a given access code made up of distinct characters. He inputs a string representing the access code into the system. The system must display every possible permutation of the characters without repetition.

Write a program to ensure that all permutations of the given string are printed in separate lines.

Input Format

The input consists of a single line containing a string S, consisting of lowercase or upper-case alphabets.

Output Format

The output prints each permutation of the string on a new line.

The permutations should be printed in the order generated by swapping characters starting from the first position

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: ABC

Output: ABC

ACB

BAC

BCA

CBA

CAB

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static void permute(char[] arr, int index) {  
        if (index == arr.length) {  
            System.out.println(new String(arr));  
            return;  
        }  
    }
```

```
    for (int i = index; i < arr.length; i++) {
```

```
        char temp = arr[index];  
        arr[index] = arr[i];  
        arr[i] = temp;
```

```
        permute(arr, index + 1);
```

```

        temp = arr[index];
        arr[index] = arr[i];
        arr[i] = temp;
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String input = sc.nextLine().trim();

    permute(input.toCharArray(), 0);
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given a number K and string str of digits denoting a positive integer, build the largest number possible by performing swap operations on the digits of str at most K times.

Input Format

The first line of input consists of a positive integer K.

The second line of input consists of a string of digits of positive integers.

Output Format

The output displays the largest number possible by performing swap operations on the digits of str at most K times.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

254

Output: 524

Answer

```
import java.util.*;

public class Main {

    static String maxNumber;

    static void findMaximum(char[] arr, int k, int index) {
        // Update max at every step
        String current = new String(arr);
        if (current.compareTo(maxNumber) > 0) {
            maxNumber = current;
        }

        if (k == 0 || index == arr.length) return;

        int n = arr.length;

        // Find max digit from index to end
        char maxDigit = arr[index];
        for (int i = index + 1; i < n; i++) {
            if (arr[i] > maxDigit) {
                maxDigit = arr[i];
            }
        }

        // If no better digit, move forward without reducing k
        if (maxDigit == arr[index]) {
            findMaximum(arr, k, index + 1);
            return;
        }

        // Otherwise, try swaps
        for (int i = n - 1; i >= index; i--) {
            if (arr[i] == maxDigit) {

                // Swap
                char temp = arr[index];
                arr[index] = arr[i];
```

```

        arr[i] = temp;

        findMaximum(arr, k - 1, index + 1);

        // Backtrack
        temp = arr[index];
        arr[index] = arr[i];
        arr[i] = temp;
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int k = sc.nextInt();
    String str = sc.next();

    maxNumber = str;

    findMaximum(str.toCharArray(), k, 0);

    System.out.print(maxNumber);
}
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Mythili has a list of item prices and a customer's gift card balance. She needs to find a combination of two or more items whose total price exactly matches the gift card balance. If such a combination exists, she should print the item prices in the order they appear. If multiple valid combinations exist, she must print the first one found. If no valid combination is possible, print "No combination of items"

Write a function that takes the list of item prices, the number of items, and the gift card balance as input, and prints the result based on the above rules.

Input Format

The first line of input consists of an integer, representing the number of items in the store.

The second line consists of an array of integers separated by a space, representing the prices of the items.

The last line consists of an integer representing the customer's gift card balance.

Output Format

If there exists a combination of items whose prices sum up to the gift card balance, then print this combination.

If no such combination exists, print "No combination of items".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
50

Output: 20 30

Answer

```
import java.util.*;

public class Main {

    static boolean found = false;

    static void findCombination(int[] arr, int index, int target, List<Integer> current) {
        if (found) return;

        if (target == 0 && current.size() >= 2) {
            for (int num : current) {
                System.out.print(num + " ");
            }
            found = true;
        }
    }
}
```

```

        return;
    }

    if (index == arr.length) return;

    current.add(arr[index]);
    findCombination(arr, index + 1, target - arr[index], current);
    current.remove(current.size() - 1);

    findCombination(arr, index + 1, target, current);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int[] arr = new int[n];

    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }

    int target = sc.nextInt();

    List<Integer> current = new ArrayList<>();
    findCombination(arr, 0, target, current);

    if (!found) {
        System.out.print("No combination of items");
    }
}
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 6_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Yuvana, an avid programmer, is keen on mastering various sorting algorithms and has recently delved into Counting Sort. As a practice exercise, she is given an array of N integers where each element ranges from 1 to $N-1$. Some numbers may appear multiple times in the array. Her task is to identify and print all numbers that appear more than once. If no such duplicates are found, she should print -1.

Help Yuvana by implementing a function that efficiently finds and prints duplicate numbers using the Counting Sort technique.

Input Format

The first line of input consists of an integer N , representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints all the duplicate numbers that appear more than once in the array, each separated by a space.

If no duplicates are found, print -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

1 3 4 2 2

Output: 2

Answer

```
#include <stdio.h>
```

```
int main() {  
    int N;  
    scanf("%d", &N);  
  
    int arr[N];  
    int count[N]; // Since elements are in 1..N-1  
    for (int i = 0; i < N; i++) count[i] = 0;
```

```
    // Read array and populate count  
    for (int i = 0; i < N; i++) {  
        scanf("%d", &arr[i]);  
        count[arr[i]]++;  
    }
```

```
    int found = 0; // Flag to check if any duplicates exist  
    for (int i = 1; i <= N - 1; i++) {  
        if (count[i] > 1) {  
            printf("%d ", i);  
            found = 1;
```

```
}  
}  
  
if (!found)  
    printf("-1");  
  
    printf("\n");  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Anu is tasked with organizing a list of fruit names stored as strings. To ensure that the list is easy to navigate, she decides to sort the strings in ascending alphabetical order.

Being a fan of efficient sorting algorithms, Anu opts to use radix sort for this task. Help Anu with a program that can take a list of fruit names as input and output the sorted list.

Input Format

The first line of input consists of an integer n , representing the number of fruit names in the list.

The next n lines each consist of a string, representing the names of the fruits.

Output Format

The output prints n lines containing the sorted list of fruit names in ascending alphabetical order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
apple

banana
orange
grape
Output: apple
banana
grape
orange

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX_N 20
#define MAX_LEN 100

// Counting sort for a specific character position
void countSort(char arr[][MAX_LEN], int n, int pos) {
    char output[n][MAX_LEN];
    int count[256] = {0}; // ASCII character range

    // Count occurrences of each character at position pos
    for (int i = 0; i < n; i++) {
        int ch = pos < strlen(arr[i]) ? (unsigned char)arr[i][pos] : 0;
        count[ch]++;
    }

    // Accumulate counts
    for (int i = 1; i < 256; i++)
        count[i] += count[i - 1];

    // Build output array (stable sort)
    for (int i = n - 1; i >= 0; i--) {
        int ch = pos < strlen(arr[i]) ? (unsigned char)arr[i][pos] : 0;
        strcpy(output[count[ch] - 1], arr[i]);
        count[ch]--;
    }

    // Copy sorted strings back to original array
    for (int i = 0; i < n; i++)
        strcpy(arr[i], output[i]);
}
```



```

// Radix sort for strings
void radixSort(char arr[][MAX_LEN], int n) {
    int maxLen = 0;
    for (int i = 0; i < n; i++)
        if (strlen(arr[i]) > maxLen)
            maxLen = strlen(arr[i]);

    // Perform counting sort for each character position from right to left
    for (int pos = maxLen - 1; pos >= 0; pos--)
        countSort(arr, n, pos);
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[MAX_N][MAX_LEN];
    for (int i = 0; i < n; i++)
        scanf("%s", arr[i]);

    radixSort(arr, n);

    // Print sorted strings
    for (int i = 0; i < n; i++)
        printf("%s\n", arr[i]);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

30000 25000 40000 35000 20000 28000 29500

5

Output: 5th largest Salary: 28000

Answer

```
#include <stdio.h>
```

```
// Swap two integers
```

```
void swap(int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// Heapify a subtree rooted at index i in a max-heap of size n
```

```
void heapify(int arr[], int n, int i) {
```

```
    int largest = i;    // Initialize largest as root
```

```
    int left = 2 * i + 1; // Left child
```

```
    int right = 2 * i + 2; // Right child
```

```
    if (left < n && arr[left] > arr[largest])
```

```

        largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            swap(&arr[i], &arr[largest]);
            heapify(arr, n, largest);
        }
    }

    // Build a max-heap from array
    void buildMaxHeap(int arr[], int n) {
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(arr, n, i);
    }

    int main() {
        int N, K;
        scanf("%d", &N);

        int arr[N];
        for (int i = 0; i < N; i++)
            scanf("%d", &arr[i]);

        scanf("%d", &K);

        buildMaxHeap(arr, N);

        int heapSize = N;
        int kthLargest;

        // Extract max K times
        for (int i = 0; i < K; i++) {
            kthLargest = arr[0]; // Root is current largest
            arr[0] = arr[heapSize - 1]; // Move last element to root
            heapSize--;
            heapify(arr, heapSize, 0); // Heapify root
        }

        printf("%dth largest Salary: %d\n", K, kthLargest);
        return 0;
    }

```

}

Status : Correct

Marks : 10/10

4. Problem Statement

Manage a warehouse inventory with unique barcode numbers utilizing a specialized sorting algorithm designed for barcodes with exactly 16 digits. Input the number of barcode numbers (N) and their corresponding values.

Employ the Bucket Sort algorithm to distribute and sort the numbers based on their digits. Display the sorted barcode numbers, ensuring an efficient and organized warehouse inventory management system tailored for barcode systems with 16 digits.

Input Format

The first line contains an integer N, representing the number of barcode numbers to be sorted.

The following N lines contain exactly 16-digit long integers, each representing a unique barcode number.

Output Format

The output displays a single line containing "Sorted barcode numbers:" followed by the sorted barcode numbers in ascending order. Each barcode number is displayed on a separate line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

2345678901234567
1234567890123456
3456789012345678
5678901234567890
4567890123456789

Output: Sorted barcode numbers:

```
1234567890123456
2345678901234567
3456789012345678
4567890123456789
5678901234567890
```

Answer

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
```

```
int main() {
    int N;
    cin >> N;
```

```
    vector<long long> barcodes(N);
```

```
    for (int i = 0; i < N; i++) {
        cin >> barcodes[i];
    }
```

```
    // Radix Sort using Bucket Sort (10 buckets for digits 0-9)
    long long divisor = 1;
```

```
    for (int digit = 0; digit < 16; digit++) {
        queue<long long> buckets[10];
```

```
        // Place elements into buckets
        for (int i = 0; i < N; i++) {
            int digitValue = (barcodes[i] / divisor) % 10;
            buckets[digitValue].push(barcodes[i]);
        }
```

```
        // Collect elements back into array
        int index = 0;
        for (int i = 0; i < 10; i++) {
            while (!buckets[i].empty()) {
                barcodes[index++] = buckets[i].front();
                buckets[i].pop();
            }
```

```
}  
    divisor *= 10;  
}  
  
// Output result  
cout << "Sorted barcode numbers:" << endl;  
for (int i = 0; i < N; i++) {  
    cout << barcodes[i] << endl;  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 7_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Dhruv is working on a project where he needs to implement a Binary Search Tree (BST) data structure and perform various operations on it.

He wants to create a program that allows him to build a BST, traverse it in different orders (inorder, preorder, postorder), and exit the program when needed.

Help Dhruv by designing a program that fulfils his requirements.

Input Format

The first input consists of the choice.

If the choice is 1, enter the number of elements N and the elements inserted into

the tree, separated by a space in a new line.

If the choice is 2, print the in-order traversal.

If the choice is 3, print the pre-order traversal.

If the choice is 4, print the post-order traversal.

If the choice is 5, exit.

Output Format

The output prints the results based on the choice.

For choice 1, print "BST with N nodes is ready to use," where N is the number of nodes inserted.

For choice 2, print the inorder traversal of the BST after "BST Traversal in INORDER"

For choice 3, print the pre-order traversal of the BST after "BST Traversal in PREORDER"

For choice 4, print the post-order traversal of the BST after "BST Traversal in POSTORDER"

For choice 5, the program exits.

If the choice is greater than 5, print "Wrong choice".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

5

12 78 96 34 55

2

3

4

5

Output: BST with 5 nodes is ready to use

BST Traversal in INORDER

12 34 55 78 96

BST Traversal in PREORDER

12 78 34 55 96

BST Traversal in POSTORDER

55 34 96 78 12

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;
```

```
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};
```

```
Node* root = NULL;
```

```
// Insert into BST
```

```
Node* insert(Node* root, int data) {
    if (root == NULL) return new Node(data);
```

```
    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);
```

```
    return root;
}
```

```
// Traversals
```

```
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
```

```
        inorder(root->right);
    }
}
```

```
void preorder(Node* root) {
    if (root != NULL) {
        cout << root->data << " ";
        preorder(root->left);
        preorder(root->right);
    }
}
```

```
void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}
```

```
int main() {
    int choice;

    while (cin >> choice) {
        if (choice == 1) {
            int n;
            cin >> n;

            root = NULL; // reset tree

            for (int i = 0; i < n; i++) {
                int val;
                cin >> val;
                root = insert(root, val);
            }

            cout << "BST with " << n << " nodes is ready to use\n";
        }
        else if (choice == 2) {
            cout << "BST Traversal in INORDER\n";
            inorder(root);
            cout << "\n";
        }
    }
}
```

```

    }
    else if (choice == 3) {
        cout << "BST Traversal in PREORDER\n";
        preorder(root);
        cout << "\n";
    }
    else if (choice == 4) {
        cout << "BST Traversal in POSTORDER\n";
        postorder(root);
        cout << "\n";
    }
    else if (choice == 5) {
        break;
    }
    else {
        cout << "Wrong choice\n";
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Singh is studying binary search trees (BSTs) in his computer science class. He is particularly interested in finding the inorder predecessor of a given node in a BST.

The inorder predecessor of a node in a BST is the node with the largest value that is smaller than the given node's value.

Given a binary search tree and a node value, Singh needs your help to write a program that finds and outputs the in-order predecessor of the given node.

Input Format

The first line of input consists of an integer n , representing the number of keys to be inserted into the Binary Search Tree (BST).

The second line of input consists of n integers, representing the value of each key to be inserted into the BST, separated by a space.

The third line of input consists of an integer value key, representing the value of the node for which the inorder predecessor is to be found.

Output Format

The output displays "Inorder predecessor of X is Y", where Y is the value of the inorder predecessor of the node and X is the key.

If the node with the given key is not found in the BST, the output displays "Key not found".

If the given key has no inorder predecessor in the BST, the output displays "No inorder predecessor found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
20 70 90
90

Output: Inorder predecessor of 90 is 70

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;
```

```
    Node(int val) {
        data = val;
        left = right = NULL;
```

```
    }
};
```

// Insert into BST

```
Node* insert(Node* root, int data) {  
    if (root == NULL) return new Node(data);
```

```
    if (data < root->data)  
        root->left = insert(root->left, data);  
    else  
        root->right = insert(root->right, data);
```

```
    return root;
```

```
}
```

// Find maximum in subtree

```
Node* findMax(Node* root) {  
    while (root && root->right != NULL)  
        root = root->right;  
    return root;  
}
```

// Find inorder predecessor

```
void findPredecessor(Node* root, int key) {  
    Node* curr = root;  
    Node* predecessor = NULL;
```

```
    while (curr != NULL) {  
        if (key > curr->data) {  
            predecessor = curr; // possible predecessor  
            curr = curr->right;  
        }  
        else if (key < curr->data) {  
            curr = curr->left;  
        }  
        else {  
            // Node found  
            if (curr->left != NULL) {  
                predecessor = findMax(curr->left);  
            }  
        }  
    }
```

```
    if (predecessor != NULL)  
        cout << "Inorder predecessor of " << key << " is " << predecessor->data;  
    else
```

```

        cout << "No inorder predecessor found";
        return;
    }
}

// If key not found
cout << "Key not found";
}

int main() {
    int n;
    cin >> n;

    Node* root = NULL;

    for (int i = 0; i < n; i++) {
        int val;
        cin >> val;
        root = insert(root, val);
    }

    int key;
    cin >> key;

    findPredecessor(root, key);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Toy loves trees, especially alphabet trees! He wants to build his alphabet tree by inserting characters one by one. Given a sequence of characters, Toy needs to construct a binary search tree where each node contains a character. After constructing the tree, Toy wants to search for a specific character in the tree to see if it exists or not.

Write a program to help Toy construct his alphabet tree and perform

searches.

Input Format

The first line contains an integer, N, denoting the number of characters Toy wants to insert into the tree.

The second line contains N space-separated characters, representing the characters Toy wants to insert into the tree.

The third line contains a single character, V, representing the character Toy wants to search for in the tree.

Output Format

If the character V is found in the constructed alphabet tree, print "Character - [V] is Found".

If the character V is not found in the constructed alphabet tree, print "Character - [V] is Not Found".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
F A B C D
B

Output: Character - B is Found

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    char data;
    Node* left;
    Node* right;

    Node(char val) {
        data = val;
    }
};
```

```
        left = right = NULL;
    }
};
```

```
// Insert into BST
```

```
Node* insert(Node* root, char data) {
    if (root == NULL) return new Node(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}
```

```
// Search in BST
```

```
bool search(Node* root, char key) {
    if (root == NULL) return false;

    if (root->data == key)
        return true;
    else if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}
```

```
int main() {
    int n;
    cin >> n;
```

```
    Node* root = NULL;
```

```
    for (int i = 0; i < n; i++) {
        char ch;
        cin >> ch;
        root = insert(root, ch);
    }
```

```
    char key;
    cin >> key;
```



```

if (search(root, key)) {
    cout << "Character - " << key << " is Found";
} else {
    cout << "Character - " << key << " is Not Found";
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Jay is studying binary search trees (BSTs) as part of his computer science curriculum. He is particularly interested in understanding operations like insertion and deletion in a BST.

Jay wants to write a program that takes a sequence of characters as input, constructs a BST using those characters, deletes the minimum element from the BST, and then outputs the elements of the tree in sorted order.

Help Jay by designing a program that achieves this.

Input Format

The first line of input contains an integer n , denoting the number of characters to be inserted into the BST.

The second line contains n space-separated characters, each character representing a node to be inserted into the BST.

Output Format

The output displays a single line displaying the space-separated characters of the BST in sorted order after deleting the minimum element in the format:

"Inorder traversal after deleting the minimum element:"

"[char1] [char2] ...[char_n]"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

K J M V H F

Output: Inorder traversal after deleting the minimum element:

H J K M V

Answer

```
#include <iostream>
using namespace std;
```

```
struct Node {
    char data;
    Node* left;
    Node* right;
```

```
    Node(char val) {
        data = val;
        left = right = NULL;
    }
};
```

```
// Insert into BST
```

```
Node* insert(Node* root, char data) {
    if (root == NULL) return new Node(data);
```

```
    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);
```

```
    return root;
}
```

```
// Delete minimum node (leftmost)
```

```
Node* deleteMin(Node* root) {
    if (root == NULL) return NULL;
```

```
    // If no left child this is minimum
```

```

    if (root->left == NULL) {
        Node* temp = root->right;
        delete root;
        return temp;
    }

    root->left = deleteMin(root->left);
    return root;
}

// Inorder traversal
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

int main() {
    int n;
    cin >> n;

    Node* root = NULL;

    for (int i = 0; i < n; i++) {
        char ch;
        cin >> ch;
        root = insert(root, ch);
    }

    // Delete minimum element
    root = deleteMin(root);

    cout << "Inorder traversal after deleting the minimum element:\n";
    inorder(root);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 4_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neo is experimenting with text transformations and decides to use a stack to reverse the sequence of words in his sentence. He provides a line of text and expects the program to utilize a stack to reorder the words so that the last word appears first and the first word appears last.

Write a program to ensure this is done correctly using a stack-based approach.

Example

Input: I am Neo

Output: Neo am I

Input Format

The input consists of a single line of input containing a sentence with words separated by spaces.

Output Format

The output prints a single line of output containing the words of the sentence reversed in order, separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: Hello World

Output: World Hello

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 100
```

```
#define WORDS 50
```

```
char stack[WORDS][MAX];
```

```
int top = -1;
```

```
void push(char word[]) {  
    strcpy(stack[++top], word);  
}
```

```
char* pop() {  
    return stack[top--];  
}
```

```
int main() {  
    char str[MAX];
```

```

fgets(str, sizeof(str), stdin);
str[strcspn(str, "\n")] = '\0';

char *token = strtok(str, " ");

while (token != NULL) {
    push(token);
    token = strtok(NULL, " ");
}

while (top != -1) {
    printf("%s", pop());
    if (top != -1) {
        printf(" ");
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Give a string, remove duplicate letters from the string *s* such that each letter appears just once using Stack. The lexicographical order of your result must be the least of all feasible outcomes.

Input Format

The input consists of a string.

Output Format

The output displays the string in the lexicographical order with unique characters.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: bcabc

Output: abc

Answer

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
```

```
#define MAX 30
```

```
int main() {
    char s[MAX];
    scanf("%s", s);

    int freq[26] = {0};
    bool visited[26] = {false};
```

```
    int n = strlen(s);
```

```
    for (int i = 0; i < n; i++) {
        freq[s[i] - 'a']++;
    }
```

```
    char stack[MAX];
    int top = -1;
```

```
    for (int i = 0; i < n; i++) {
        char c = s[i];
        int idx = c - 'a';
```

```
        freq[idx]--;
```

```
        if (visited[idx])
            continue;
```

```

while (top >= 0 && stack[top] > c && freq[stack[top] - 'a'] > 0) {
    visited[stack[top] - 'a'] = false;
    top--;
}

stack[++top] = c;
visited[idx] = true;
}

for (int i = 0; i <= top; i++) {
    printf("%c", stack[i]);
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Mira is learning about palindromes and wants to build a program that can check whether a given string is a palindrome using a stack-based approach. A palindrome reads the same backward as forward.

Write a program to ensure that the task is done by pushing characters to a stack and comparing them in reverse order to determine if the string is a palindrome.

Input Format

The input contains a single line containing a string with space-separated characters.

Output Format

The output consists of ,

If the string is a palindrome, print "It's a palindrome!"

Otherwise, print "Not a palindrome"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam

Output: It's a palindrome!

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 105

int main() {
    char str[MAX];
    char stack[MAX];
    int top = -1;

    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    int len = strlen(str);

    for (int i = 0; i < len; i++) {
        if (str[i] != ' ') {
            stack[++top] = tolower(str[i]);
        }
    }

    for (int i = 0; i < len; i++) {
        if (str[i] != ' ') {
            if (tolower(str[i]) != stack[top--]) {
                printf("Not a palindrome");
                return 0;
            }
        }
    }
```

```
}  
}  
  
printf("It's a palindrome!");  
return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Gokul is working on a program to rearrange elements in a queue using an array. Given a queue of integers, he needs to modify it by interleaving its first half with the second half. Write a program to help Gokul achieve this task.

Note: Interleaving involves merging elements from two separate halves in an alternating fashion to create a new sequence.

Example

Input

6

1 2 3 7 9 5

Output

1 7 2 9 3 5

Explanation

The interleave rearranges elements in a queue by alternating between two halves. For instance, given input [1, 2, 3, 7, 9, 5], it splits into [1, 2, 3] and [7, 9, 5], then interleaves them as [1, 7, 2, 9, 3, 5]. It achieves this by splitting the queue into two halves and then merging them interleaved.

Input Format

The first line of the input consists of an integer n, representing the number of elements in the queue.

The second line consists of n space-separated integers, representing the queue elements.

Output Format

The output prints n space-separated integers, representing the rearranged queue after interleaving its first half with the second half.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

1 2 3 7 9 5

Output: 1 7 2 9 3 5

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[10];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int half = n / 2;
```

```
    // Interleave elements
```

```
    for (int i = 0; i < half; i++) {
```

```
        printf("%d %d ", arr[i], arr[i + half]);
```

```
    }
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Sanjay is learning about queues in his data structures class and he wants to write a program that counts the number of distinct elements in the queue. Your task is to guide him in this.

Example

Input:

7

45 33 29 45 45 46 39

Output:

5

Explanation:

The distinct elements in the queue are 45, 33, 29, 46, 39. So, the count is 5.

Input Format

The first line of input consists of an integer M, representing the number of elements in the queue.

The second line consists of M space-separated integers, representing the queue elements.

Output Format

The output prints an integer, representing the number of distinct elements in the queue.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

30 24 43 31 30

Output: 4

Answer

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int count = 0;

    // Count distinct elements
    for (int i = 0; i < n; i++) {
        int isDuplicate = 0;

        for (int j = 0; j < i; j++) {
            if (arr[i] == arr[j]) {
                isDuplicate = 1;
                break;
            }
        }

        if (!isDuplicate) {
            count++;
        }
    }

    printf("%d", count);

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 1_COD_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

In the field of cryptography, especially in public-key algorithms like RSA, it is crucial to identify pairs of numbers that are coprime — i.e., they do not share any common divisors other than 1.

Before generating secure keys, cryptographic systems often verify whether two large numbers are coprime. This process uses a classical method known as Euclid's Algorithm, which efficiently determines the greatest common divisor (GCD).

You are given a list of number pairs. Your task is to determine whether each pair of integers is coprime using Euclid's Algorithm. If the GCD of the two numbers is 1, output "Coprime". Otherwise, output "Not Coprime".

Example

Input:

3

15 28

12 35

8 8

Explanation:

$\text{GCD}(15, 28) = 1$ Coprime

$\text{GCD}(12, 35) = 1$ Coprime

$\text{GCD}(8, 8) = 8$ Not Coprime

Output:

Coprime

Coprime

Not Coprime

Input Format

- The first line contains an integer 't' — the number of pairs to check.
- Each of the next 't' lines contains two space-separated integers 'a' and 'b'.

Output Format

- For each pair, output a single line:
- "Coprime" if the two numbers are coprime
- "Not Coprime" otherwise

- Each output must appear on a new line, in the same order as the input pairs.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

17 29

Output: Coprime

Answer

```
#include <stdio.h>
```

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}
```

```
int main() {  
    int t;  
    scanf("%d", &t);  
  
    for (int i = 0; i < t; i++) {  
        int a, b;  
        scanf("%d %d", &a, &b);  
        if (gcd(a, b) == 1) {  
            printf("Coprime\n");  
        } else {  
            printf("Not Coprime\n");  
        }  
    }  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Sai Kishore is working on a tool that identifies special numbers in a range.

These numbers must have only two divisors: 1 and themselves. His task is to count how many of these special numbers exist below a given threshold N. Since N can be as large as one million, Sai Kishore needs an efficient way to find these numbers in a given range.

Sai Kishore decides to use a technique called Sieve of Eratosthenes, which is well-suited for efficiently identifying these special numbers.

Can you help him implement a program that counts how many such numbers exist below N?

Example:

Input:

10

Output:

4

Explanation:

The numbers less than 10 that have only two divisors (1 and themselves) are: 2, 3, 5, and 7. Thus, the output is 4.

Input Format

The input consists of a single integer N, representing the upper bound for the search.

Output Format

The output should be a single integer, representing the count of special numbers less than N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

Output: 4

Answer

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int N;
    scanf("%d", &N);

    bool isPrime[1000001] = {false};

    // Initialize all numbers as prime
    for (int i = 0; i <= N; i++) {
        isPrime[i] = true;
    }

    // Sieve of Eratosthenes
    for (int p = 2; p * p <= N; p++) {
        if (isPrime[p] == true) {
            // Mark all multiples of p as not prime
            for (int j = p * p; j <= N; j += p) {
                isPrime[j] = false;
            }
        }
    }

    // Count primes less than N (exclude 1)
    int count = 0;
    for (int i = 2; i < N; i++) {
        if (isPrime[i]) {
            count++;
        }
    }

    printf("%d\n", count);
    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Arjun is analyzing a numerical range to find special pairs of prime numbers. He is given a range defined by two integers L and R, and a target value T. Within the range [L,R], Arjun wants to identify all pairs of prime numbers whose sum equals T.

Write a program to ensure that all such valid prime pairs are printed in ascending order.

Input Format

The first line contains three space-separated integers:

- L starting value of the range
- R ending value of the range
- T target sum

Output Format

The output prints each valid pair of prime numbers whose sum equals T, one pair per line.

Each line should contain two space-separated integers representing a valid prime pair.

If no such pairs exist, no output should be printed.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 1 40 10

Output: 3 7
5 5

Answer

```
#include <stdio.h>
#include <stdbool.h>
```

```
int main() {
    int L, R, T;
    scanf("%d %d %d", &L, &R, &T);
```

```

// Sieve up to max(R, T) to handle all possible pairs
int maxN = (R > T) ? R : T;
bool isPrime[1000001] = {false};

// Initialize
for (int i = 0; i <= maxN; i++) {
    isPrime[i] = true;
}
isPrime[0] = isPrime[1] = false;

// Sieve of Eratosthenes
for (int p = 2; p * p <= maxN; p++) {
    if (isPrime[p]) {
        for (int j = p * p; j <= maxN; j += p) {
            isPrime[j] = false;
        }
    }
}

// Find pairs (p1, p2) where L ≤ p1 ≤ p2 ≤ R and p1 + p2 = T
for (int p1 = L; p1 <= R; p1++) {
    int p2 = T - p1;
    if (p1 <= p2 && p2 <= R && isPrime[p1] && isPrime[p2]) {
        printf("%d %d\n", p1, p2);
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Bella is developing a number puzzle game where each level involves simplifying equations using number theory. In one level, she comes across two large positive integers and is asked to reduce them using their greatest common divisor (GCD).

However, the puzzle has an extra twist:

She must express the GCD as a linear combination of the two numbers – that is, she needs to find two integers x and y such that: $\text{GCD}(a, b) = a * x + b * y$

Help Bella write a program that, given two integers a and b , finds:

The GCD of a and b The coefficients ' x ' and ' y ' that satisfy the linear combination condition

Example

Input

48 18

Output:

6 -1 3

Explanation:

The GCD of 48 and 18 is 6.

Using the Extended Euclidean Algorithm, we can find integers $x = -1$ and $y = 3$ such that:

$$48 \times (-1) + 18 \times 3 = -48 + 54 = 6$$

This satisfies Bézout's Identity: $\text{GCD}(48, 18) = 48x + 18y = 6$.

Input Format

- The input consists of a single line with two space-separated integers ' a ' and ' b '

Output Format

The output prints three space-separated integers in a single line:

- The first integer is the GCD of a and b .
- The second integer is the coefficient x such that $a * x + b * y = \text{GCD}$.
- The third integer is the coefficient y such that $a * x + b * y = \text{GCD}$.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 45 21

Output: 3 1 -2

Answer

```
import java.util.Scanner;
```

```
public class Main {  
    public static int extendedGcd(int a, int b, int[] x, int[] y) {  
        if (a == 0) {  
            x[0] = 0;  
            y[0] = 1;  
            return b;  
        }  
    }
```

```
        int[] x1 = new int[1];  
        int[] y1 = new int[1];  
        int gcd = extendedGcd(b % a, a, x1, y1);
```

```
        x[0] = y1[0] - (b / a) * x1[0];  
        y[0] = x1[0];
```

```
        return gcd;  
    }
```

```
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int a = sc.nextInt();  
        int b = sc.nextInt();
```

```
        int[] x = new int[1];  
        int[] y = new int[1];  
        int gcd = extendedGcd(a, b, x, y);
```

```
        System.out.println(gcd + " " + x[0] + " " + y[0]);
```

```
        sc.close();
```

```
    }  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 3_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Suresh is working on a text-processing system where he needs to efficiently count how many times a smaller pattern appears inside a larger string. To achieve fast pattern matching, he uses the Z-Algorithm, with the Z-array as the main data structure.

Given multiple test cases, each containing a text string and a pattern string, write a program to ensure that the task is done by computing the number of occurrences of the pattern in the text using the Z-algorithm approach.

For example:

If S = "ababa", and P = "ab", then "ab" occurs twice in "ababa".

Input Format

The first line of input contains T, representing the number of test cases.

For each test case:

Line 1 Contains two space separated integers N and M representing the length of the text and pattern respectively.

Line 2: Contains string S of length N representing the text.

Line 3: A string P of length M representing the pattern.

Output Format

For each test case, print a single integer on a new line indicating the number of times the pattern occurs in the text.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
10 2
ababababa
ab

Output: 4

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 205
```

```
// Compute Z-array for a string
void computeZArray(char* s, int Z[]) {
    int n = strlen(s);
    int L = 0, R = 0;
    Z[0] = 0;
    for (int i = 1; i < n; i++) {
```

```

    if (i > R) {
        L = R = i;
        while (R < n && s[R - L] == s[R])
            R++;
        Z[i] = R - L;
        R--;
    } else {
        int k = i - L;
        if (Z[k] < R - i + 1)
            Z[i] = Z[k];
        else {
            L = i;
            while (R < n && s[R - L] == s[R])
                R++;
            Z[i] = R - L;
            R--;
        }
    }
}
}
}

```

```

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int N, M;
        scanf("%d %d", &N, &M);

        char S[101], P[101];
        scanf("%s", S);
        scanf("%s", P);

        char concat[MAX]; // pattern + $ + text
        sprintf(concat, "%s$%s", P, S);

        int Z[MAX];
        computeZArray(concat, Z);

        int count = 0;
        int lenConcat = strlen(concat);
        for (int i = M + 1; i < lenConcat; i++) {

```

```

    if (Z[i] == M)
        count++;
    }

    printf("%d\n", count);
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is working as an NLP engineer to improve the smart assistant chatbot in his company. His team wants the chatbot to understand user emotions better, especially when users repeat or emphasize words.

They noticed that palindromic patterns (like "wow", "pop", "madam") in messages can help detect such emotions or expressions.

To help with this, Arjun is asked to write a program that finds out how many palindromic substrings are centered at each character in the user's message.

Since the message could be long, Arjun decides to use Manacher's Algorithm for fast and efficient processing.

Example:

Input

bananas

Output

1 1 3 5 3 1 1

Explanation

At the first character 'b', the only palindrome is "b" itself length 1
At 'a', only "a" is a palindrome length 1
At 'n', we get "ana" which is a palindrome length 3
At 'a', we get "anana" which is a palindrome length 5
At 's', we get "sanas" which is a palindrome length 3
At 'a', we get "asana" which is a palindrome length 1
At 's', we get "sas" which is a palindrome length 1

centered at 'n' length 3At the middle 'a', we get "anana" which is a bigger palindrome length 5At the next 'n', again "ana" is a palindrome length 3At the second 'a', only "a" length 1Finally at 's', only "s" length 1

Input Format

The first line of input contains a string s, representing a message typed by a user in the chatbot conversation.

Output Format

The output prints the length of the longest palindromic substring centered at each character in the input string, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ab

Output: 1 1

Answer

```
#include <stdio.h>
#include <string.h>
```

```
// Preprocess string by inserting '#' between characters and sentinels
```

```
void preprocess(char* s, char* t) {
    int n = strlen(s);
    int j = 0;
    t[j++] = '^'; // start sentinel
    for (int i = 0; i < n; i++) {
        t[j++] = '#';
        t[j++] = s[i];
    }
    t[j++] = '#';
    t[j++] = '$'; // end sentinel
    t[j] = '\0';
}
```

```
void manacher(char* s, int* res) {
    char t[2*1000 + 3]; // max transformed length
```

```

preprocess(s, t);
int n = strlen(t);
int p[n];
int center = 0, right = 0;

for (int i = 1; i < n - 1; i++) {
    int mirror = 2*center - i;
    p[i] = 0;
    if (i < right)
        p[i] = (p[mirror] < (right - i)) ? p[mirror] : (right - i);

    // expand around center i
    while (t[i + 1 + p[i]] == t[i - 1 - p[i]])
        p[i]++;

    if (i + p[i] > right) {
        center = i;
        right = i + p[i];
    }
}

```

```

// extract lengths of palindromes centered at each character in original string
int len = strlen(s);
for (int i = 0; i < len; i++) {
    res[i] = p[2*i + 2]; // mapping to original character positions
}
}

```

```

int main() {
    char s[1001];
    scanf("%1000s", s);
    int n = strlen(s);
    int res[1000];

    manacher(s, res);

    for (int i = 0; i < n; i++) {
        printf("%d", res[i]);
        if (i != n - 1) printf(" ");
    }
    printf("\n");
}

```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Given a text `txt` $[0 \dots N-1]$ and a pattern `pat` $[0 \dots M-1]$, write a function `search (char pat [], char txt [])` that prints all occurrences of `pat []` in `txt []`. You may assume that $N > M$. Use zero-based indexing while returning the indices.

Example:

Input: `txt [] = "THIS IS A TEST TEXT"`, `pat [] = "TEST"`

Output: Pattern found at index 10

Input: `txt [] = "AABAACAADAABAABA"`, `pat [] = "AABA"`

Output: Pattern found at index 0, Pattern found at index 9, Pattern found at index 12

Input Format

The first line of input consists of a string `txt`, in which the pattern is to be searched.

The second line of input consists of a string `pat`, the pattern that needs to be found in the text.

Output Format

For each occurrence of the pattern in the text, print "Pattern found at index X" in a new line, where X the indices where the pattern is found within the text string.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: THIS IS A TEST TEXT
TEST

Output: Pattern found at index 10

Answer

```
#include <stdio.h>
#include <string.h>

void search(char pat[], char txt[]) {
    int N = strlen(txt);
    int M = strlen(pat);

    for (int i = 0; i <= N - M; i++) {
        int j;

        for (j = 0; j < M; j++) {
            if (txt[i + j] != pat[j]) {
                break;
            }
        }

        if (j == M) {
            printf("Pattern found at index %d\n", i);
        }
    }
}

int main() {
    char txt[51], pat[11];

    fgets(txt, sizeof(txt), stdin);
    fgets(pat, sizeof(pat), stdin);

    txt[strcspn(txt, "\n")] = '\0';
    pat[strcspn(pat, "\n")] = '\0';

    search(pat, txt);

    return 0;
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Ritika is a software engineer working on an AI-based spam detection system for a popular email client. To efficiently scan email content, she uses the Boyer–Moore string matching algorithm, specifically the Bad Character Heuristic as the main data structure/technique.

Given an email message and a list of spam keywords, write a program to ensure that the task is done by detecting all keywords present in the email (case-insensitive), reporting their first occurrence positions, and computing a spam score based on the number of detected keywords.

Input Format

The first line of input contains a string representing the email content (may contain spaces and special characters).

The second line contains an integer n , representing the number of spam keywords.

The next n lines each contain a spam keyword.

Output Format

The first line prints a string "Detected Spam Keywords:"

For each detected keyword, print "<index>: "<keyword>" at position <starting_index>" where indexing starts from 0.

The final line prints "Spam Score: x/y " where x is the number of detected keywords and y is the total number of keywords.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: Buy this book now and get another free!

3
buy
now
free

Output: Detected Spam Keywords:

1: "buy" at position 0
2: "now" at position 14
3: "free" at position 34
Spam Score: 3/3

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAX 1005
```

```
void toLowerCase(char *str) {
    for (int i = 0; str[i]; i++) {
        str[i] = tolower(str[i]);
    }
}
```

```
void badCharHeuristic(char *str, int size, int badchar[256]) {
    for (int i = 0; i < 256; i++)
        badchar[i] = -1;

    for (int i = 0; i < size; i++)
        badchar[(int)str[i]] = i;
}
```

```
int boyerMooreSearch(char *txt, char *pat) {
    int m = strlen(pat);
    int n = strlen(txt);
```

```
    int badchar[256];
    badCharHeuristic(pat, m, badchar);
```

```
    int shift = 0;
```

```
    while (shift <= (n - m)) {
```

```

int j = m - 1;

while (j >= 0 && pat[j] == txt[shift + j])
    j--;

if (j < 0) {
    return shift;
} else {
    int badIndex = txt[shift + j];
    int move = j - badchar[badIndex];
    if (move < 1)
        move = 1;
    shift += move;
}
}
return -1;
}

```

```

int main() {
    char txt[MAX];
    int n;

    fgets(txt, sizeof(txt), stdin);
    txt[strcspn(txt, "\n")] = '\0';

    scanf("%d", &n);
    getchar();

    char keywords[100][105];
    char txtLower[MAX];

    strcpy(txtLower, txt);
    toLowerCase(txtLower);

    int foundCount = 0;
    int positions[100];
    char detected[100][105];

    for (int i = 0; i < n; i++) {
        fgets(keywords[i], sizeof(keywords[i]), stdin);
        keywords[i][strcspn(keywords[i], "\n")] = '\0';
    }
}

```

```
char patLower[105];
strcpy(patLower, keywords[i]);
toLowerCase(patLower);

int pos = boyerMooreSearch(txtLower, patLower);

if (pos != -1) {
    positions[foundCount] = pos;
    strcpy(detected[foundCount], keywords[i]);
    foundCount++;
}
}

if (foundCount == 0) {
    printf("No spam keywords detected.\n");
} else {
    printf("Detected Spam Keywords:\n");
    for (int i = 0; i < foundCount; i++) {
        printf("%d: \"%s\" at position %d\n", i + 1, detected[i], positions[i]);
    }
}

printf("Spam Score: %d/%d", foundCount, n);

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 6_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Alex is working on a data analysis project and needs to find specific elements within a dataset. He has an array of integers where some numbers may be repeated. His goal is to identify the Kth smallest element in this array. The challenge is that the array can contain duplicate values, so finding the exact Kth smallest element requires careful consideration.

Alex has decided to use a counting sort approach to solve this problem. Given an array of integers and a number K, where K is smaller than the size of the array, he needs to find and print the Kth smallest element in the array.

Help Alex by implementing a function that efficiently finds and prints the Kth smallest element using the counting sort approach.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

The third line of input consists of an integer K, representing the position of the smallest element to find.

Output Format

The output prints the Kth smallest element in the array as "Kth Smallest element is <smallest element>".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

7 10 4 3 20 15

3

Output: Kth Smallest element is 7

Answer

```
# Read input
```

```
N = int(input())
```

```
arr = list(map(int, input().split()))
```

```
K = int(input())
```

```
# Counting sort approach
```

```
# Step 1: Initialize a count array for numbers from 1 to 100
```

```
count = [0] * 101 # since elements are from 1 to 100
```

```
# Step 2: Count occurrences of each number
```

```
for num in arr:
```

```
    count[num] += 1
```

```
# Step 3: Find Kth smallest element
```

```
cumulative_count = 0
```

```
kth_smallest = 0
```

```
for i in range(1, 101):  
    cumulative_count += count[i]  
    if cumulative_count >= K:  
        kth_smallest = i  
        break
```

```
# Print the result  
print(f"Kth Smallest element is {kth_smallest}")
```

Status : Correct

Marks : 10/10

2. Problem Statement

Rohan is developing a text analysis tool and wants to sort a list of words based on the count of vowels in each word. He believes that sorting the words based on vowel count will help him analyze the text more effectively.

To achieve this, he plans to use radix sort. Assist Rohan with a suitable program to implement this sorting method for his project.

Input Format

The first line of input consists of an integer n , representing the number of words.

The next n lines each consist of a string, representing the words.

Output Format

The output prints n lines containing the sorted list of words based on the count of vowels in each word.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
another

stuck
door
you

Output: stuck
door
you
another

Answer

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
```

```
// Function to count vowels in a word
```

```
int countVowels(string s) {
    int count = 0;
    for (char c : s) {
        c = tolower(c);
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
            count++;
        }
    }
    return count;
}
```

```
int main() {
    int n;
    cin >> n;
```

```
    vector<string> words(n);
    for (int i = 0; i < n; i++) {
        cin >> words[i];
    }
```

```
// Find maximum vowel count
```

```
int maxVowels = 0;
vector<int> vowelCount(n);
```

```
for (int i = 0; i < n; i++) {
    vowelCount[i] = countVowels(words[i]);
    if (vowelCount[i] > maxVowels) {
```

```

        maxVowels = vowelCount[i];
    }
}

// Create buckets (0 to maxVowels)
vector<vector<string>> buckets(maxVowels + 1);

// Place words into buckets (stable)
for (int i = 0; i < n; i++) {
    buckets[vowelCount[i]].push_back(words[i]);
}

// Output sorted words
for (int i = 0; i <= maxVowels; i++) {
    for (string word : buckets[i]) {
        cout << word << endl;
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

In a manufacturing plant, each product is assigned an integer production code. To streamline quality control, the products have to be sorted in ascending order based on the last digit of their production codes.

Develop a program that uses Radix Sort to sort the products according to their code's last digit.

Input Format

The first line consists of an integer n , representing the number of products.

The second line consists of n space-separated integers each representing a product code.

Output Format

The output prints the sorted product codes based on the last digit in ascending order, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

24570 61064 65391

Output: 24570 65391 61064

Answer

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
```

```
int main() {
```

```
    int n;
    cin >> n;
```

```
    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
```

```
    // Create 10 buckets for digits 0-9
    queue<int> buckets[10];
```

```
    // Place elements into buckets based on last digit
    for (int i = 0; i < n; i++) {
        int lastDigit = arr[i] % 10;
        buckets[lastDigit].push(arr[i]);
    }
```

```
    // Collect elements back in order
    for (int i = 0; i < 10; i++) {
        while (!buckets[i].empty()) {
            cout << buckets[i].front() << " ";
            buckets[i].pop();
        }
```

```
}  
}  
return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Meera, a librarian, wants to organize a list of book titles based on how many words each title contains. If two titles have the same number of words, they should be arranged in lexicographical (dictionary) order.

Write a program to sort the given book titles using a bucket-based approach, where titles are first grouped by word count and then alphabetically sorted within each group.

Input Format

The first line contains an integer N, representing the number of book titles.

The next N lines: Each line contains a book title (may contain spaces and special characters).

Output Format

The output prints "Sorted book titles:"

Then print the sorted book titles, each on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
Harry Potter and the Philosopher's Stone
To Kill a Mockingbird
The Great Gatsby
1984

Output: Sorted book titles:

1984

The Great Gatsby

To Kill a Mockingbird

Harry Potter and the Philosopher's Stone

Answer

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>
using namespace std;
```

```
// Function to count words in a title
int countWords(const string &title) {
    stringstream ss(title);
    string word;
    int count = 0;
    while (ss >> word) {
        count++;
    }
    return count;
}
```

```
int main() {
    int N;
    cin >> N;
    cin.ignore(); // to consume newline after integer input
```

```
    vector<string> titles(N);
    int maxWords = 0;
```

```
    // Read titles and find max word count
    for (int i = 0; i < N; i++) {
        getline(cin, titles[i]);
        int wc = countWords(titles[i]);
        if (wc > maxWords) {
            maxWords = wc;
        }
    }
```

```

// Create buckets based on word count
vector<vector<string>> buckets(maxWords + 1);

for (int i = 0; i < N; i++) {
    int wc = countWords(titles[i]);
    buckets[wc].push_back(titles[i]);
}

// Output result
cout << "Sorted book titles:" << endl;

for (int i = 0; i <= maxWords; i++) {
    if (!buckets[i].empty()) {
        sort(buckets[i].begin(), buckets[i].end()); // lexicographical sort
        for (string &title : buckets[i]) {
            cout << title << endl;
        }
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 1_COD_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Haritha is working on a project to identify pairs of numbers in a given range $[L, R]$ where each number in the pair is divisible only by 1 and itself.

She needs to find pairs of such numbers that differ by exactly 2. These pairs of numbers should be displayed efficiently within the range.

Help her to implement the task.

Input Format

The input consists of two integers L and R separated by a new line.

Output Format

The output prints each pair of numbers within the range $[L, R]$ on a new line, in the format: $a\ b$ where a and b are numbers that differ by exactly 2 and are

divisible only by 1 and themselves.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 30

60

Output: 41 43

Answer

```
#include <stdio.h>
#include <stdbool.h>

bool isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}

int main() {
    int L, R;
    scanf("%d %d", &L, &R);

    for (int i = L; i <= R - 2; i++) {
        if (isPrime(i) && isPrime(i + 2)) {
            printf("%d %d\n", i, i + 2);
        }
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Neha is designing a secret code system for her cybersecurity class. She discovers that certain numbers with exactly two distinct divisors (1 and themselves) create the most secure keys when their reversed digit sum is divisible by a given validation number K.

Neha needs to find all such numbers up to a specified limit N that meet both conditions. These numbers are crucial for the security of her code system. Given the large range of numbers, she needs an efficient solution to identify them.

Help her write a program to find all such numbers $\leq N$ that meet this criteria.

Input Format

The input consists of two space separated integers N and K

Output Format

The output prints all numbers that meet both conditions. Each number should be printed space-separated.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 10 1

Output: 2 3 5 7

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
bool isPrime(int n) {  
    if (n <= 1) return false;  
    if (n <= 3) return true;  
    if (n % 2 == 0 || n % 3 == 0) return false;
```

```
    for (int i = 5; i * i <= n; i += 6) {
```

```

    if (n % i == 0 || n % (i + 2) == 0) return false;
}
return true;
}

```

```

int reverseDigitSum(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}

```

```

int main() {
    int N, K;
    scanf("%d %d", &N, &K);

```

```

    bool first = true;
    for (int i = 2; i <= N; i++) {
        if (isPrime(i)) {
            int revSum = reverseDigitSum(i);
            if (revSum % K == 0) {
                if (!first) printf(" ");
                printf("%d", i);
                first = false;
            }
        }
    }
    printf("\n");

```

```

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Ashwin is working on a system to identify numbers within a given range [L, R] that have a special property: they are divisible only by 1 and themselves.

Ashwin needs to find pairs of such numbers where both numbers are rearrangements of each other's digits. Ashwin must efficiently identify all such pairs in the given range.

Help him to implement the task.

Input Format

The input consists of two space separated integers L and R

Output Format

The output prints each pair of numbers within the range [L, R] that satisfy both conditions: they are divisible only by 1 and themselves, and they are rearrangements of each other's digits. Each pair is printed in the format: p1 p2, where p1 and p2 are numbers that are rearrangements of each other's digits.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 10 100

Output: 13 31

17 71

37 73

79 97

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <string.h>
```

```
bool isPrime(int n) {
```

```
    if (n <= 1) return false;
```

```
    if (n <= 3) return true;
```

```
    if (n % 2 == 0 || n % 3 == 0) return false;
```

```
    for (int i = 5; i * i <= n; i += 6) {
```

```
        if (n % i == 0 || n % (i + 2) == 0) return false;
```

```
    }
```

```

    return true;
}

bool isAnagram(int n1, int n2) {
    char s1[10], s2[10];
    sprintf(s1, "%d", n1);
    sprintf(s2, "%d", n2);

    int len1 = strlen(s1);
    int len2 = strlen(s2);

    if (len1 != len2) return false;

    int count[10] = {0};
    for (int i = 0; i < len1; i++) {
        count[s1[i] - '0']++;
        count[s2[i] - '0']--;
    }

    for (int i = 0; i < 10; i++) {
        if (count[i] != 0) return false;
    }
    return true;
}

int main() {
    int L, R;
    scanf("%d %d", &L, &R);

    for (int i = L; i <= R; i++) {
        if (isPrime(i)) {
            for (int j = i + 1; j <= R; j++) {
                if (isPrime(j) && isAnagram(i, j)) {
                    printf("%d %d\n", i, j);
                }
            }
        }
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Suresh is studying prime numbers and is interested in identifying a special category of primes. Given a number N, he wants to find all prime numbers less than or equal to N whose sum of digits is also a prime number. To solve this efficiently, Suresh decides to use the Sieve of Eratosthenes for prime number generation.

Write a program to ensure that all such prime numbers are printed in increasing order.

Input Format

The input consists of single integer N, representing the upper limit of the range.

Output Format

The output prints all prime numbers less than or equal to N whose digit sum is also a prime number.

The numbers must be printed in ascending order, separated by a single space.

If no such numbers exist, print nothing.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 50

Output: 2 3 5 7 11 23 29 41 43 47

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <math.h>
```

```
bool isPrimeSum(int n) {
    int sum = 0;
```

```

int temp = n;
while (temp > 0) {
    sum += temp % 10;
    temp /= 10;
}

if (sum <= 1) return false;
if (sum <= 3) return true;
if (sum % 2 == 0 || sum % 3 == 0) return false;

for (int i = 5; i * i <= sum; i += 6) {
    if (sum % i == 0 || sum % (i + 2) == 0) return false;
}
return true;
}

```

```

int main() {
    int N;
    scanf("%d", &N);

```

```

    bool prime[1000001];
    for (int i = 0; i <= N; i++) {
        prime[i] = true;
    }
    prime[0] = prime[1] = false;

```

```

    for (int p = 2; p * p <= N; p++) {
        if (prime[p]) {
            for (int j = p * p; j <= N; j += p) {
                prime[j] = false;
            }
        }
    }
}

```

```

bool first = true;
for (int i = 2; i <= N; i++) {
    if (prime[i] && isPrimeSum(i)) {
        if (!first) printf(" ");
        printf("%d", i);
        first = false;
    }
}
}

```

```
printf("\n");  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 3_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Suresh is working on a text-processing system where he needs to efficiently count how many times a smaller pattern appears inside a larger string. To achieve fast pattern matching, he uses the Z-Algorithm, with the Z-array as the main data structure.

Given multiple test cases, each containing a text string and a pattern string, write a program to ensure that the task is done by computing the number of occurrences of the pattern in the text using the Z-algorithm approach.

For example:

If S = "ababa", and P = "ab", then "ab" occurs twice in "ababa".

Input Format

The first line of input contains T, representing the number of test cases.

For each test case:

Line 1 Contains two space separated integers N and M representing the length of the text and pattern respectively.

Line 2: Contains string S of length N representing the text.

Line 3: A string P of length M representing the pattern.

Output Format

For each test case, print a single integer on a new line indicating the number of times the pattern occurs in the text.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
10 2
ababababa
ab

Output: 4

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 205
```

```
// Compute Z-array for a string
void computeZArray(char* s, int Z[]) {
    int n = strlen(s);
    int L = 0, R = 0;
    Z[0] = 0;
    for (int i = 1; i < n; i++) {
```

```

    if (i > R) {
        L = R = i;
        while (R < n && s[R - L] == s[R])
            R++;
        Z[i] = R - L;
        R--;
    } else {
        int k = i - L;
        if (Z[k] < R - i + 1)
            Z[i] = Z[k];
        else {
            L = i;
            while (R < n && s[R - L] == s[R])
                R++;
            Z[i] = R - L;
            R--;
        }
    }
}
}
}

```

```

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int N, M;
        scanf("%d %d", &N, &M);

        char S[101], P[101];
        scanf("%s", S);
        scanf("%s", P);

        char concat[MAX]; // pattern + $ + text
        sprintf(concat, "%s$%s", P, S);

        int Z[MAX];
        computeZArray(concat, Z);

        int count = 0;
        int lenConcat = strlen(concat);
        for (int i = M + 1; i < lenConcat; i++) {

```



```
    if (Z[i] == M)
        count++;
    }

    printf("%d\n", count);
}

return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is working as an NLP engineer to improve the smart assistant chatbot in his company. His team wants the chatbot to understand user emotions better, especially when users repeat or emphasize words.

They noticed that palindromic patterns (like "wow", "pop", "madam") in messages can help detect such emotions or expressions.

To help with this, Arjun is asked to write a program that finds out how many palindromic substrings are centered at each character in the user's message.

Since the message could be long, Arjun decides to use Manacher's Algorithm for fast and efficient processing.

Example:

Input

bananas

Output

1 1 3 5 3 1 1

Explanation

At the first character 'b', the only palindrome is "b" itself length 1
At 'a', only "a" is a palindrome length 1
At 'n', we get "ana" which is a palindrome length 3
At 'a', we get "anana" which is a palindrome length 5
At 's', we get "sanas" which is a palindrome length 3
At 'a', we get "asana" which is a palindrome length 1
At 's', we get "sas" which is a palindrome length 1

centered at 'n' length 3At the middle 'a', we get "anana" which is a bigger palindrome length 5At the next 'n', again "ana" is a palindrome length 3At the second 'a', only "a" length 1Finally at 's', only "s" length 1

Input Format

The first line of input contains a string s, representing a message typed by a user in the chatbot conversation.

Output Format

The output prints the length of the longest palindromic substring centered at each character in the input string, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ab

Output: 1 1

Answer

```
#include <stdio.h>
#include <string.h>
```

```
// Preprocess string by inserting '#' between characters and sentinels
```

```
void preprocess(char* s, char* t) {
    int n = strlen(s);
    int j = 0;
    t[j++] = '^'; // start sentinel
    for (int i = 0; i < n; i++) {
        t[j++] = '#';
        t[j++] = s[i];
    }
    t[j++] = '#';
    t[j++] = '$'; // end sentinel
    t[j] = '\0';
}
```

```
void manacher(char* s, int* res) {
    char t[2*1000 + 3]; // max transformed length
```

```

preprocess(s, t);
int n = strlen(t);
int p[n];
int center = 0, right = 0;

for (int i = 1; i < n - 1; i++) {
    int mirror = 2*center - i;
    p[i] = 0;
    if (i < right)
        p[i] = (p[mirror] < (right - i)) ? p[mirror] : (right - i);

    // expand around center i
    while (t[i + 1 + p[i]] == t[i - 1 - p[i]])
        p[i]++;

    if (i + p[i] > right) {
        center = i;
        right = i + p[i];
    }
}

```

```

// extract lengths of palindromes centered at each character in original string
int len = strlen(s);
for (int i = 0; i < len; i++) {
    res[i] = p[2*i + 2]; // mapping to original character positions
}
}

```

```

int main() {
    char s[1001];
    scanf("%1000s", s);
    int n = strlen(s);
    int res[1000];

    manacher(s, res);

    for (int i = 0; i < n; i++) {
        printf("%d", res[i]);
        if (i != n - 1) printf(" ");
    }
    printf("\n");
}

```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Given a text `txt` $[0 \dots N-1]$ and a pattern `pat` $[0 \dots M-1]$, write a function `search (char pat [], char txt [])` that prints all occurrences of `pat []` in `txt []`. You may assume that $N > M$. Use zero-based indexing while returning the indices.

Example:

Input: `txt [] = "THIS IS A TEST TEXT"`, `pat [] = "TEST"`

Output: Pattern found at index 10

Input: `txt [] = "AABAACAADAABAABA"`, `pat [] = "AABA"`

Output: Pattern found at index 0, Pattern found at index 9, Pattern found at index 12

Input Format

The first line of input consists of a string `txt`, in which the pattern is to be searched.

The second line of input consists of a string `pat`, the pattern that needs to be found in the text.

Output Format

For each occurrence of the pattern in the text, print "Pattern found at index X" in a new line, where X the indices where the pattern is found within the text string.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: THIS IS A TEST TEXT
TEST

Output: Pattern found at index 10

Answer

```
#include <stdio.h>
#include <string.h>

void search(char pat[], char txt[]) {
    int N = strlen(txt);
    int M = strlen(pat);

    for (int i = 0; i <= N - M; i++) {
        int j;

        for (j = 0; j < M; j++) {
            if (txt[i + j] != pat[j]) {
                break;
            }
        }

        if (j == M) {
            printf("Pattern found at index %d\n", i);
        }
    }
}

int main() {
    char txt[51], pat[11];

    fgets(txt, sizeof(txt), stdin);
    fgets(pat, sizeof(pat), stdin);

    txt[strcspn(txt, "\n")] = '\0';
    pat[strcspn(pat, "\n")] = '\0';

    search(pat, txt);

    return 0;
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Ritika is a software engineer working on an AI-based spam detection system for a popular email client. To efficiently scan email content, she uses the Boyer–Moore string matching algorithm, specifically the Bad Character Heuristic as the main data structure/technique.

Given an email message and a list of spam keywords, write a program to ensure that the task is done by detecting all keywords present in the email (case-insensitive), reporting their first occurrence positions, and computing a spam score based on the number of detected keywords.

Input Format

The first line of input contains a string representing the email content (may contain spaces and special characters).

The second line contains an integer n , representing the number of spam keywords.

The next n lines each contain a spam keyword.

Output Format

The first line prints a string "Detected Spam Keywords:"

For each detected keyword, print "<index>: "<keyword>" at position <starting_index>" where indexing starts from 0.

The final line prints "Spam Score: x/y " where x is the number of detected keywords and y is the total number of keywords.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: Buy this book now and get another free!

3
buy
now
free

Output: Detected Spam Keywords:

1: "buy" at position 0
2: "now" at position 14
3: "free" at position 34
Spam Score: 3/3

Answer

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 1005
```

```
void toLowerCase(char *str) {
    for (int i = 0; str[i]; i++) {
        str[i] = tolower(str[i]);
    }
}
```

```
void badCharHeuristic(char *str, int size, int badchar[256]) {
    for (int i = 0; i < 256; i++)
        badchar[i] = -1;

    for (int i = 0; i < size; i++)
        badchar[(int)str[i]] = i;
}
```

```
int boyerMooreSearch(char *txt, char *pat) {
    int m = strlen(pat);
    int n = strlen(txt);

    int badchar[256];
    badCharHeuristic(pat, m, badchar);
```

```
    int shift = 0;
```

```
    while (shift <= (n - m)) {
```

```

int j = m - 1;

while (j >= 0 && pat[j] == txt[shift + j])
    j--;

if (j < 0) {
    return shift;
} else {
    int badIndex = txt[shift + j];
    int move = j - badchar[badIndex];
    if (move < 1)
        move = 1;
    shift += move;
}
}
return -1;
}

```

```

int main() {
    char txt[MAX];
    int n;

    fgets(txt, sizeof(txt), stdin);
    txt[strcspn(txt, "\n")] = '\0';

    scanf("%d", &n);
    getchar();

    char keywords[100][105];
    char txtLower[MAX];

    strcpy(txtLower, txt);
    toLowerCase(txtLower);

    int foundCount = 0;
    int positions[100];
    char detected[100][105];

    for (int i = 0; i < n; i++) {
        fgets(keywords[i], sizeof(keywords[i]), stdin);
        keywords[i][strcspn(keywords[i], "\n")] = '\0';
    }
}

```



```

char patLower[105];
strcpy(patLower, keywords[i]);
toLowerCase(patLower);

int pos = boyerMooreSearch(txtLower, patLower);

if (pos != -1) {
    positions[foundCount] = pos;
    strcpy(detected[foundCount], keywords[i]);
    foundCount++;
}
}

if (foundCount == 0) {
    printf("No spam keywords detected.\n");
} else {
    printf("Detected Spam Keywords:\n");
    for (int i = 0; i < foundCount; i++) {
        printf("%d: \"%s\" at position %d\n", i + 1, detected[i], positions[i]);
    }
}

printf("Spam Score: %d/%d", foundCount, n);

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 12_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 12

Section 1 : MCQ

1. The first step in the naive greedy algorithm is _____.

Answer

adding flows with higher values

Status : Wrong

Marks : 0/1

2. Suppose you have coins of denominations 1,3 and 4. You use a greedy algorithm, in which you choose the largest denomination coin which is not greater than the remaining sum. For which of the following sums, will the algorithm doesn't produce an optimal answer?

Answer

10

Status : Correct

Marks : 1/1

3. Which data structure is most commonly used to efficiently implement a greedy algorithm that repeatedly selects the minimum (or maximum) element at each step?

Answer

Heap

Status : Correct

Marks : 1/1

4. What is a greedy algorithm?

Answer

An algorithm that makes the best choice at each step, without considering the overall impact

Status : Correct

Marks : 1/1

5. Which of the following problems is typically solved using a greedy algorithm?

Answer

Finding the shortest path in a graph.

Status : Correct

Marks : 1/1

6. What is the primary advantage of greedy algorithms over other approaches for certain problems?

Answer

Greedy algorithms are easy to implement

Status : Correct

Marks : 1/1

7. Which of the following is a disadvantage of greedy algorithm?

Answer

It may not always find the globally optimal solution

Status : Correct

Marks : 1/1

8. Which of the following approaches should be used to find the solution to the activity selection problem?

Answer

Greedy approach

Status : Correct

Marks : 1/1

9. Suppose two activities A and B, have start and finish times as S_A , F_A and S_B , F_B respectively. Both activities are said to be compatible, under which of the following conditions?

Answer

$S_A \leq F_B$ or $S_B \leq F_A$

Status : Correct

Marks : 1/1

10. In the activity selection problem, if two activities have the same finish time, which one is selected?

Answer

The one with the later start time

Status : Wrong

Marks : 0/1

11. Consider the following number of activities with their start and finish time given below. In which sequence will the activity be selected in order to maximize the number of activities, without any conflicts?

Answer

A1, A2, A5

Status : Correct

Marks : 1/1

12. Which of the following is a characteristic of the greedy algorithm?

Answer

It makes the best choice at each step by considering only the local problem state.

Status : Correct

Marks : 1/1

13. What is the primary reason why the greedy algorithm does not work for some optimization problems?

Answer

It does not reconsider previous decisions and makes choices based only on the current state.

Status : Correct

Marks : 1/1

14. In the greedy approach using a stack, what condition do we check before popping from the stack

Answer

If the top is greater than current digit and $k > 0$

Status : Correct

Marks : 1/1

15. Why is sorting not used in this greedy solution?

Answer

We need to maintain digit order

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 8_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Maya is developing a route planning system for a large theme park. Each attraction in the park is represented as a vertex in a directed graph, and each pathway connecting attractions is a directed edge. To help visitors explore all possible paths between two attractions, Maya needs a function that finds all possible routes from a given starting attraction to a destination attraction using Depth-First Search (DFS) traversal.

Write a program to help Maya list all such possible routes from source to destination.

Input Format

The first line contains two space-separated integers V and E, representing the number of attractions and roads in the city.

Each of the next E lines contains two integers, u and v, separated by a space, representing a road from landmark u to attractions v.

The last line contains two integers, s, and d, separated by a space, representing the source attraction and the destination attraction. for which routes need to be found.

Output Format

The output prints all the possible routes from the source attraction to the destination attraction, each route on a separate line.

Each route should be represented as a space-separated sequence of attractions visited in the order of traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4 6

0 1

0 2

0 3

2 0

2 1

1 3

2 3

Output: 2 0 1 3

2 0 3

2 1 3

Answer

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int adj[MAX][MAX];
```

```
int visited[MAX];
```

```
int path[MAX];
```

```
int V;
```

```

// DFS to print all paths
void dfs(int u, int d, int pathLen) {
    visited[u] = 1;
    path[pathLen] = u;
    pathLen++;

    // If destination reached print path
    if (u == d) {
        for (int i = 0; i < pathLen; i++) {
            printf("%d", path[i]);
            if (i < pathLen - 1)
                printf(" ");
        }
        printf("\n");
    } else {
        // Explore neighbors
        for (int v = 0; v < V; v++) {
            if (adj[u][v] && !visited[v]) {
                dfs(v, d, pathLen);
            }
        }
    }
}

// Backtrack
visited[u] = 0;
}

```

```

int main() {
    int E;
    scanf("%d %d", &V, &E);

    // Initialize
    for (int i = 0; i < V; i++) {
        visited[i] = 0;
        for (int j = 0; j < V; j++) {
            adj[i][j] = 0;
        }
    }
}

```

```

// Read edges (directed)
for (int i = 0; i < E; i++) {

```



```
int u, v;
scanf("%d %d", &u, &v);
adj[u][v] = 1;
}

int s, d;
scanf("%d %d", &s, &d);

dfs(s, d, 0);

return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Daniel is working on a navigation system for a city, where each intersection is represented as a vertex, and the roads between them are edges. He needs to calculate the shortest path between two intersections using the Breadth-First Search (BFS) algorithm.

Given the adjacency matrix of the city's road network, Daniel needs to find the shortest path from a source intersection to a destination intersection. If there are multiple shortest paths, the path with the smaller indexed intersections should be preferred.

Can you assist Daniel in implementing the solution for this problem?

Example

Input:

5

0 1 1 0 0

1 0 1 0 0

1 1 0 1 0

0 0 1 0 1

0 0 0 1 0

0 4

Output:

3

0 2 3 4

Explanation:

The graph has 5 vertices (0 to 4).

The adjacency matrix represents an undirected graph where 1 indicates an edge between two vertices.

The starting vertex is 0, and the ending vertex is 4.

The BFS starts from vertex 0, exploring its neighbors 1 and 2.

BFS continues to vertex 2, discovering vertex 3.

From vertex 3, it reaches the destination vertex 4.

The shortest path found is 0 2 3 4, with a length of 3.

The program prints the path length (3) and the path itself (0 2 3 4).

Input Format

The first line contains an integer N, the number of intersections (vertices) in the city.

The next N lines each contain N integers, representing the adjacency matrix. If there is a road between intersection i and intersection j, the matrix entry will be 1, otherwise 0.

The last line contains two integers, source and goal, representing the source and destination intersections (indexed from 0).

Output Format

The first line should print the length of the shortest path from the source to the goal intersection.

The second line should print the path from the source to the goal, including both

the source and the goal.

If the source or goal index is invalid, print "Not found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0 1 1 0 0

1 0 1 0 0

1 1 0 1 0

0 0 1 0 1

0 0 0 1 0

0 4

Output: 3

0 2 3 4

Answer

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int adj[MAX][MAX];
```

```
int visited[MAX];
```

```
int parent[MAX];
```

```
int queue[MAX];
```

```
int main() {
```

```
    int V;
```

```
    scanf("%d", &V);
```

```
    // Read adjacency matrix
```

```
    for (int i = 0; i < V; i++) {
```

```
        for (int j = 0; j < V; j++) {
```

```
            scanf("%d", &adj[i][j]);
```

```
        }
```

```
    }
```

```
    int src, dest;
```

```
scanf("%d %d", &src, &dest);
```

```
// Initialize
```

```
for (int i = 0; i < V; i++) {  
    visited[i] = 0;  
    parent[i] = -1;  
}
```

```
int front = 0, rear = 0;
```

```
// Start BFS
```

```
queue[rear++] = src;  
visited[src] = 1;
```

```
while (front < rear) {  
    int u = queue[front++];
```

```
// Traverse neighbors in increasing order
```

```
for (int v = 0; v < V; v++) {  
    if (adj[u][v] == 1 && !visited[v]) {  
        visited[v] = 1;  
        parent[v] = u;  
        queue[rear++] = v;  
    }  
}
```

```
}
```

```
// If destination not reachable
```

```
if (!visited[dest]) {  
    printf("No path exists");  
    return 0;  
}
```

```
// Reconstruct path
```

```
int path[MAX], count = 0;  
int cur = dest;
```

```
while (cur != -1) {  
    path[count++] = cur;  
    cur = parent[cur];  
}
```

```
// Print path in correct order
for (int i = count - 1; i >= 0; i--) {
    printf("%d ", path[i]);
}

return 0;
}
```

Status : Wrong

Marks : 0/10

3. Problem Statement

Sarah is working on a traffic management system for a city's road network. She needs to find the shortest cycle in the road network, where each intersection is represented as a vertex and each road between intersections is a directed edge with a specific travel time (weight). The goal is to identify the shortest cycle in the road network, where a cycle is defined as a path that starts and ends at the same intersection, and the length of the cycle is the total travel time of all roads in the cycle.

Can you assist Sarah by writing a program to find the length of the shortest cycle in the city's road network?

Example1

Input:

4

4

0 1 2

1 2 3

2 0 4

3 1 5

Output:

Length of the shortest cycle: 9

Explanation:

After exploring all starting vertices, the code checks for cycles by examining the distances. If an edge leads back to the starting vertex, it means there is a cycle. In this case, edge 1 (vertex 1) leads back to the starting vertex 0. The code calculates the length of the cycle by adding the distances of the vertices along the cycle: $2 (0 \rightarrow 1) + 3 (1 \rightarrow 2) + 4 (2 \rightarrow 0) = 9$.

Therefore, the output is "Length of the shortest cycle: 9".

Example2

Input:

3

2

0 1 2

1 2 3

Output:

No cycle was found in the graph.

Explanation:

After exploring all starting vertices, the code checks for cycles by examining the distances. If an edge leads back to the starting vertex, it means there is a cycle. However, in this case, there are no edges that lead back to the starting vertex. Hence, there is no cycle in the graph.

Therefore, the output is "No cycle found in the graph."

Input Format

The first line contains an integer v , representing the number of intersections (vertices) in the city's road network.

The second line contains an integer e , representing the number of roads (edges) in the city.

The next e lines each contain three integers: source, destination, and weight,

representing a directed road from intersection source to intersection destination with a travel time (weight).

Output Format

If a cycle exists, the output prints "Length of the shortest cycle: X", where X is the length of the shortest cycle.

If no cycle exists, then the output prints "No cycle found in the graph."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

4

0 1 2

1 2 3

2 0 4

3 1 5

Output: Length of the shortest cycle: 9

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int V, E;
```

```
int adj[MAX][MAX];
```

```
// Dijkstra to find shortest cycle starting from src
```

```
int dijkstra(int src) {
```

```
    int dist[MAX];
```

```
    int visited[MAX];
```

```
    // Initialize
```

```
    for (int i = 0; i < V; i++) {
```

```
        dist[i] = INT_MAX;
```

```
        visited[i] = 0;
```

```
    }
```

```

dist[src] = 0;

for (int count = 0; count < V - 1; count++) {
    int u = -1, min = INT_MAX;

    // Find minimum distance vertex
    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            u = i;
        }
    }

    if (u == -1) break;

    visited[u] = 1;

    for (int v = 0; v < V; v++) {
        if (adj[u][v] != 0 && !visited[v] && dist[u] != INT_MAX) {
            if (dist[u] + adj[u][v] < dist[v]) {
                dist[v] = dist[u] + adj[u][v];
            }
        }
    }
}

int minCycle = INT_MAX;

// Check cycle back to src
for (int i = 0; i < V; i++) {
    if (i != src && adj[i][src] != 0 && dist[i] != INT_MAX) {
        int cycleLen = dist[i] + adj[i][src];
        if (cycleLen < minCycle) {
            minCycle = cycleLen;
        }
    }
}

return minCycle;
}

```



```

int main() {
    scanf("%d", &V);
    scanf("%d", &E);

    // Initialize adjacency matrix
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            adj[i][j] = 0;

    // Read edges
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        adj[u][v] = w;
    }

    int ans = INT_MAX;

    // Try all vertices as starting point
    for (int i = 0; i < V; i++) {
        int res = dijkstra(i);
        if (res < ans) {
            ans = res;
        }
    }

    if (ans == INT_MAX)
        printf("No cycle found in the graph.");
    else
        printf("Length of the shortest cycle: %d", ans);

    return 0;
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

Ravi is developing a network simulator that checks if a given undirected graph contains a cycle. His task is to build a program that determines whether any loop exists in the graph and also counts the number of nodes

visited during the cycle detection process.

Write a program to ensure that the task is done by printing whether a cycle exists (1 or 0) and reporting the total number of recursive visits during traversal.

Input Format

The first line of the input consists of an integer V, representing the number of vertices.

The second line of the input consists of an integer E, representing the number of edges.

The next E lines each contain two space-separated integers u and v, representing an edge between vertices u and v.

Output Format

The first line prints 1 if the graph contains a cycle, else it prints 0

The second line prints "Number of nodes visited: X" where X is the number of nodes visited during the cycle detection function.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

3

0 1

1 2

2 0

Output: 1

Number of nodes visited: 3

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int adj[MAX][MAX];
int visited[MAX];
int V;
int visitCount = 0;

// DFS function
int dfs(int node, int parent) {
    visited[node] = 1;
    visitCount++; // count this node visit
```

```
    for (int i = 0; i < V; i++) {
        if (adj[node][i]) {
            if (!visited[i]) {
                if (dfs(i, node))
                    return 1;
            }
            else if (i != parent) {
                return 1; // cycle found
            }
        }
    }
    return 0;
}
```

```
int main() {
    int E;
    scanf("%d", &V);
    scanf("%d", &E);

    // Initialize
    for (int i = 0; i < V; i++) {
        visited[i] = 0;
        for (int j = 0; j < V; j++) {
            adj[i][j] = 0;
        }
    }
```

```
    // Read edges (undirected)
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
```

```

        adj[v][u] = 1;
    }

    int hasCycle = 0;

    // Handle disconnected graph
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            if (dfs(i, -1)) {
                hasCycle = 1;
                break;
            }
        }
    }

    printf("%d\n", hasCycle);
    printf("Number of nodes visited: %d", visitCount);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 1_COD_SB

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Arjun is working on a project where he needs to analyze a series of integers within a specified range $[A, B]$. The project involves identifying certain numbers that are not divisible by any smaller positive integers other than 1 and themselves. These numbers are crucial for the next step of his analysis, as they have unique mathematical properties.

Arjun has been asked to implement an efficient way to find all such numbers in a given range. Given that the range can be large, he needs to ensure the solution is optimized.

Can you help him implement this task using the Simple Sieve of Eratosthenes?

Input Format

The first line of input consists of an integer A, representing the starting range.

The second line of input consists of an integer B, representing the ending range.

Output Format

The output prints a space-separated list of numbers that are not divisible by any smaller positive integers other than 1 and themselves. within the range [A, B].

Refer to the sample output for formatting specifications

Sample Test Case

Input: 2

10

Output: 2 3 5 7

Answer

```
#include <stdio.h>
#include <stdbool.h>
int main(){
    int A, B;
    scanf("%d", &A);
    scanf("%d", &B);
    bool prime[B+1];
    for(int i=0;i<=B;i++){
        prime[i]=true;
    }
    prime[0]=false;
    prime[1]=false;
    for(int i=2;i*i<=B;i++){
        if(prime[i]){
            for(int j=i*i;j<=B;j+=i){
                prime[j]=false;
            }
        }
    }
    for(int i=A;i<=B;i++){
        if(prime[i]){
            printf("%d",i);
        }
    }
}
```

```
}  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Pradeep Narwal is analyzing multiple numerical ranges to identify prime numbers within each range. He is given several ranges defined by a starting number A and an ending number B. Using the Sieve of Eratosthenes, Pradeep wants to efficiently determine all prime numbers that lie within each range.

Write a program to ensure that all prime numbers between A and B (inclusive) are printed for every given range.

Input Format

The first line of input consists of an integer Q, representing the number of ranges.

The next Q lines each contain two integers A and B , representing the start and end of each range.

Output Format

For each range, print all prime numbers between A and B (inclusive) in a single line.

Prime numbers must be printed in ascending order, separated by a single space.

If a range contains no prime numbers, print an empty line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2

1 5
10 15
Output: 2 3 5
11 13

Answer

```
#include <iostream>
using namespace std;

const int MAX=1000000;

int main(){
    bool prime[MAX+1];
    for(int i=0;i<=MAX;i++)prime[i]=true;
    prime[0]=prime[1]=false;
    for(int i=2;i*i<=MAX;i++)
        if(prime[i])
            for(int j=i*i;j<=MAX;j+=i)
                prime[j]=false;
    int Q,A,B;
    cin>>Q;

    while(Q--){
        cin>>A>>B;
        for(int i=A;i<=B;i++)
            if(prime[i]) cout<<i<<" ";
        cout<<endl;
    }
    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Joanna is developing a utility tool for a math club's quiz event. One of the rounds involves finding the greatest common divisor (GCD) of a list of numbers. Instead of computing it manually for each pair, she wants to automate the task using code.

She decides to use Euclid's Algorithm, a well-known and efficient method to compute the GCD of two numbers. For a list of numbers, she will apply the algorithm iteratively across all elements in the array.

Help Joanna implement a program that takes an array of positive integers and computes the GCD of the entire array using Euclid's Algorithm.

Make sure the program also handles special cases:

If the array is empty, return -1. If the array has only one number, return that number as the GCD.

Example

Input:

3

12 18 24

Output:

6

Explanation:

$\text{GCD}(12, 18) = 6$ $\text{GCD}(6, 24) = 6$ Final result = 6

Input Format

The first line of input contains an integer n , representing the number of elements in the array.

The second line contains n space-separated positive integers, each representing an element of the array.

Output Format

The output consists of a single line:

- If the array contains two or more elements, print a single integer representing the GCD of all array elements.
- If the array contains only one element, print that element as the GCD.

- If the array is empty, print -1 as a convention to indicate undefined GCD.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2

8 12

Output: 4

Answer

```
#include <stdio.h>
int gcd(int a,int b){
    while(b!=0){
        int temp=b;
        b=a%b;
        a=temp;
    }
    return a;
}
int main(){
    int n;
    scanf("%d",&n);
    if(n==0){
        printf("-1");
        return 0;
    }
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    if(n==1){
        printf("%d",arr[0]);
        return 0;
    }
    int result=arr[0];
    for(int i=1;i<n;i++){
        result=gcd(result,arr[i]);
    }
    printf("%d",result);
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Raychel is working on a calendar synchronization tool where she needs to determine how often two repeating tasks will coincide. Each task occurs after a fixed number of days, and she must find the least number of days after which both tasks align again — in other words, the least common multiple (LCM) of the two intervals.

Raychel knows that the most efficient way to compute the LCM is by first calculating the greatest common divisor (GCD) using Euclid's Algorithm, and then using that result to derive the LCM. She also wants to make sure the program works for large inputs without running into overflow errors.

Help Raychel implement a program that calculates the LCM of two non-negative integers by first using Euclid's Algorithm to compute their GCD, and then deriving the LCM from it.

Example

Input:

15 20

Explanation:

$\text{GCD}(15, 20) = 5$
 $\text{LCM} = (15 / 5) \times 20 = 3 \times 20 = 60$

Output:

60

Input Format

The input consists of a single line containing two space-separated integers 'a' and 'b', representing the two input numbers.

Output Format

The output consists of a single line:

- If both 'a' and 'b' are non-zero, print a single integer — the least number that is divisible by both.
- If either 'a' or b is '0', print 0 as the result since no common multiple exists in such cases.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 15 20

Output: 60

Answer

```
#include <stdio.h>
long long gcd(long long a,long long b){
    while(b!=0){
        long long temp=b;
        b=a%b;
        a=temp;
    }
    return a;
}
int main(){
    long long a,b;
    scanf("%lld %lld",&a,&b);
    if(a==0||b==0){
        printf("0");
        return 0;
    }
    long long g=gcd(a,b);
    long long lcm=(a/g)*b;
    printf("%lld",lcm);
    return 0;
}
```

}

Status : Correct

Marks : 10/10

5. Problem Statement:

A software company is developing a mathematical tool for students to help them understand factorization and modular arithmetic. As part of the tool, they need a function that calculates the product of all positive divisors (factors) of a given integer, with the result taken modulo 1000000007. The tool is intended to assist in solving mathematical problems and providing insights into number theory.

Your task is to implement a function that computes the product of all factors of a given integer n , and then display the result modulo 1000000007.

Examples :

Input: 12

Output: 1728

$$1 * 2 * 3 * 4 * 6 * 12 = 1728$$

Input Format

The input consists of a single integer n .

Output Format

The output displays a single integer which is the product of all factors of n modulo 1000000007.

Refer to the sample output for better understanding.

Sample Test Case

Input: 12

Output: 1728

Answer

```
#include <stdio.h>
int main(){
    int n;
    long long product=1;
    const long long MOD=1000000007;
    scanf("%d",&n);
    for(int i=1;i<=n;i++){
        if(n%i==0){
            product=(product*i)%MOD;
        }
    }
    printf("%lld",product);
    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 12_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 42.5

Section 1 : Coding

1. Problem Statement

Given an array of non-negative integers, the task is to find the minimum number of elements such that their sum should be greater than the sum of the rest of the elements of the array.

Example:

Input: arr[] = [3 , 1 , 7, 1]

Output: 1

Explanation: Smallest subset is {7}. Sum of this subset is greater than the sum of all other elements left after removing subset {7} from the array

Input: arr[] = [2 , 1 , 2]

Output: 2

Explanation: Smallest subset is {2 , 1}. Sum of this subset is greater than the sum of all other elements left after removing subset {2 , 1} from the array

Input Format

The first line contains an Integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single integer representing the minimum number of elements whose sum is greater than the sum of the remaining elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3 1 7 1

Output: 1

Answer

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        Integer[] arr = new Integer[n];

        int total = 0;

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            total += arr[i];
        }
```



```

Arrays.sort(arr, Collections.reverseOrder());

int pickedSum = 0;
int count = 0;

for (int i = 0; i < n; i++) {
    pickedSum += arr[i];
    count++;

    if (pickedSum > total - pickedSum) {
        break;
    }
}

System.out.println(count);
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an array `prices[]` of n different candies, where `prices[i]` represents the price of the i th candy and all prices are distinct. The store has an offer where for every candy you buy, you can get up to k other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies.

Note: In both cases, you must take the maximum number of free candies possible during each purchase.

Examples:

Input: `prices[] = [3, 2, 1, 4]`, $k = 2$

Output: `[3, 7]`

Explanation: For minimum cost, buy the candies costing 1 and 2. for maximum cost, buy the candies costing 4 and 3.

Input: `prices[] = [3, 2, 1, 4, 5]`, $k = 4$

Output: [1, 5]

Explanation: For minimum cost, buy the candy costing 1. for maximum cost, buy the candy costing 5.

Input Format

The first line contains an Integer n, representing the number of candies.

The second line contains n space-separated integers, representing the prices of the candies.

The third line contains an integer k, representing the number of free candies per purchase.

Output Format

The output prints two integers separated by a space:

minCost maxCost

representing the minimum and maximum money needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3 2 1 4

2

Output: 3 7

Answer

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();  
        int[] arr = new int[n];
```

```

for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

int k = sc.nextInt();

// MIN COST
Arrays.sort(arr);
int minCost = 0;
for (int i = 0; i < n; i += (k + 1)) {
    minCost += arr[i];
}

// MAX COST
Integer[] arr2 = new Integer[n];
for (int i = 0; i < n; i++) {
    arr2[i] = arr[i];
}

Arrays.sort(arr2, Collections.reverseOrder());

int maxCost = 0;
for (int i = 0; i < n; i += (k + 1)) {
    maxCost += arr2[i];
}

System.out.println(minCost + " " + maxCost);
}
}

```

Status : Partially correct

Marks : 2.5/10

3. Problem Statement

Given a fraction represented by two integers — a numerator n and a denominator d — express the fraction as a sum of unique unit fractions (fractions with numerator 1). This representation is known as the Egyptian Fraction.

Write a program to ensure that the given fraction is converted into its Egyptian Fraction form.

Input Format

The input contains two space-separated integers n and d, representing the numerator and denominator of the fraction.

Output Format

The output prints the Egyptian Fraction representation of the given fraction in the form of unit fractions separated by +.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 14

Output: 1/3 + 1/11 + 1/231

Answer

```
import java.util.*;

public class Main {

    static int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int d = sc.nextInt();

        StringBuilder sb = new StringBuilder();

        while (n != 0) {
            int x = (int) Math.ceil((double) d / n);
```

```

if (sb.length() > 0) sb.append(" + ");
sb.append("1/").append(x);

// update fraction: n/d - 1/x
n = n * x - d;
d = d * x;

int g = gcd(n, d);
if (g != 0) {
    n /= g;
    d /= g;
}
}
System.out.print(sb.toString());
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Emily is building a data compression feature in her file organizing system. She wants filenames to have no repeated characters and appear in the smallest possible order so users can locate files easily. For example, if a corrupted filename is "cbacdcabc", she wants it cleaned up as "acdb" — which preserves one instance of each letter, keeps relative order, and is the smallest alphabetically.

This function helps ensure filenames are clean, minimal, and alphabetically organized!

Example 1:

Input: s = "bcabc"

Output: "ABC"

Example 2:

Input: s = "cbacdcabc"

Output: "acdb"

Input Format

The first line of input contains a string num representing a non-negative integer.

The second line contains an integer k representing how many digits to remove.

Output Format

The output prints the string representing the smallest possible number after removing exactly k digits.

Leading zeros should be removed, except if the number is zero itself.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: bcabc

Output: abc

Answer

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        String s = sc.next();
```

```
        int[] lastIndex = new int[26];  
        boolean[] visited = new boolean[26];  
        Stack<Character> st = new Stack<>();
```

```
        // store last occurrence  
        for (int i = 0; i < s.length(); i++) {  
            lastIndex[s.charAt(i) - 'a'] = i;  
        }
```

```
        for (int i = 0; i < s.length(); i++) {  
            char c = s.charAt(i);
```

```

int idx = c - 'a';

if (visited[idx]) continue;

while (!st.isEmpty() &&
       st.peek() > c &&
       lastIndex[st.peek() - 'a'] > i) {
    visited[st.pop() - 'a'] = false;
}

st.push(c);
visited[idx] = true;
}

StringBuilder sb = new StringBuilder();
for (char c : st) sb.append(c);

System.out.print(sb.toString());
}
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Given the arrival and departure times of all trains that reach a railway station, find the minimum number of platforms required for the station so that no train is kept waiting. Consider that all the trains arrive on the same day and leave on the same day.

Arrival and departure times can never be the same for a train, but we can have the arrival time of one train equal to the departure time of the other. At any given instance, the same platform cannot be used for both the departure of a train and the arrival of another train. In such cases, we need different platforms.

Example 1

Input:

N=6

arr[] = {0900, 0940, 0950, 1100, 1500, 1800}, dep[] = {0910, 1200, 1120, 1130, 1900, 2000}

Output: 3

Explanation: A minimum of three platforms are required to safely arrive and depart all trains.

Input Format

The first line consists of an integer N, which represents the total number of trains.

The second line consists of N space-separated integers representing the arrival time.

The third line consists of N space-separated integers representing the departure time.

Output Format

The output displays an integer which is the minimum number of platforms required to safely arrive and depart the train.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

0900 0940 0950 1100 1500 1800

0910 1200 1120 1130 1900 2000

Output: 3

Answer

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```



```
int n = sc.nextInt();
```

```
int[] arr = new int[n];
```

```
int[] dep = new int[n];
```

```
for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
```

```
for (int i = 0; i < n; i++) dep[i] = sc.nextInt();
```

```
Arrays.sort(arr);
```

```
Arrays.sort(dep);
```

```
int i = 0, j = 0;
```

```
int platforms = 0, maxPlatforms = 0;
```

```
while (i < n && j < n) {
```

```
    if (arr[i] <= dep[j]) {
```

```
        platforms++;
```

```
        i++;
```

```
        maxPlatforms = Math.max(maxPlatforms, platforms);
```

```
    } else {
```

```
        platforms--;
```

```
        j++;
```

```
    }
```

```
}
```

```
System.out.println(maxPlatforms);
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 11_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 21.5

Section 1 : Coding

1. Problem Statement

Given a set of coin denominations and a target value V, determine the minimum number of coins required to make the value V.

If it's not possible to make the value with the given coins, return -1.

Input Format

The first line contains an integer n, representing the number of coin denominations.

The second line contains n space-separated integers, representing the coin denominations.

The third line contains an integer amount, representing the target value V.

Output Format

The output prints the minimum number of coins required to make the target value.

Print -1 if the value cannot be formed using the given denominations.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 5

11

Output: 3

Answer

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();  
        int[] coins = new int[n];
```

```
        for (int i = 0; i < n; i++) {  
            coins[i] = sc.nextInt();  
        }
```

```
        int amount = sc.nextInt();
```

```
        int INF = 1000000000;  
        int[] dp = new int[amount + 1];
```

```
        Arrays.fill(dp, INF);  
        dp[0] = 0;
```

```
        for (int coin : coins) {  
            for (int i = coin; i <= amount; i++) {  
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);  
            }  
        }
```

```

    }
    if (dp[amount] == INF) {
        System.out.print("-1");
    } else {
        System.out.print(dp[amount]);
    }
}
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement:

Rajesh runs a delivery company and needs to deliver n packages to different cities. Each package i has:

A weight $wt[i]$ A profit $val[i]$ if delivered on time A delivery deadline $deadline[i]$ (in hours from now) Rajesh has a truck with maximum weight capacity W and can only deliver one package at a time. He can choose the order of deliveries, but if a package is delivered after its deadline, he earns zero profit for it.

Task: Determine the maximum total profit Rajesh can earn by selecting which packages to deliver and in what order, without exceeding the truck's capacity at any point.

Input Format

First line: integer n — number of packages

Second line: n integers — values of each package (profit)

Third line: n integers — weights of each package

Fourth line: n integers — deadlines of each package

Fifth line: integer W — truck capacity

Output Format

The output prints the single integer maximum total profit.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
100 200 150 120
10 20 15 15
5 3 4 2
30

Output: 470

Answer

```
import java.util.*;

public class Main {

    static class Item {
        int val, wt, dl;
        Item(int v, int w, int d) {
            val = v;
            wt = w;
            dl = d;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] val = new int[n];
        int[] wt = new int[n];
        int[] dl = new int[n];

        for (int i = 0; i < n; i++) val[i] = sc.nextInt();
        for (int i = 0; i < n; i++) wt[i] = sc.nextInt();
        for (int i = 0; i < n; i++) dl[i] = sc.nextInt();

        int W = sc.nextInt();
```

```

Item[] arr = new Item[n];
int maxDeadline = 0;

for (int i = 0; i < n; i++) {
    arr[i] = new Item(val[i], wt[i], dl[i]);
    maxDeadline = Math.max(maxDeadline, dl[i]);
}

// Sort by profit descending
Arrays.sort(arr, (a, b) -> b.val - a.val);

boolean[] slot = new boolean[maxDeadline + 1];

int totalWeight = 0;
int profit = 0;

for (int i = 0; i < n; i++) {

    // try to find slot before deadline
    for (int t = Math.min(maxDeadline, arr[i].dl); t > 0; t--) {

        if (!slot[t] && totalWeight + arr[i].wt <= W) {
            slot[t] = true;
            totalWeight += arr[i].wt;
            profit += arr[i].val;
            break;
        }
    }
}

System.out.print(profit);
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

You are given n items, each with a certain weight and value. Your task is to select items to put in a knapsack of capacity W so that the total value is

maximized. You cannot break an item; you must either take the whole item or leave it.

Formally, given two integer arrays, $val[0..n-1]$ and $wt[0..n-1]$, representing the values and weights of n items respectively, and an integer W representing the knapsack capacity, determine the maximum value subset of items such that the total weight does not exceed W .

Solve this problem using dynamic programming.

Input Format

The first line contains an integer n , the number of items.

The second line contains n integers, the values of each item separated by spaces.

The third line contains n integers, the weights of each item separated by spaces.

The fourth line contains an integer W , the maximum capacity of the knapsack.

Output Format

Print a single integer representing the maximum value that can be carried in the knapsack.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

50 60

5 10

10

Output: 60

Answer

```
import java.util.*;

public class Main {
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int[] val = new int[n];
    int[] wt = new int[n];

    for (int i = 0; i < n; i++) {
        val[i] = sc.nextInt();
    }

    for (int i = 0; i < n; i++) {
        wt[i] = sc.nextInt();
    }

    int W = sc.nextInt();

    int[][] dp = new int[n + 1][W + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {

            // not take item
            dp[i][w] = dp[i - 1][w];

            // take item if possible
            if (wt[i - 1] <= w) {
                dp[i][w] = Math.max(dp[i][w],
                    val[i - 1] + dp[i - 1][w - wt[i - 1]]);
            }
        }
    }

    System.out.print(dp[n][W]);
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you are a tourist in Moscow and have booked a bus tour to see some amazing attractions just outside of town. The bus first drives around town for a while (a long while, since Moscow is a big city) picking up people at their respective hotels. It then proceeds to the amazing attraction, and after a few hours go back into the city, again driving to each hotel, this time to drop people off.

For some reason, whenever you do this, your hotel is always the first to be visited for pickup and the last to be visited for drop-off, meaning that you have to suffer through two not-so-amazing sightseeing tours of all the local hotels.

This is clearly not what you want to do (unless for some reason you are really into hotels), so let's fix it. We will develop some software to enable the sightseeing company to route its bus tours more fairly—though it may sometimes mean a longer total distance for everyone, fair is fair, right?

For this problem, there is a starting location (the sightseeing company headquarters), h hotels that need to be visited for pickups and dropoffs, and a destination location (the amazing attraction). We need to find a route that goes from the headquarters, through all the hotels, to the attraction, then back through all the hotels again (possibly in a different order), and finally back to the headquarters.

To guarantee that none of the tourists (and, in particular, you) is forced to suffer through two full tours of the hotels, we require that every hotel that is visited among the first $h/2$ hotels on the way to the attraction is also visited among the first $h/2$ hotels on the way back.

Subject to these restrictions, we would like to make the complete bus tour as short as possible. Note that these restrictions may force the bus to drive past a hotel without stopping there (this is not considered visiting) and then visit it later, as illustrated in the sample input.

Input Format

The first line of each test case consists of two integers n and m , where n is the number of locations (hotels, headquarters, attraction) and m is the number of pairs of locations between which the bus can travel.

The n different locations are numbered from 0 to $n-1$, where 0 is the headquarters, 1 through $n-2$ are the hotels, and $n-1$ is the attraction. Assume that there is at most one direct connection between any pair of locations and it is possible to travel from any location to any other location (but not necessarily directly).

Following the first line are m lines, each containing three integers u , v , and t , indicating that the bus can go directly between locations u and v in t seconds (in either direction).

Output Format

For each test case, print the case number and the minimum total travel time of the complete tour.

The output should follow this format: "Case X: Y"

where:

X is the test case number starting from 1.

Y is the shortest possible total time (in seconds) for the bus tour.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 4

0 1 10

1 2 20

2 3 30

3 4 40

4 6

0 1 1

0 2 1

0 3 1

1 2 1

1 3 1

2 3 1

Output: Case 1: 300

Case 2: 6

Answer

```
import java.util.*;

public class Main {

    static final int INF = 1000000000;

    static int n, m;
    static int[][] dist;
    static int[] hotel;
    static int h;

    static int permCost(int[] order, int start, int end) {
        int cost = 0;

        cost += dist[start][order[0]];
        for (int i = 0; i < order.length - 1; i++) {
            cost += dist[order[i]][order[i + 1]];
        }
        cost += dist[order[order.length - 1]][end];

        return cost;
    }

    static int permCostBack(int[] order, int start) {
        int cost = 0;

        cost += dist[start][order[0]];
        for (int i = 0; i < order.length - 1; i++) {
            cost += dist[order[i]][order[i + 1]];
        }
        cost += dist[order[order.length - 1]][0];

        return cost;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int tc = 1;

        while (sc.hasNextInt()) {
```

```
n = sc.nextInt();  
m = sc.nextInt();
```

```
dist = new int[n][n];
```

```
for (int i = 0; i < n; i++) {  
    Arrays.fill(dist[i], INF);  
    dist[i][i] = 0;  
}
```

```
for (int i = 0; i < m; i++) {  
    int u = sc.nextInt();  
    int v = sc.nextInt();  
    int t = sc.nextInt();  
  
    dist[u][v] = Math.min(dist[u][v], t);  
    dist[v][u] = Math.min(dist[v][u], t);  
}
```

```
// Floyd Warshall  
for (int k = 0; k < n; k++)  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n; j++)  
            dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
```

```
h = n - 2;  
hotel = new int[h];
```

```
for (int i = 0; i < h; i++) hotel[i] = i + 1;
```

```
int ans = INF;
```

```
int[] perm = new int[h];
```

```
do {  
    System.arraycopy(hotel, 0, perm, 0, h);  
  
    int forward = permCost(perm, 0, n - 1);  
    int backward = permCostBack(perm, n - 1);  
  
    ans = Math.min(ans, forward + backward);  
}
```

```

    } while (nextPermutation(perm));

    System.out.println("Case " + tc + ": " + ans);
    tc++;
}
}

```

```

static boolean nextPermutation(int[] a) {
    int i = a.length - 2;

    while (i >= 0 && a[i] >= a[i + 1]) i--;
    if (i < 0) return false;

    int j = a.length - 1;
    while (a[j] <= a[i]) j--;

    int temp = a[i];
    a[i] = a[j];
    a[j] = temp;

    reverse(a, i + 1, a.length - 1);
    return true;
}

```

```

static void reverse(int[] a, int i, int j) {
    while (i < j) {
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
        i++;
        j--;
    }
}
}

```

Status : Partially correct

Marks : 1.5/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 2_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Karthik is organizing marbles of three different colors: red, white, and blue. Each color is represented by an integer—0 for red, 1 for white, and 2 for blue. Karthik wants to sort the marbles so that all marbles of the same color are adjacent, and they are arranged in the order red, white, and blue. Help Karthik solve this problem efficiently using the two-pointer technique.

Input Format

The first line of input consists of an integer n , representing the number of marbles in the array `nums[]`.

The second line contains n integers, representing the marbles' colors, where each integer is either 0, 1, or 2.

Output Format

The output prints the sorted array, where all marbles are grouped by color in the order red (0), white (1), and blue (2).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

2 0 2 1 1 0

Output: 0 0 1 1 2 2

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int nums[n];
```

```
    for(int i = 0; i < n; i++) {
```

```
        scanf("%d", &nums[i]);
```

```
    }
```

```
    int low = 0, mid = 0, high = n - 1;
```

```
    while(mid <= high) {
```

```
        if(nums[mid] == 0) {
```

```
            int temp = nums[low];
```

```
            nums[low] = nums[mid];
```

```
            nums[mid] = temp;
```

```
            low++;
```

```
            mid++;
```

```
        }
```

```
        else if(nums[mid] == 1) {
```

```
            mid++;
```

```
        }
```

```
        else { // nums[mid] == 2
```

```
            int temp = nums[mid];
```

```
            nums[mid] = nums[high];
```

```
            nums[high] = temp;
```

```
        high--;  
    }  
}  
  
    for(int i = 0; i < n; i++) {  
        printf("%d ", nums[i]);  
    }  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Kavya is an avid reader who loves finding hidden patterns in texts. One day, she stumbled upon a string of characters and wondered about the longest palindromic substring it contains. Help Kavya identify this substring using an efficient algorithm based on the two-pointer technique.

Input Format

The input consists of a single string *s*.

Output Format

The output displays "Longest Palindromic Substring: " followed by the longest palindrome found.

Refer the sample output for format specifications.

Sample Test Case

Input: babad

Output: Longest Palindromic Substring: bab

Answer

```
#include <stdio.h>  
#include <string.h>
```



```

void expandCenter(char s[], int left, int right, int *start, int *maxLen) {
    int n = strlen(s);

    while(left >= 0 && right < n && s[left] == s[right]) {
        if(right - left + 1 > *maxLen) {
            *start = left;
            *maxLen = right - left + 1;
        }
        left--;
        right++;
    }
}

```

```

int main() {
    char s[25];
    scanf("%s", s);

    int start = 0, maxLen = 0;
    int n = strlen(s);

    for(int i = 0; i < n; i++) {
        expandCenter(s, i, i, &start, &maxLen); // odd length
        expandCenter(s, i, i + 1, &start, &maxLen); // even length
    }

    printf("Longest Palindromic Substring: ");
    for(int i = start; i < start + maxLen; i++) {
        printf("%c", s[i]);
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rohit is analyzing an array of integers to find special combinations of numbers. Using the Two-Pointer technique, he wants to identify all unique triplets in the array whose sum is equal to 0. Given an integer array, Rohit must find and print all such triplets, ensuring that no duplicate triplets are

included in the result.

Write a program to ensure that all valid triplets with sum zero are printed correctly.

Example 1

Input:

6

-1 0 1 2 -1 -4

Output:

-1 -1 2

-1 0 1

Explanation:

$\text{nums}[0] + \text{nums}[1] + \text{nums}[2] = (-1) + 0 + 1 = 0.$

$\text{nums}[1] + \text{nums}[2] + \text{nums}[4] = 0 + 1 + (-1) = 0.$

$\text{nums}[0] + \text{nums}[3] + \text{nums}[4] = (-1) + 2 + (-1) = 0.$

The distinct triplets are $[-1, 0, 1]$ and $[-1, -1, 2]$.

Notice that the order of the output and the order of the triplets does not matter.

Example 2

Input:

3

0 1 1

Output:

No triplets found with sum 0.

Explanation:

The only possible triplet does not sum up to 0.

Example 3

Input:

3

0 0 0

Output:

0 0 0

Explanation:

The only possible triplet sums up to 0.

Input Format

The first line contains an integer n , representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints each unique triplet whose sum is 0 on a separate line.

Each triplet must be printed in ascending order (as they appear after sorting).

If no such triplets exist, print "No triplets found with sum 0."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6

-1 0 1 2 -1 -4

Output: -1 -1 2

-1 0 1

Answer

```
#include <stdio.h>
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    int nums[n];
```

```
    for(int i = 0; i < n; i++) {  
        scanf("%d", &nums[i]);  
    }
```

```
    // Sorting the array (Bubble Sort since  $n \leq 10$ )
```

```
    for(int i = 0; i < n - 1; i++) {  
        for(int j = 0; j < n - i - 1; j++) {  
            if(nums[j] > nums[j + 1]) {  
                int temp = nums[j];  
                nums[j] = nums[j + 1];  
                nums[j + 1] = temp;  
            }  
        }  
    }
```

```
    int found = 0;
```

```
    for(int i = 0; i < n - 2; i++) {  
        if(i > 0 && nums[i] == nums[i - 1])  
            continue;
```

```
        int left = i + 1;  
        int right = n - 1;
```

```
        while(left < right) {
```

```
            int sum = nums[i] + nums[left] + nums[right];
```

```
            if(sum == 0) {  
                printf("%d %d %d \n", nums[i], nums[left], nums[right]);  
                found = 1;
```

```

while(left < right && nums[left] == nums[left + 1])
    left++;
while(left < right && nums[right] == nums[right - 1])
    right--;

    left++;
    right--;
}
else if(sum < 0)
    left++;
else
    right--;
}
}
if(!found) {
    printf("No triplets found with sum 0.");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Rahul is given an array of integers and a target value T. Using the Two-Pointer technique, he wants to find two distinct elements in the array such that the sum of the pair is closest to the target value. To apply this technique efficiently, the array must first be sorted.

Write a program to ensure that the pair of numbers whose sum is closest to T is identified and printed correctly.

Input Format

The first line contains two space-separated integers n and T, where:

- n is the number of elements in the array
- T is the target sum

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints two space separated integers representing the pair whose sum is closest to the target.

The output pair must be printed in ascending order.

If multiple pairs have the same closest sum, print the first pair found by the two-pointer traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 50
10 22 28 29 30 40

Output: 10 40

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}
```

```
int main() {
    int n, T;
    scanf("%d %d", &n, &T);
```

```
    int arr[n];
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    // Sort the array
    qsort(arr, n, sizeof(int), compare);
```

```
int left = 0, right = n - 1;
int minDiff = 1000000000;
int a = arr[0], b = arr[1];

while(left < right) {
    int sum = arr[left] + arr[right];
    int diff = abs(sum - T);

    if(diff < minDiff) {
        minDiff = diff;
        a = arr[left];
        b = arr[right];
    }

    if(sum > T)
        right--;
    else
        left++;
}

printf("%d %d", a, b);

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 7_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 13

Section 1 : MCQ

1. Which of the following statements is true about deleting a node with one child from a binary search tree?

Answer

The child node replaces the deleted node

Status : Correct

Marks : 1/1

2. Consider the given binary search tree, if the root node is deleted, then the new root can be _____.

Answer

30 or 63

Status : Wrong

Marks : 0/1

3. In a binary search tree (BST), if the value to be searched is smaller than the value of the root, where should the search proceed?

Answer

Left subtree

Status : Correct

Marks : 1/1

4. Find the pre-order traversal of the given binary search tree.

Answer

13, 2, 1, 4, 14, 18

Status : Correct

Marks : 1/1

5. Which of the following nodes would replace node 40 if it were to be deleted?

Answer

90

Status : Wrong

Marks : 0/1

6. The preorder traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 42. Which one of the following is the post-order traversal sequence of the same tree?

Answer

15, 10, 23, 25, 20, 35, 42, 39, 30

Status : Correct

Marks : 1/1

7. In a binary search tree with nodes 18, 28, 12, 11, 16, 14, 17, what is the value of the left child of the node 16?

Answer

14

Status : Correct

Marks : 1/1

8. Find the inorder traversal of the given binary search tree.

Answer

1, 2, 4, 6, 7, 9, 10, 14

Status : Correct

Marks : 1/1

9. What will be the in-order traversal sequence of the following binary search tree, after deleting node 40?

Answer

8 12 20 22 30

Status : Correct

Marks : 1/1

10. In a binary search tree, which of the following traversals is used for getting ascending order values?

Answer

In-order

Status : Correct

Marks : 1/1

11. In a binary search tree with nodes 18, 28, 12, 11, 16, 14, 17, what is the value of the right child of the node 12?

Answer

16

Status : Correct

Marks : 1/1

12. When comparing a value to be searched with the root node in a BST?

Answer

If it's larger, go to the right subtree

Status : Correct

Marks : 1/1

13. Find the postorder traversal of the given binary search tree.

Answer

1, 4, 2, 18, 14, 13

Status : Correct

Marks : 1/1

14. Which of the following operations can be used to traverse a Binary Search Tree (BST) in ascending order?

Answer

Inorder traversal

Status : Correct

Marks : 1/1

15. After deleting a node in a binary search tree, how is the replacement node determined if the node to be deleted has only one child?

Answer

Replacement node is the child node itself

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 6_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Yuvana, an avid programmer, is keen on mastering various sorting algorithms and has recently delved into Counting Sort. As a practice exercise, she is given an array of N integers where each element ranges from 1 to $N-1$. Some numbers may appear multiple times in the array. Her task is to identify and print all numbers that appear more than once. If no such duplicates are found, she should print -1.

Help Yuvana by implementing a function that efficiently finds and prints duplicate numbers using the Counting Sort technique.

Input Format

The first line of input consists of an integer N , representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints all the duplicate numbers that appear more than once in the array, each separated by a space.

If no duplicates are found, print -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

1 3 4 2 2

Output: 2

Answer

```
#include <stdio.h>
```

```
int main() {  
    int N;  
    scanf("%d", &N);  
  
    int arr[N];  
    int count[N]; // Since elements are in 1..N-1  
    for (int i = 0; i < N; i++) count[i] = 0;
```

```
    // Read array and populate count  
    for (int i = 0; i < N; i++) {  
        scanf("%d", &arr[i]);  
        count[arr[i]]++;  
    }
```

```
    int found = 0; // Flag to check if any duplicates exist  
    for (int i = 1; i <= N - 1; i++) {  
        if (count[i] > 1) {  
            printf("%d ", i);  
            found = 1;
```

```
}  
}  
  
if (!found)  
    printf("-1");  
  
printf("\n");  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Anu is tasked with organizing a list of fruit names stored as strings. To ensure that the list is easy to navigate, she decides to sort the strings in ascending alphabetical order.

Being a fan of efficient sorting algorithms, Anu opts to use radix sort for this task. Help Anu with a program that can take a list of fruit names as input and output the sorted list.

Input Format

The first line of input consists of an integer n , representing the number of fruit names in the list.

The next n lines each consist of a string, representing the names of the fruits.

Output Format

The output prints n lines containing the sorted list of fruit names in ascending alphabetical order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
apple

banana
orange
grape
Output: apple
banana
grape
orange

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX_N 20
#define MAX_LEN 100

// Counting sort for a specific character position
void countSort(char arr[][MAX_LEN], int n, int pos) {
    char output[n][MAX_LEN];
    int count[256] = {0}; // ASCII character range

    // Count occurrences of each character at position pos
    for (int i = 0; i < n; i++) {
        int ch = pos < strlen(arr[i]) ? (unsigned char)arr[i][pos] : 0;
        count[ch]++;
    }

    // Accumulate counts
    for (int i = 1; i < 256; i++)
        count[i] += count[i - 1];

    // Build output array (stable sort)
    for (int i = n - 1; i >= 0; i--) {
        int ch = pos < strlen(arr[i]) ? (unsigned char)arr[i][pos] : 0;
        strcpy(output[count[ch] - 1], arr[i]);
        count[ch]--;
    }

    // Copy sorted strings back to original array
    for (int i = 0; i < n; i++)
        strcpy(arr[i], output[i]);
}
```

```

// Radix sort for strings
void radixSort(char arr[][MAX_LEN], int n) {
    int maxLen = 0;
    for (int i = 0; i < n; i++)
        if (strlen(arr[i]) > maxLen)
            maxLen = strlen(arr[i]);

    // Perform counting sort for each character position from right to left
    for (int pos = maxLen - 1; pos >= 0; pos--)
        countSort(arr, n, pos);
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[MAX_N][MAX_LEN];
    for (int i = 0; i < n; i++)
        scanf("%s", arr[i]);

    radixSort(arr, n);

    // Print sorted strings
    for (int i = 0; i < n; i++)
        printf("%s\n", arr[i]);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

30000 25000 40000 35000 20000 28000 29500

5

Output: 5th largest Salary: 28000

Answer

```
#include <stdio.h>
```

```
// Swap two integers
```

```
void swap(int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// Heapify a subtree rooted at index i in a max-heap of size n
```

```
void heapify(int arr[], int n, int i) {
```

```
    int largest = i;    // Initialize largest as root
```

```
    int left = 2 * i + 1; // Left child
```

```
    int right = 2 * i + 2; // Right child
```

```
    if (left < n && arr[left] > arr[largest])
```

```

        largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            swap(&arr[i], &arr[largest]);
            heapify(arr, n, largest);
        }
    }

    // Build a max-heap from array
    void buildMaxHeap(int arr[], int n) {
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(arr, n, i);
    }

    int main() {
        int N, K;
        scanf("%d", &N);

        int arr[N];
        for (int i = 0; i < N; i++)
            scanf("%d", &arr[i]);

        scanf("%d", &K);

        buildMaxHeap(arr, N);

        int heapSize = N;
        int kthLargest;

        // Extract max K times
        for (int i = 0; i < K; i++) {
            kthLargest = arr[0]; // Root is current largest
            arr[0] = arr[heapSize - 1]; // Move last element to root
            heapSize--;
            heapify(arr, heapSize, 0); // Heapify root
        }

        printf("%dth largest Salary: %d\n", K, kthLargest);
        return 0;
    }

```

}

Status : Correct

Marks : 10/10

4. Problem Statement

Manage a warehouse inventory with unique barcode numbers utilizing a specialized sorting algorithm designed for barcodes with exactly 16 digits. Input the number of barcode numbers (N) and their corresponding values.

Employ the Bucket Sort algorithm to distribute and sort the numbers based on their digits. Display the sorted barcode numbers, ensuring an efficient and organized warehouse inventory management system tailored for barcode systems with 16 digits.

Input Format

The first line contains an integer N, representing the number of barcode numbers to be sorted.

The following N lines contain exactly 16-digit long integers, each representing a unique barcode number.

Output Format

The output displays a single line containing "Sorted barcode numbers:" followed by the sorted barcode numbers in ascending order. Each barcode number is displayed on a separate line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

2345678901234567
1234567890123456
3456789012345678
5678901234567890
4567890123456789

Output: Sorted barcode numbers:

```
1234567890123456
2345678901234567
3456789012345678
4567890123456789
5678901234567890
```

Answer

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
```

```
int main() {
    int N;
    cin >> N;
```

```
    vector<long long> barcodes(N);
```

```
    for (int i = 0; i < N; i++) {
        cin >> barcodes[i];
    }
```

```
    // Radix Sort using Bucket Sort (10 buckets for digits 0-9)
    long long divisor = 1;
```

```
    for (int digit = 0; digit < 16; digit++) {
        queue<long long> buckets[10];
```

```
        // Place elements into buckets
        for (int i = 0; i < N; i++) {
            int digitValue = (barcodes[i] / divisor) % 10;
            buckets[digitValue].push(barcodes[i]);
        }
```

```
        // Collect elements back into array
        int index = 0;
        for (int i = 0; i < 10; i++) {
            while (!buckets[i].empty()) {
                barcodes[index++] = buckets[i].front();
                buckets[i].pop();
            }
```

```
}  
    divisor *= 10;  
}  
  
// Output result  
cout << "Sorted barcode numbers:" << endl;  
for (int i = 0; i < N; i++) {  
    cout << barcodes[i] << endl;  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 3_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : MCQ

1. The main idea behind the Knuth-Morris-Pratt algorithm is to:

Answer

Avoid redundant checking of characters that have already been matched

Status : Correct

Marks : 1/1

2. The Boyer-Moore algorithm uses which two rules to determine the shift amount?

Answer

Good suffix rule and bad character rule

Status : Correct

Marks : 1/1

3. The Boyer-Moore algorithm is particularly efficient when

Answer

The pattern contains repetitive elements

Status : Correct

Marks : 1/1

4. In the KMP algorithm, if the prefix table at position i has a value of k , what does it imply?

Answer

The length of the longest proper prefix which is also a suffix up to position i is k .

Status : Correct

Marks : 1/1

5. The concept of prefix and suffix is used in which of the following algorithms?

Answer

Knuth-Morris-Pratt (KMP) algorithm

Status : Correct

Marks : 1/1

6. What is the main purpose of Manacher's Algorithm?

Answer

To find the longest palindromic substring in linear time

Status : Correct

Marks : 1/1

7. In the Z-algorithm, the Z-array is used to store:

Answer

The length of the longest substring starting at each position that matches the prefix

Status : Correct

Marks : 1/1

8. Which application is most efficiently solved using Z-algorithm?

Answer

Finding all occurrences of a pattern in a text

Status : Correct

Marks : 1/1

9. In Z-algorithm, when $Z[k]$ is greater than 0, what does it imply?

Answer

Substring starting at k matches the prefix for length $Z[k]$

Status : Correct

Marks : 1/1

10. Which of the following best explains why Manacher's algorithm is preferable for palindrome problems over expand-around-center approaches?

Answer

Avoids recomputation of overlapping palindromes by mirroring results around the center

Status : Correct

Marks : 1/1

11. In bioinformatics applications, KMP is often used instead of Boyer-Moore. Why?

Answer

DNA has only 4 characters, reducing Boyer-Moore's skipping advantage

Status : Correct

Marks : 1/1

12. In KMP, the LPS array for pattern "ABABAC" is:

Answer

[0,0,1,2,3,0]

Status : Correct

Marks : 1/1

13. Which is true about Manacher's algorithm with respect to handling even-length palindromes?

Answer

Uses a modified string with separators between characters

Status : Correct

Marks : 1/1

14. How is Z-algorithm used for pattern search in text efficiently?

Answer

Concatenate $P + \$ + T$ and compute Z-array

Status : Correct

Marks : 1/1

15. Which of the following strings has the largest number of palindromic substrings using Manacher's algorithm?

Answer

"aaaaa"

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 10_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Aarav, a computer science student, is working on a heap-based data processing system. He needs to find the second-largest element from a collection of integers represented as a max-heap. Aarav first constructs the max-heap, then removes the largest element to extract the next largest value.

Write a program to help Aarav determine and print the second largest element, or display a message if it does not exist.

Input Format

The first line contains an integer n, representing the number of elements in the heap.

The second line contains n space-separated integers representing the elements of the heap.

Output Format

The output prints the second-largest element in the heap.

If there is no second-largest element, print the message:

"There is no second largest element in the heap."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
100 70 10 30

Output: 70

Answer

```
import java.util.*;

public class Main {

    static void swap(int[] heap, int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    // Max heapify
    static void heapify(int[] heap, int n, int i) {
        int largest = i;
        int l = 2 * i + 1;
        int r = 2 * i + 2;

        if (l < n && heap[l] > heap[largest])
            largest = l;

        if (r < n && heap[r] > heap[largest])
            largest = r;
```

```

        if (largest != i) {
            swap(heap, i, largest);
            heapify(heap, n, largest);
        }
    }

    static void buildHeap(int[] heap, int n) {
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(heap, n, i);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] heap = new int[n];

        for (int i = 0; i < n; i++) {
            heap[i] = sc.nextInt();
        }

        if (n < 2) {
            System.out.print("There is no second largest element in the heap.");
            return;
        }

        buildHeap(heap, n);

        // remove largest (root)
        heap[0] = heap[n - 1];
        n--;

        heapify(heap, n, 0);

        System.out.print(heap[0]);
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Elijah wants to verify if an array represents a valid min-heap. Implement a program to check whether a given array satisfies the min-heap property.

In a min-heap, for every node i , the value at that node must be less than or equal to the values of its child nodes (if they exist). Write a program to read an array and determine whether it maintains the min-heap structure.

Input Format

The first line contains an integer n , representing the number of elements.

The next line contains n space-separated integers representing the elements to be inserted into the min heap.

Output Format

The first line prints "The array represents a valid min-heap" if the array satisfies the min-heap property. Otherwise, print "The array does not represent a valid min-heap".

The second line prints the space-separated elements of the array as given in the input.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

Output: The array represents a valid min-heap

1 2 3

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

int n = sc.nextInt();
int[] arr = new int[n];

for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}

boolean isHeap = true;

for (int i = 0; i <= (n / 2) - 1; i++) {
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[i] > arr[left]) {
        isHeap = false;
        break;
    }

    if (right < n && arr[i] > arr[right]) {
        isHeap = false;
        break;
    }
}

if (isHeap) {
    System.out.println("The array represents a valid min-heap");
} else {
    System.out.println("The array does not represent a valid min-heap");
}

for (int x : arr) {
    System.out.print(x + " ");
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Given a list of daily transaction amounts over N days. Each transaction can be positive (deposit) or negative (withdrawal).

You must efficiently:

Find the sum of transactions between two given days. Update the transaction amount for any specific day.

To perform these operations efficiently, implement a Segment Tree.

Input Format

The first line contains two integers, N and Q, representing the number of days and number of operations.

The second line contains N space-separated integers representing the transaction amount for each day.

The next Q lines contain operations in one of the following formats:

- S L R represents Query sum from index L to R
- U i val Updates index i with new value val

All indices are 0-based.

Output Format

For each query S L R, print the sum of transactions in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
10 -5 20 15 -10 5
S 0 2
U 1 7
S 0 2
S 3 5
S 0 5

Output: 25

37
10
47

Answer

```
import java.util.*;

public class Main {

    static int[] seg;
    static int n;

    static void build(int[] arr, int idx, int l, int r) {
        if (l == r) {
            seg[idx] = arr[l];
            return;
        }

        int mid = (l + r) / 2;

        build(arr, 2 * idx + 1, l, mid);
        build(arr, 2 * idx + 2, mid + 1, r);

        seg[idx] = seg[2 * idx + 1] + seg[2 * idx + 2];
    }

    static void update(int idx, int l, int r, int pos, int val) {
        if (l == r) {
            seg[idx] = val;
            return;
        }

        int mid = (l + r) / 2;

        if (pos <= mid) {
            update(2 * idx + 1, l, mid, pos, val);
        } else {
            update(2 * idx + 2, mid + 1, r, pos, val);
        }

        seg[idx] = seg[2 * idx + 1] + seg[2 * idx + 2];
    }
}
```



```

    }

    static int query(int idx, int l, int r, int ql, int qr) {

        if (qr < l || r < ql) return 0;

        if (ql <= l && r <= qr) return seg[idx];

        int mid = (l + r) / 2;

        return query(2 * idx + 1, l, mid, ql, qr)
            + query(2 * idx + 2, mid + 1, r, ql, qr);
    }

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

```

```

    n = sc.nextInt();
    int q = sc.nextInt();

```

```

    int[] arr = new int[n];

```

```

    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }

```

```

    seg = new int[4 * n];

```

```

    build(arr, 0, 0, n - 1);

```

```

    while (q-- > 0) {
        char type = sc.next().charAt(0);

```

```

        if (type == 'S') {
            int l = sc.nextInt();
            int r = sc.nextInt();
            System.out.println(query(0, 0, n - 1, l, r));
        } else {
            int i = sc.nextInt();
            int val = sc.nextInt();
            update(0, 0, n - 1, i, val);
        }
    }

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given a string `s` and a string array dictionary, return the longest string in the dictionary that can be formed by deleting some of the given string characters.

If there is more than one possible result, return the longest word with the smallest lexicographical order.

Example

Input:

`s = "abpcplea"`

`dictionary = ["ale", "apple", "monkey", "plea"]`

Output:

`"apple"`

Input Format

The first line of input consists of the string.

The second line of input consists of the array size.

The following lines of input consist of the array elements, each in a separate line.

Output Format

The output prints the longest string in the dictionary that can be formed by deleting some of the given string characters.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: abpcplea

4

ale

apple

monkey

plea

Output: apple

Answer

```
import java.util.*;

public class Main {

    static boolean isSubsequence(String s, String word) {
        int i = 0, j = 0;

        while (i < s.length() && j < word.length()) {
            if (s.charAt(i) == word.charAt(j)) {
                j++;
            }
            i++;
        }
        return j == word.length();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String s = sc.nextLine().trim();
        int n = Integer.parseInt(sc.nextLine().trim());

        String best = "";

        for (int i = 0; i < n; i++) {
            String word = sc.nextLine().trim();

            if (isSubsequence(s, word)) {
```

```
        if (word.length() > best.length() ||  
            (word.length() == best.length() && word.compareTo(best) < 0)) {  
            best = word;  
        }  
    }  
}  
  
System.out.print(best);  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 5_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Akash wants to write a program that populates values in a singly linked list by inserting each element into the front of the list. After insertion, the program should delete a node immediately after a node with a specific value X.

Assist him with a suitable program.

Input Format

The first line of input consists of an integer N, representing the number of elements.

The second line consists of N space-separated integers, representing the elements of the linked list.

The third line consists of an integer X, representing the value after which the node has to be deleted.

Output Format

The output consists of two lines:

The first line shows the original linked list in the format: Original Linked List: <elements> -> NULL.

The second line shows the modified linked list after deleting the node following the node with value X: After deleting node after <X>: <modified_elements> -> NULL.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
15 28 36 43
36

Output: Original Linked List: 43 -> 36 -> 28 -> 15 -> NULL
After deleting node after 36: 43 -> 36 -> 15 -> NULL

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Insert at front
struct Node* insertAtFront(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head;
    return newNode;
}
```

```
// Print list in required format
void printList(struct Node* head) {
    while (head != NULL) {
        printf("%d", head->data);
        if (head->next != NULL) printf(" -> ");
        head = head->next;
    }
    printf(" -> NULL\n");
}
```

```
// Delete node after value X
struct Node* deleteAfterValue(struct Node* head, int X) {
    struct Node* temp = head;

    while (temp != NULL && temp->next != NULL) {
        if (temp->data == X) {
            struct Node* nodeToDelete = temp->next;
            temp->next = nodeToDelete->next;
            free(nodeToDelete);
            break; // Only delete the first occurrence after X
        }
        temp = temp->next;
    }
}
```

```
    return head;
}
```

```
// Free entire list
void freeList(struct Node* head) {
    while (head != NULL) {
        struct Node* temp = head;
        head = head->next;
        free(temp);
    }
}
```

```
int main() {
    int N;
    scanf("%d", &N);

    struct Node* head = NULL;
```

```

// Insert each element at the front
for (int i = 0; i < N; i++) {
    int value;
    scanf("%d", &value);
    head = insertAtFront(head, value);
}

int X;
scanf("%d", &X);

// Print original list
printf("Original Linked List: ");
printList(head);

// Delete node after X
head = deleteAfterValue(head, X);

printf("After deleting node after %d: ", X);
printList(head);

freeList(head);
return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Emma is managing her daily to-do list. Each task is represented by an integer ID. Due to a change in her schedule, Emma needs to update her to-do list by removing the first task, another task from the end of the list, and a third task from a specific position within the list.

Your task is to help Emma implement a function to perform these deletions and manage her to-do list efficiently.

Input Format

The first line of input contains an integer T, representing the number of tasks currently in the list.

The second line contains T integers, each representing the ID of a task.

The third line contains an integer p, representing the position (0-based index) of the task to be removed.

Output Format

The first line of output prints the linked list after deleting the first node, with each element separated by a space.

The second line of output prints the linked list after deleting the last node, with each element separated by a space.

The third line of output prints the linked list after deleting the node at the specified position, with each element separated by a space

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

11 22 33 44 55

2

Output: 22 33 44 55

22 33 44

22 33

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
// Append node at end
```

```
struct Node* append(struct Node* head, int value) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
newNode->data = value;
newNode->next = NULL;
```

```
if (head == NULL) return newNode;
```

```
struct Node* temp = head;
while (temp->next != NULL) temp = temp->next;
temp->next = newNode;
return head;
}
```

```
// Delete first node
```

```
struct Node* deleteFirst(struct Node* head) {
    if (head == NULL) return NULL;
    struct Node* temp = head;
    head = head->next;
    free(temp);
    return head;
}
```

```
// Delete last node
```

```
struct Node* deleteLast(struct Node* head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) {
        free(head);
        return NULL;
    }
}
```

```
struct Node* temp = head;
while (temp->next->next != NULL) temp = temp->next;
```

```
free(temp->next);
temp->next = NULL;
return head;
}
```

```
// Delete node at position p (0-based)
```

```
struct Node* deleteAtPosition(struct Node* head, int p) {
    if (head == NULL) return NULL;
    if (p == 0) return deleteFirst(head);
    struct Node* temp = head;
```

```

    int count = 0;
    while (temp != NULL && count < p - 1) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL || temp->next == NULL) return head; // position out of bounds

    struct Node* nodeToDelete = temp->next;
    temp->next = nodeToDelete->next;
    free(nodeToDelete);
    return head;
}

// Print list space-separated
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

// Free entire list
void freeList(struct Node* head) {
    while (head != NULL) {
        struct Node* temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    int T;
    scanf("%d", &T);

    struct Node* head = NULL;
    for (int i = 0; i < T; i++) {
        int value;
        scanf("%d", &value);
    }
}

```

```

        head = append(head, value);
    }

    int p;
    scanf("%d", &p);

    // Delete first node
    head = deleteFirst(head);
    printList(head);

    // Delete last node
    head = deleteLast(head);
    printList(head);

    // Delete node at position p
    head = deleteAtPosition(head, p);
    printList(head);

    freeList(head);
    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Emma is working with a doubly linked list and needs a program to delete a specific value from the list. Write a program that reads a list of integers, constructs a doubly linked list, and then deletes a given value if it exists. The program should print appropriate messages if the value is found and deleted or if it is not found in the list.

Input Format

The first line contains an integer n, representing the number of elements in the list.

The second line contains n space-separated integers, representing the elements of the doubly linked list.

The third line contains an integer x, representing the value to be deleted from the

list.

Output Format

The first line prints the initial doubly linked list, with elements separated by a space.

The second line prints "Value x deleted from the list." on a new line if x is found and deleted, If x is not found, print "Value x not found in the list." on a new line.

The third line prints the updated doubly linked list on a new line after deletion, with elements separated by a space,

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

10 20 30

20

Output: 10 20 30

Value 20 deleted from the list.

10 30

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Doubly linked list node
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* prev;
```

```
    struct Node* next;
```

```
};
```

```
// Append node at end
```

```
struct Node* append(struct Node* head, int value) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->prev = NULL;
```

```
    newNode->next = NULL;
```

```
if (head == NULL) return newNode;
```

```
    struct Node* temp = head;  
    while (temp->next != NULL) temp = temp->next;
```

```
    temp->next = newNode;  
    newNode->prev = temp;  
    return head;
```

```
}
```

```
// Print list space-separated
```

```
void printList(struct Node* head) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        printf("%d", temp->data);  
        if (temp->next != NULL) printf(" ");  
        temp = temp->next;  
    }  
    printf("\n");  
}
```

```
// Delete node with value x
```

```
struct Node* deleteValue(struct Node* head, int x, int* found) {  
    struct Node* temp = head;
```

```
    while (temp != NULL) {  
        if (temp->data == x) {  
            *found = 1;  
            // Node is head  
            if (temp->prev == NULL) {  
                head = temp->next;  
                if (head != NULL) head->prev = NULL;  
            } else {  
                temp->prev->next = temp->next;  
                if (temp->next != NULL) temp->next->prev = temp->prev;  
            }  
            free(temp);  
            return head;
```

```
        }  
        temp = temp->next;  
    }  
}
```

```
*found = 0;  
return head;  
}
```

```
// Free entire list  
void freeList(struct Node* head) {  
    while (head != NULL) {  
        struct Node* temp = head;  
        head = head->next;  
        free(temp);  
    }  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    struct Node* head = NULL;  
    for (int i = 0; i < n; i++) {  
        int value;  
        scanf("%d", &value);  
        head = append(head, value);  
    }
```

```
    int x;  
    scanf("%d", &x);
```

```
    // Print initial list  
    printList(head);
```

```
    int found;  
    head = deleteValue(head, x, &found);
```

```
    if (found) {  
        printf("Value %d deleted from the list.\n", x);  
    } else {  
        printf("Value %d not found in the list.\n", x);  
    }
```

```
    printList(head);  
    freeList(head);
```

```
} return 0;
```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you're managing a store's inventory list, and some products were accidentally entered multiple times. You need to remove the duplicate products from the list to ensure each product appears only once.

You have an unsorted doubly linked list of product IDs. Some of these product IDs may appear more than once, and your goal is to remove any duplicates.

Note: Insert elements at the front of the doubly linked list.

Input Format

The first line of input consists of an integer n , representing the number of elements in the list.

The second line of input consists of n space-separated integers representing the list elements.

Output Format

The output prints the final list after removing duplicate nodes, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
12 12 10 4 8 4 6 4 4 8
Output: 8 4 6 10 12

Answer


```

#include <stdio.h>
#include <stdlib.h>

// Node structure for doubly linked list
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Insert at front
struct Node* insertAtFront(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;
    if (head != NULL) head->prev = newNode;
    return newNode;
}

// Print list space-separated
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) printf(" ");
        temp = temp->next;
    }
    printf("\n");
}

// Remove duplicates from unsorted doubly linked list
struct Node* removeDuplicates(struct Node* head) {
    struct Node* current = head;
    while (current != NULL) {
        struct Node* runner = current->next;
        while (runner != NULL) {
            if (runner->data == current->data) {
                struct Node* duplicate = runner;
                runner = runner->next;
                // Remove duplicate node
            }
        }
        current = current->next;
    }
}

```

```

        if (duplicate->prev != NULL) duplicate->prev->next = duplicate->next;
        if (duplicate->next != NULL) duplicate->next->prev = duplicate->prev;

        free(duplicate);
    } else {
        runner = runner->next;
    }
}
current = current->next;
}
return head;
}

```

```

// Free entire list
void freeList(struct Node* head) {
    while (head != NULL) {
        struct Node* temp = head;
        head = head->next;
        free(temp);
    }
}

```

```

int main() {
    int n;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        int value;
        scanf("%d", &value);
        head = insertAtFront(head, value);
    }

    head = removeDuplicates(head);

    printList(head);
    freeList(head);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 10_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 32

Section 1 : Coding

1. Problem Statement

Sai has been studying various algorithms related to heaps. Recently, he came across a problem where he needed to filter elements from a given array based on certain criteria related to heap properties.

He wants to build a max heap from a given array of integers, delete elements that fall outside the specified range, and then rearrange the elements in the form of a Max Heap.

Write a program to help Sai in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the elements of the array.

The third line consists of two integers, a and b, separated by a space, representing the range to filter the elements.

Output Format

The output prints the space-separated integers, representing the elements that remain in the max heap after removing elements outside the given range.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 7 8 3 12
1 6

Output: 5 3

Answer

```
import java.util.*;

public class Main {

    static void swap(ArrayList<Integer> heap, int i, int j) {
        int temp = heap.get(i);
        heap.set(i, heap.get(j));
        heap.set(j, temp);
    }

    // Heapify UP (important: insertion method)
    static void insert(ArrayList<Integer> heap, int val) {
        heap.add(val);

        int i = heap.size() - 1;

        while (i > 0) {
            int parent = (i - 1) / 2;
```

```

        if (heap.get(parent) < heap.get(i)) {
            swap(heap, parent, i);
            i = parent;
        } else {
            break;
        }
    }
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    ArrayList<Integer> heap = new ArrayList<>();

    for (int i = 0; i < n; i++) {
        int x = sc.nextInt();
        insert(heap, x); // IMPORTANT FIX
    }
}

```

```

int a = sc.nextInt();
int b = sc.nextInt();

```

```

ArrayList<Integer> filtered = new ArrayList<>();

```

```

for (int x : heap) {
    if (x >= a && x <= b) {
        filtered.add(x);
    }
}

```

```

// rebuild heap using insertion again (IMPORTANT)
ArrayList<Integer> result = new ArrayList<>();
for (int x : filtered) {
    insert(result, x);
}

```

```

for (int x : result) {
    System.out.print(x + " ");
}
}
}

```

Status : Partially correct

Marks : 2/10

2. Problem Statement

Maxwell, an enthusiastic computer science student, is exploring the fascinating world of data structures. His task is to create a program that constructs a min heap from a list of integers and identifies negative numbers within it.

Help Maxwell in completing the program.

Input Format

The first line of input consists of an integer N, representing the number of integers in the array.

The second line consists of N space-separated integers, representing the elements of the list.

Output Format

The first line of output prints the min-heap as a space-separated list of integers.

The second line prints the negative numbers, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

15 -5 20 -19 23 42 29

Output: -19 -5 20 15 23 42 29

-19 -5

Answer

```
import java.util.*;
```

```
public class Main {
```

```
static void swap(ArrayList<Integer> heap, int i, int j) {  
    int temp = heap.get(i);  
    heap.set(i, heap.get(j));  
    heap.set(j, temp);  
}
```

// Min heapify (downward)

```
static void heapify(ArrayList<Integer> heap, int n, int i) {  
    int smallest = i;  
    int l = 2 * i + 1;  
    int r = 2 * i + 2;
```

```
    if (l < n && heap.get(l) < heap.get(smallest))  
        smallest = l;
```

```
    if (r < n && heap.get(r) < heap.get(smallest))  
        smallest = r;
```

```
    if (smallest != i) {  
        swap(heap, i, smallest);  
        heapify(heap, n, smallest);  
    }
```

```
}
```

```
static void buildHeap(ArrayList<Integer> heap) {  
    int n = heap.size();  
    for (int i = n / 2 - 1; i >= 0; i--) {  
        heapify(heap, n, i);  
    }  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);
```

```
    int n = sc.nextInt();  
    ArrayList<Integer> heap = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++) {  
        heap.add(sc.nextInt());  
    }
```

```
    // Build MIN HEAP using heapify (IMPORTANT FIX)
```



```

buildHeap(heap);

// Print heap
for (int x : heap) {
    System.out.print(x + " ");
}
System.out.println();

// Print negatives in heap order
for (int x : heap) {
    if (x < 0) {
        System.out.print(x + " ");
    }
}
}
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are given an array of n integers. You need to perform q queries of two types:

Q l r — Find the minimum value in the subarray from index l to r (inclusive).
 U idx val — Update the element at index idx to val .

Implement an efficient solution using a Segment Tree to handle both query and update operations.

Input Format

The first line contains two space-separated integers n and q , representing the size of the array and the number of queries

The second line contains n space-separated integers representing the elements of the array.

Next q lines: Each line contains a query in one of the following formats:

- Q l r
- U idx val

Output Format

For each query of type Q, print the minimum value in the specified range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3 2

4 2 5

Q 0 2

Q 1 1

Output: 2

2

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    static int[] seg;
```

```
    static int n;
```

```
    static void build(int[] arr, int idx, int l, int r) {
```

```
        if (l == r) {
```

```
            seg[idx] = arr[l];
```

```
            return;
```

```
        }
```

```
        int mid = (l + r) / 2;
```

```
        build(arr, 2 * idx + 1, l, mid);
```

```
        build(arr, 2 * idx + 2, mid + 1, r);
```

```
        seg[idx] = Math.min(seg[2 * idx + 1], seg[2 * idx + 2]);
```

```
    }
```

```
    static void update(int idx, int l, int r, int pos, int val) {
```

```
if (l == r) {  
    seg[idx] = val;  
    return;  
}
```

```
int mid = (l + r) / 2;
```

```
if (pos <= mid) {  
    update(2 * idx + 1, l, mid, pos, val);  
} else {  
    update(2 * idx + 2, mid + 1, r, pos, val);  
}
```

```
seg[idx] = Math.min(seg[2 * idx + 1], seg[2 * idx + 2]);  
}
```

```
static int query(int idx, int l, int r, int ql, int qr) {
```

```
    if (qr < l || r < ql) return Integer.MAX_VALUE;
```

```
    if (ql <= l && r <= qr) return seg[idx];
```

```
    int mid = (l + r) / 2;
```

```
    int left = query(2 * idx + 1, l, mid, ql, qr);
```

```
    int right = query(2 * idx + 2, mid + 1, r, ql, qr);
```

```
    return Math.min(left, right);  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);
```

```
    n = sc.nextInt();
```

```
    int q = sc.nextInt();
```

```
    int[] arr = new int[n];
```

```
    for (int i = 0; i < n; i++) {  
        arr[i] = sc.nextInt();  
    }
```

```
    seg = new int[4 * n];
```

```

    build(arr, 0, 0, n - 1);

    while (q-- > 0) {
        String type = sc.next();

        if (type.equals("Q")) {
            int l = sc.nextInt();
            int r = sc.nextInt();
            System.out.println(query(0, 0, n - 1, l, r));
        } else {
            int idx = sc.nextInt();
            int val = sc.nextInt();
            update(0, 0, n - 1, idx, val);
        }
    }
}
}
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Meera is working on a text processing task where she wants to identify the longest common prefix among a given set of words. She decides to use a Trie structure to efficiently compute this prefix.

Write a program to help Meera insert the given words into a Trie and then determine the longest common prefix among them.

Input Format

The first line contains an integer n , representing the number of words.

The next n lines each contain a single word.

Output Format

The output prints "The longest common prefix is: <prefix>" if a common prefix exists.

Otherwise, print "There is no common prefix"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
aeroplane
aerospace
aerial
aesthetic

Output: The longest common prefix is: ae

Answer

```
import java.util.*;

class TrieNode {
    TrieNode[] child = new TrieNode[26];
    boolean isEnd = false;
}

class Trie {
    TrieNode root = new TrieNode();

    void insert(String word) {
        TrieNode node = root;

        for (char ch : word.toCharArray()) {
            int idx = ch - 'a';

            if (node.child[idx] == null) {
                node.child[idx] = new TrieNode();
            }

            node = node.child[idx];
        }

        node.isEnd = true;
    }

    String longestCommonPrefix() {
```

```

TrieNode node = root;
StringBuilder sb = new StringBuilder();

while (true) {
    int count = 0;
    int index = -1;

    for (int i = 0; i < 26; i++) {
        if (node.child[i] != null) {
            count++;
            index = i;
        }
    }

    // stop if not exactly one child or end of word
    if (count != 1 || node.isEnd) break;

    node = node.child[index];
    sb.append((char) (index + 'a'));
}

return sb.toString();
}
}

```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine();

        Trie trie = new Trie();

        for (int i = 0; i < n; i++) {
            trie.insert(sc.nextLine().trim());
        }

        String ans = trie.longestCommonPrefix();

        if (ans.length() == 0) {
            System.out.print("There is no common prefix");
        }
    }
}

```

```
    } else {  
        System.out.print("The longest common prefix is: " + ans);  
    }  
}  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 3_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Dhiya is a data analyst working for a data compression company. One of her tasks is to analyze logs that are compressed using a simple format where each character is followed by the number of times it appears. For example, "A3B2" represents the string "AAABB".

Her goal is to identify whether a specific pattern exists in the original, uncompressed message and, if so, determine all starting and ending indices of the pattern's occurrence in the expanded string. To perform this efficiently, Dhiya decides to first expand the compressed string and then apply pattern matching.

Your task is to help Dhiya by building a program using Boyer-Moore pattern matching algorithm that performs this operation.

Input Format

The first line of input contains a string *s* consisting of characters followed by digits representing the compressed text.

The second line of input contains a string *p* representing the pattern string.

Output Format

The first line of output prints "Expanded Text: *X*" where *X* represents the expanded text.

The second line of output prints "Matches found (start_index end_index):" if match found.

For each match of the pattern in the expanded text, print the starting and ending indices (0-based) separated by a space (Print one pair per line).

If no match is found, print "No match found."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: A3B2C4D1

CCC

Output: Expanded Text: AAABBCCCCD

Matches found (start_index end_index):

5 7

6 8

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define NO_OF_CHARS 256
```

```
// Function to expand the compressed string
```

```
void expandCompressed(char* s, char* expanded) {
```

```

int i = 0, k = 0;
while (s[i]) {
    char c = s[i++];
    int count = 0;
    while (isdigit(s[i])) {
        count = count * 10 + (s[i] - '0');
        i++;
    }
    for (int j = 0; j < count; j++) {
        expanded[k++] = c;
    }
}
expanded[k] = '\0';
}

// Build Bad Character Table
void badCharHeuristic(char* pat, int m, int badchar[]) {
    for (int i = 0; i < NO_OF_CHARS; i++)
        badchar[i] = -1;
    for (int i = 0; i < m; i++)
        badchar[(unsigned char)pat[i]] = i;
}

```

// Boyer-Moore search using Bad Character Heuristic

```

int BMsearch(char* txt, char* pat, int positions[][2]) {

```

```

    int n = strlen(txt);
    int m = strlen(pat);
    int badchar[NO_OF_CHARS];
    badCharHeuristic(pat, m, badchar);

```

```

    int s = 0; // shift of pattern
    int matchCount = 0;

```

```

    while (s <= n - m) {
        int j = m - 1;
        while (j >= 0 && pat[j] == txt[s + j])
            j--;

```

```

        if (j < 0) {
            // match found
            positions[matchCount][0] = s;
            positions[matchCount][1] = s + m - 1;

```

```

        matchCount++;
        s += (s + m < n) ? m - badchar[(unsigned char)txt[s + m]] : 1;
    } else {
        int shift = j - badchar[(unsigned char)txt[s + j]];
        s += (shift > 1) ? shift : 1;
    }
}
return matchCount;
}

```

```

int main() {
    char compressed[1001], pattern[1001];
    char expanded[10000]; // max expanded length

    scanf("%1000s", compressed);
    scanf("%1000s", pattern);

    expandCompressed(compressed, expanded);

    printf("Expanded Text: %s\n", expanded);

    int positions[10000][2];
    int matches = BMsearch(expanded, pattern, positions);

    if (matches == 0) {
        printf("No match found.\n");
    } else {
        printf("Matches found (start_index end_index):\n");
        for (int i = 0; i < matches; i++) {
            printf("%d %d\n", positions[i][0], positions[i][1]);
        }
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Sanya is developing a text analysis tool and wants to identify the longest

palindromic sequence in a given string. A palindrome is a string that reads the same forwards and backwards.

Your task is to write a program that takes a string as input and returns the longest palindromic substring it contains. If there are multiple substrings of the same maximum length, returning any one of them is acceptable.

Input Format

The input consists of a single line containing the string S.

S may consist of uppercase and lowercase English letters.

Output Format

The output prints a single string representing the longest palindromic substring found in S.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: babad

Output: bab

Answer

```
#include <stdio.h>
#include <string.h>
```

```
void longestPalindrome(char* s, char* result) {
    int n = strlen(s);
    if (n == 0) {
        result[0] = '\0';
        return;
    }
}
```

```
int start = 0, maxLen = 1;
```

```
for (int i = 0; i < n; i++) {
    // Odd length palindrome
    int l = i, r = i;
```

```

while (l >= 0 && r < n && s[l] == s[r]) {
    if (r - l + 1 > maxLen) {
        start = l;
        maxLen = r - l + 1;
    }
    l--;
    r++;
}

// Even length palindrome
l = i; r = i + 1;
while (l >= 0 && r < n && s[l] == s[r]) {
    if (r - l + 1 > maxLen) {
        start = l;
        maxLen = r - l + 1;
    }
    l--;
    r++;
}

strncpy(result, s + start, maxLen);
result[maxLen] = '\0';
}

int main() {
    char s[101], result[101];
    scanf("%100s", s);

    longestPalindrome(s, result);
    printf("%s\n", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rita is working on string pattern matching using the Z-Algorithm. She wants to find all starting indices where a given pattern string occurs in a

text string efficiently.

Write a program that takes a text string and a pattern string as input and outputs a list of indices (0-based) where the pattern occurs. If the pattern does not occur in the text, output an empty list.

Input Format

The first line contains the text string txt.

The second line contains the pattern string pat.

Output Format

The output prints a list of integers representing the starting indices of all occurrences of the pattern in the text.

If no match is found, print [].

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abesdu
edu

Output: []

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 1100
```

```
// Compute Z-array for a string
void computeZArray(char* s, int Z[]) {
    int n = strlen(s);
    int L = 0, R = 0;
    Z[0] = 0;
    for (int i = 1; i < n; i++) {
        if (i > R) {
```

```

    L = R = i;
    while (R < n && s[R - L] == s[R])
        R++;
    Z[i] = R - L;
    R--;
} else {
    int k = i - L;
    if (Z[k] < R - i + 1)
        Z[i] = Z[k];
    else {
        L = i;
        while (R < n && s[R - L] == s[R])
            R++;
        Z[i] = R - L;
        R--;
    }
}
}
}
}

```

```

int main() {
    char txt[MAX], pat[101];
    scanf("%s", txt);
    scanf("%s", pat);

    int n = strlen(txt);
    int m = strlen(pat);

    char concat[MAX + 102]; // pat + $ + txt
    sprintf(concat, "%s%s", pat, txt);

    int Z[MAX + 102];
    computeZArray(concat, Z);

    int first = 1;
    printf("[");
    for (int i = m + 1; i < strlen(concat); i++) {
        if (Z[i] == m) {
            if (!first) printf(", ");
            printf("%d", i - m - 1);
            first = 0;
        }
    }
}

```

```
}  
printf("]\n");  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Neha is building a plagiarism-detection feature where she must compare two text strings to find similarities. To efficiently detect overlaps between the strings, she uses the KMP (Knuth–Morris–Pratt) string matching algorithm, relying on the LPS (Longest Prefix Suffix) array as the main data structure.

Given two strings, write a program to ensure that the task is done by finding the longest common substring present in both strings. If no common substring exists, the program should clearly indicate that result.

Input Format

The first line contains a string representing the first text.

The second line contains a string representing the second text.

Output Format

The output prints the longest common substring if it exists.

If there is no common substring, print "There is no common substring".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abcdef

abc

Output: abc

Answer

```
#include <stdio.h>
#include <string.h>
```

```
void computeLPS(char *pat, int m, int *lps) {
    int len = 0;
    lps[0] = 0;
    int i = 1;

    while (i < m) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0)
                len = lps[len - 1];
            else {
                lps[i] = 0;
                i++;
            }
        }
    }
}
```

```
// KMP search: returns 1 if pat exists in txt, 0 otherwise
int KMPSearch(char *txt, char *pat) {
    int n = strlen(txt);
    int m = strlen(pat);
    int lps[m];
    computeLPS(pat, m, lps);
```

```
    int i = 0, j = 0;
    while (i < n) {
        if (pat[j] == txt[i]) {
            i++;
            j++;
        }
        if (j == m) {
            return 1; // match found
        } else if (i < n && pat[j] != txt[i]) {
            if (j != 0)
```

```

        j = lps[j - 1];
    else
        i++;
    }
}
return 0;
}

```

```

int main() {
    char s1[101], s2[101];
    scanf("%100s", s1);
    scanf("%100s", s2);

```

```

    char *longest = NULL;
    int maxLen = 0;

```

```

    int len1 = strlen(s1);
    int len2 = strlen(s2);

```

```

    // choose smaller string to generate substrings
    char *smaller = (len1 <= len2) ? s1 : s2;
    char *larger = (len1 <= len2) ? s2 : s1;
    int smallLen = strlen(smaller);

```

```

    for (int l = smallLen; l > 0; l--) { // substring length
        for (int i = 0; i <= smallLen - l; i++) {
            char substr[101];
            strncpy(substr, smaller + i, l);
            substr[l] = '\0';
            if (KMPSearch(larger, substr)) {
                printf("%s\n", substr);
                return 0; // longest found
            }
        }
    }
}

```

```

    printf("There is no common substring\n");
    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOTAGIRI AKSHAYA NAIDU

Email: vtu28432@veltech.edu.in

Roll no: vtu28432

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Competitive Coding II_L3

VTU_Week 4_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : MCQ

1. The user performs the following operations on a stack of size 5 then, which of the following is the correct statement for the 1 stack?

```
push(1);  
pop();  
push(2);  
push(3);  
pop();  
push(2);  
pop();  
pop();  
push(4);  
pop();  
pop();  
push(5);
```

Answer

Underflow occurs

Status : Correct

Marks : 1/1

2. If the sequence of operations - push (1), push (2), pop, push (1), push (2), pop, pop, pop, push (2), pop are performed on a stack, the sequence of popped out values

Answer

2,2,1,1,2

Status : Correct

Marks : 1/1

3. What is the result of the following operation?

Top (Push (S, X))

Answer

X

Status : Correct

Marks : 1/1

4. Stack A has the entries a, b, c (with a on top). Stack B is empty. An entry popped out of stack A can be printed immediately or pushed to stack B. An entry popped out of the stack B can only be printed. In this arrangement, which of the following permutations of a, b, c are not possible?

Answer

c a b

Status : Correct

Marks : 1/1

5. Pushing an element into the stack already has five elements. The stack size is 5, then the stack becomes

Answer

Overflow

Status : Correct

Marks : 1/1

6. In a Stack, if a user tries to remove an Element from Empty stack it is called

Answer

Underflow

Status : Correct

Marks : 1/1

7. The user performs the following operations on the stack of size 5 then at the end of the last operation, the total number of elements present in the stack is

```
push(1);  
pop();  
push(2);  
push(3);  
pop();  
push(4);  
pop();  
pop();  
push(5);
```

Answer

1

Status : Correct

Marks : 1/1

8. What will be the output after performing these sequence of operations

```
push(20);  
push(4);  
top();  
pop();
```

```
pop();  
pop();  
push(5);  
top();
```

Answer

Stack Underflow

Status : Correct

Marks : 1/1

9. A normal queue, if implemented using an array of size MAX_SIZE, gets full when _____.

Answer

rear = MAX_SIZE - 1

Status : Correct

Marks : 1/1

10. If the elements 'A', 'B', 'C', and 'D' are placed in a queue and are deleted one at a time, in what order will they be removed?

Answer

ABCD

Status : Correct

Marks : 1/1

11. After performing this set of operations, what does the final queue contain?

```
InsertFront(10);  
InsertFront(20);  
InsertRear(30);  
DeleteFront();  
InsertRear(40);  
InsertRear(10);  
DeleteRear();  
InsertRear(15);  
display();
```

Answer

10 30 40 15

Status : Correct

Marks : 1/1

12. What is the purpose of defining the constant SIZE in a queue implementation using an array?

Answer

To limit the maximum number of elements in the queue

Status : Correct

Marks : 1/1

13. What does the condition front = NULL indicate in inserting an element into a queue?

Answer

The queue is empty

Status : Correct

Marks : 1/1

14. Which of the following is a valid condition for a successful dequeue operation in the array implementation of a queue?

Answer

None of the mentioned options

Status : Correct

Marks : 1/1

15. The essential condition that is checked before insertion in a queue is?

Answer

Overflow

Status : Correct

Marks : 1/1